

帶補間功能的 4 軸馬達控制 LSI

MCD451

使用說明書（補間機能編）

LSI

目 录

1. 概要	1
2. 功能電路圖	1
3. 補間驅動的種類	3
3-1 直線補間 (Straight Drive)	3
3-2 Endless直線補間 (Endless Staight Drive)	6
3-3 順時針圓弧補間 (Right Circular Drive)	9
3-4 逆時針圓弧補間 (Left Circular Drive)	11
4. 其他功能	13
4-1 圓周速補償功能	13
4-2 連續補間	13
4-3 補間用的脈衝發生器自動停止功能	14
4-4 軟件界限越界判定功能	14
4-5 物理界限越界判定功能	14
4-6 馬達反饋的狀態判定功能	14
5. 8Bit數據總線時補間回路的I/O端口映射	15
5-1 IP Data1 Write Port	15
5-2 IP Data2 Write Port	15
5-3 IP Data3 Write Port	15
5-4 IP Data4 Write Port	15
5-5 IP Command Write Port	15
5-6 IP Mode-A Write Port	16
5-7 IP Mode-B Write Port	17
5-8 IP Mode-C Write Port	17
5-9 IP Data1 Read Port	18
5-10 IP Data2 Read Port	18
5-11 IP Data3 Read Port	18
5-12 IP Data4 Read Port	18
5-13 IP Status-1 Read Port	18

5-14 IP Status-2 Read Port	19
5-15 IP Status-3 Read Port	20
5-16 IP Mode-A Read Port	21
5-17 IP Mode-B Read Port	21
5-18 IP Mode-C Read Port	21
6. 16Bit數據總線時補間回路的I/O端口映射	22
6-1 IP Data-H Write Port	22
6-2 IP Data-L Write Port	22
6-3 IP Command Write Port	22
6-4 IP Mode-A Write Port	22
6-5 IP Mode-B Write Port	23
6-6 IP Mode-C Write Port	24
6-7 IP Data-H Read Port	25
6-8 IP Data-L Read Port	25
6-9 IP Status-A Read Port	25
6-10 IP Status-B Read Port	26
6-11 IP Mode-A Read Port	27
6-12 IP Mode-B Read Port	28
6-13 IP Mode-C Read Port	28
7. 補間運算回路的命令	29
7-1 補間回路命令表	29
7-2 在不同寬度的數據總線中存取方法的差異	31
7-3 補間運算回路命令詳解	32
8. 補間回路掃執行命令的步驟	38
8-1 向芯片中寫入4 Bytes數據的命令的執行步驟	38
8-2 從芯片中讀取4 Bytes數據的命令的執行步驟	39
9. 補間運算回路編程實例	40
9-1 初始設定條件	40
9-2 動作例1	40

9-3 動作例2	46
9-4 動作例3	47
9-5 動作例4	48
10. (驅動所需的)脈衝數的算出方法.....	50
10-1 直線補間驅動的脈衝數.....	50
10-2 圓弧補間驅動的脈衝數(無圓周速補償時)	52
10-3 圓弧補間驅動的脈衝數(有圓周速補償時)	55
11. 連續補間時的注意事項.....	57
11-1 連續補間驅動的執行步驟.....	57
11-2 驅動類型引起的限制.....	57
11-3 丟失的驅動脈衝.....	58
12. 補間驅動時的注意事項.....	59
12-1 Range Data的設定範圍的限制.....	59
12-2 關於直線補間時目標位置的設定.....	59
12-3 關於直線補間時的分母值.....	59
12-4 關於圓弧補間時的開始位置的設定.....	59
12-5 關於圓弧補間時的目標位置的設定.....	60
12-6 關於360°圓弧補間(也就是畫一個圓)	60
12-7 關於圓補間時,指定的目標位置無法檢測到的情形.....	60
12-8 關於當前點坐標溢出時的情形.....	61
12-9 關於圓補間時目標位置偏移(有誤差)時的處理.....	62
13. 補充說明.....	63
13-1 標準輸入時鐘脈衝的選擇.....	63
13-2 收到RESET信號時的芯片初始化.....	63
13-3 5軸以上的補間操作.....	65
13-4 關於不進行加減速操作的補間驅動.....	65
14. 輸入輸出信號的時序.....	66
14-1 同步信號的輸出時序.....	66
14-2 實動作時序.....	68

本手冊於2009年3月9日更新。今天可能不預先通知，對產品進行改良，從而變更本手冊的內容。

■ 本LSI為RoHS適用品

■ 未經本公司同意，不可轉載

■ 本公司盡最大努力改善產品品質、提高產品的可信度。但半導體產品總是存在一定可能，在某種情況下失效（產生某些故障）。

為防止本產品失效，而導致人身事故、火災事故，或造成社會財產損失，應當進行必要的冗餘設計、阻燃設計、誤動作預防設計等，以保證安全。

■ 本芯片的引腳未使用時，為防止靜電損壞，應當明確接電源或接地。

■ 本LSI，並不在日本國貿易管理法輸出管制列表中。但是，搭載有本產品的裝置整體是否屬於以上列表，還需顧客自行判斷。

1. 概要

本手冊為MCD451補間回路的功能說明。MCD451的規格說明、引腳配置、外形圖、引腳功能說明，PG回路的I/O端口說明、獨立軸驅動點的說明，請詳見《MCD451產品說明書（基本功能編）》。

2. 功能電路圖

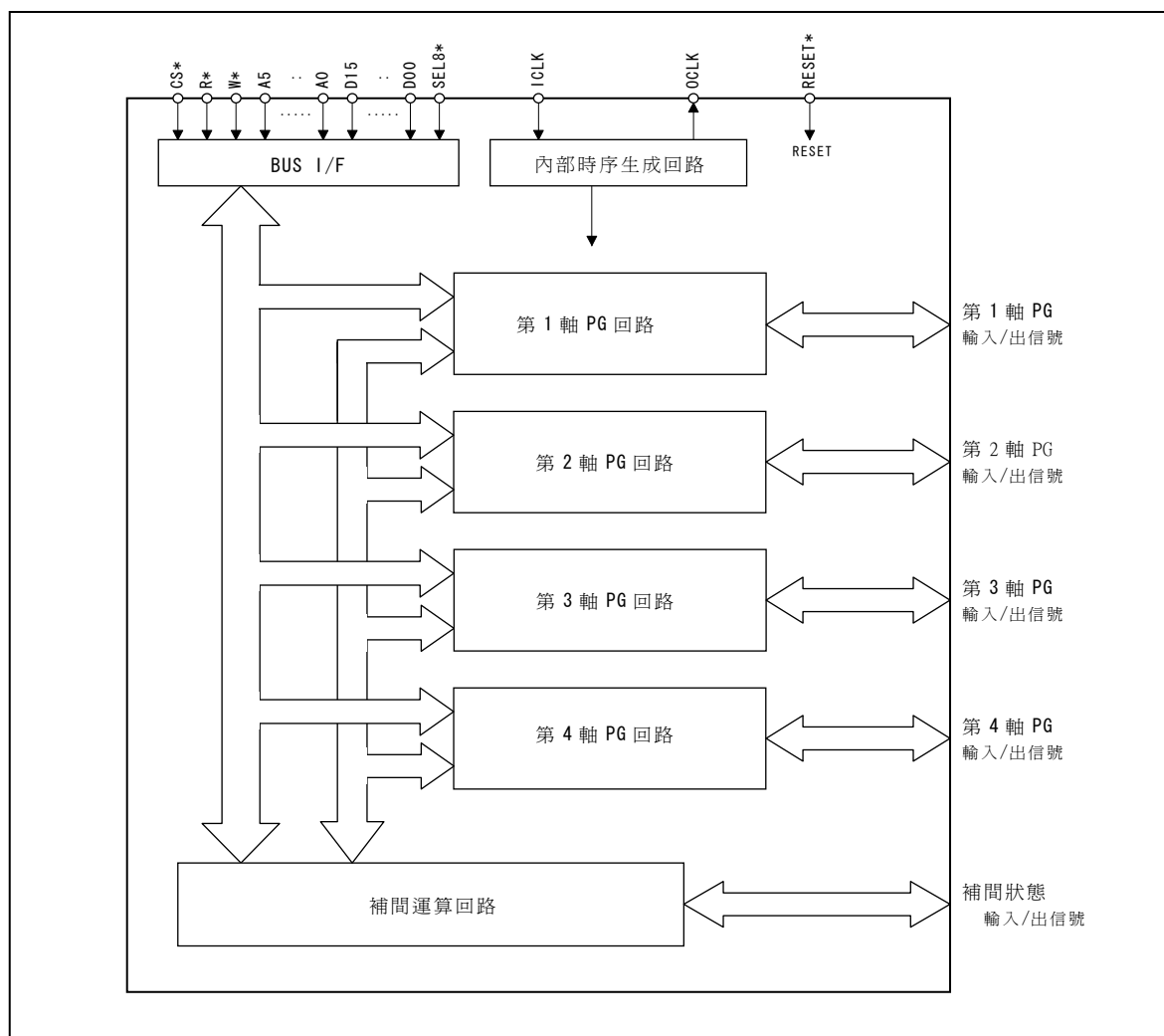
MCD451擁有4個獨立的馬達驅動PG回路（相對補間而言，稱為獨立軸），可同時進行相互獨立的單軸驅動。MCD451可進行4軸間的補間驅動操作，即最多可達4軸的直線補間，及2軸的圓弧補間。

補間操作需要某個軸給出速度脈衝（稱為脈衝發生器），補間以此決定當前的速度頻率。MCD451即允許使用自身的軸，也允許通過MCB9~MCB0電纜互聯的其他MCD451兼容芯片的其他軸作為脈衝發生器。

MCD451中的每個軸，都可獨立指定為“獨立模式”/“補間模式”。補間回路輸出的脈衝，只對指定為“補間模式”的軸有效。

指定為“獨立模式”的軸，只能進行獨立軸驅動，而和補間回路輸出的脈衝無任何關係。因此，可以同時進行補間驅動和獨立軸驅動。

内部电路图



如上圖所示，補間回路產生的補間脈衝，必須經過PG回路才能輸出。故不管是在補間驅動還是在獨立軸驅動時，即時位置管理、驅動脈衝輸出方式等，都是在PG回路中指定。

此外，被指定為“補間模式”的各軸的狀態（物理界限越界信號、反饋的馬達狀態、軟件界限越界狀態等），集中到補間回路中統一管理，並會被反映到做為補間脈衝發生器的軸的狀態寄存器中。

也即，做為脈衝發生器的軸的狀態寄存器，反映了所有參與補間的軸的狀態。

另外，在MCD451中，脈衝發生器產生的脈衝，可以來自其他MCD451芯片，也可以輸出到其它MCD451芯片。因此可以進行4xn軸間的補間操作。

此時，被指定為“補間模式”的各軸的狀態（物理界限越界信號、反饋的馬達狀態、軟件界限越界狀態等）信息，將會通過MCB9~MCB0總線，發送給互聯的其它軸。

在芯片未進行補間運算時，可以隨時變更，任意組合。

往MCD451芯片的補間運算回路中寫入補間驅動命令後，BUSY_IP即變為高電平，芯片開始補間運算。此外，根據脈衝發生器產生的脈衝，計算並輸出補間驅動脈衝。到達預定的位置後停止補間運算，並使BUSY_IP回到低電平。亦即，MCD451的補間回路的BUSY_IP為高電平，意味著“芯片正在進行補間運算”，也即意味著“補間驅動執行中”。在本手冊中，此三者意義相同。

.BUSY_IP為高電平 $\longleftrightarrow$ 芯片正在進行補間運算 $\longleftrightarrow$ 補間驅動執行中

.BUSY_IP為低電平 $\longleftrightarrow$ 補間運算停止中 $\longleftrightarrow$ 補間驅動停止中

.BUSY_IP從低電平變為高電平 $\longleftrightarrow$ 補間運算開始 $\longleftrightarrow$ 補間驅動開始

.BUSY_IP從高電平變為低電平 $\longleftrightarrow$ 補間運算結束 $\longleftrightarrow$ 補間驅動結束

本手冊根據正下文分別使用以上語句。

3. 補間驅動的種類

MCD451可以進行直線補間（Straight Drive）、Endless直線補間（Endless Straight Drive）、順時針圓弧補間（Right Circular Drive）、逆時針圓弧補間（Left Circular Drive）。以下分別介紹此4種補間。

3-1 直線補間（Straight Drive）

單個MCD451芯片，可最多進行4軸間的直線補間驅動。

執行直線補間驅動時，將認為當前位置坐標為（0，0，0，0）[譯注：4個軸的情況]，並指定目標位置的坐標（N1,N2,N3,N4）[或稱為N1 Obj，諸如此例]。驅動開始前應當在程序中分別指定N1,N2,N3,N4。

直線補間驅動使用其同的分母D進行驅動運算，使用Denominator Pre-Buffer寄存器可設定該分母值。

進行以上設定後，各軸輸出的補間脈衝如下（根據脈衝發生器算出）：

第1軸的補間驅動脈衝＝脈衝發生器的當前輸出頻率×|N1|÷D

第2軸的補間驅動脈衝＝脈衝發生器的當前輸出頻率×|N2|÷D

第3軸的補間驅動脈衝＝脈衝發生器的當前輸出頻率×|N3|÷D

第4軸的補間驅動脈衝＝脈衝發生器的當前輸出頻率×|N4|÷D

注1：

. 不輸出補間脈衝的軸，及指定為“獨立模式”的軸，目標位置設定為0

. 分母D應當滿足以下條件

$D \geq |N1|、|N2|、|N3|、|N4|$ 的最大值

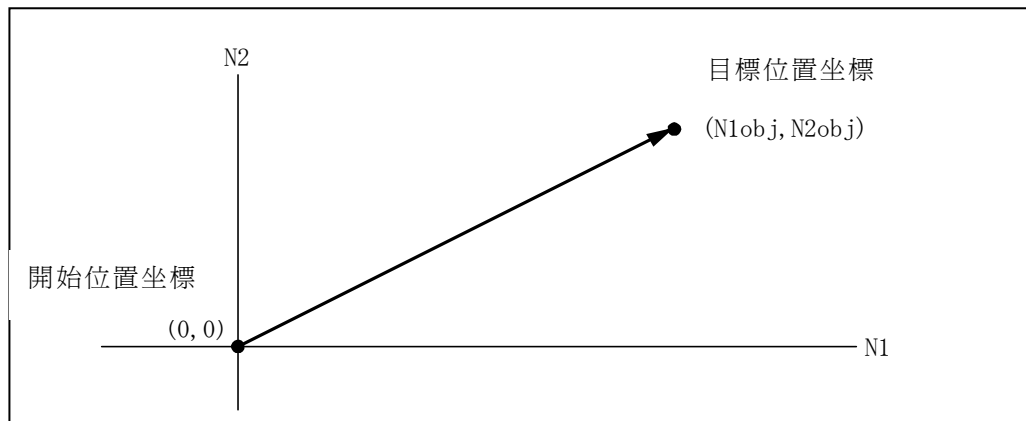
. 要使補間各軸的合成速度最大，可設定D如下

$D = |N1|、|N2|、|N3|、|N4|$ 的最大值

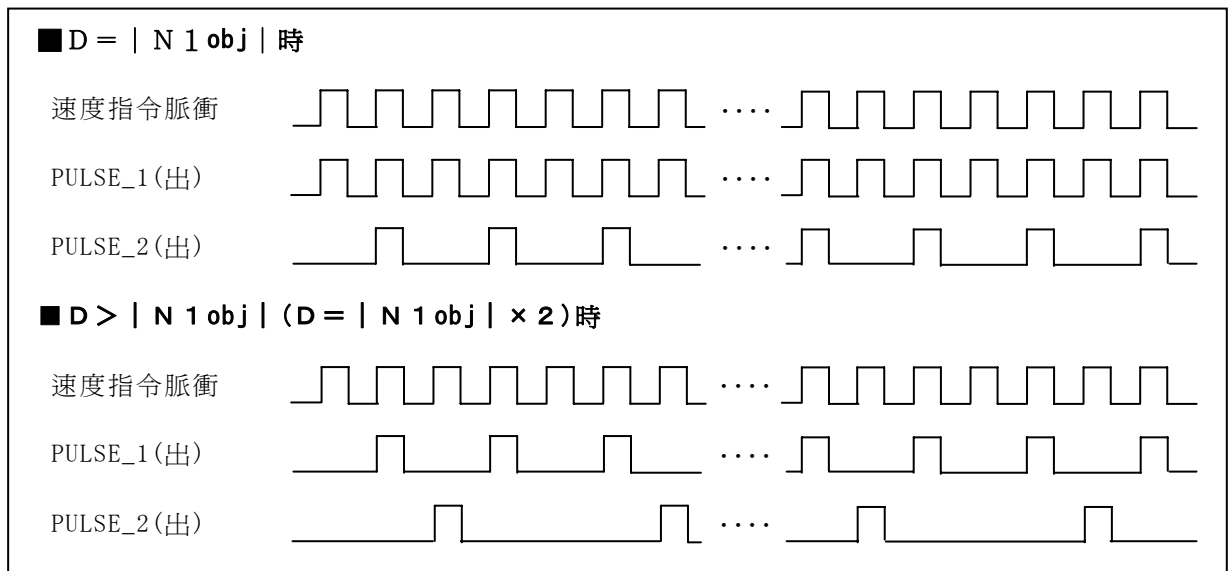
. 不管補間方向如何，合成速度一定的情況下，分母D應滿足以下式子（僅適於2軸或3軸正交坐標）

$$D = \sqrt{N1\text{ obj}^2 + N2\text{ obj}^2 + N3\text{ obj}^2 + N4\text{ obj}^2}$$

■直線補間示例圖



該例為第 1 軸和第 2 軸進行 2 軸間補間的情形 ($N3=N4=0$)。脈衝發生器的頻率和補間脈衝頻率的關係如下圖所示：



- 本例中 $N1>0$ ， $N2>0$ 。因此執行的驅動是平面坐標的第1象限的補間驅動。依設定值的符號組合，可進行第2、3、4象限的補間驅動；
- 本例為單脈衝方式（有效電平為高電平[譯注：是指電平和脈衝發生器的電平相位相同。反之，日語中的有效電平為低電平，有修飾脈衝時，有時指反相脈衝]）的情形；
- 脈衝發生器產生的脈衝的當前頻率，和補間運算輸出的當前脈衝頻率關係如下：
脈衝發生器產生的脈衝的當前頻率： $\text{Pulse}_1(\text{輸出}) : \text{Pulse}_2(\text{輸出}) = D : |N1| : |N2|$
- 補間運算和指軸的“獨立模式”、“補間模式”的指定值間沒有直接關係，但補間回路，只對設定為“補間模式”的軸輸出補間脈衝。

直線補間驅動開始後，以下值被裝載到補間回路的相關寄存器中。以後每接收到一個脈衝發生

器送來的脈衝，Denominator Counter將加1[本寄存器和Denominator寄存器為不同的寄存器，譯注]，各個軸每輸出一個補間脈衝，對應軸的N1/N2/N3/N4 Now Point Register將增1或減1。

Denominator Pre-Buffer→Denominator Register（輸出比的分母值）

0（00000000H）→Denominator Counter（分母計數器）

N1 Object Point Pre-Buffer→N1 Object Point Register（第1軸的目標坐標寄存器）

N2 Object Point Pre-Buffer→N2 Object Point Register（第2軸的目標坐標寄存器）

N3 Object Point Pre-Buffer→N3 Object Point Register（第3軸的目標坐標寄存器）

N4 Object Point Pre-Buffer→N4 Object Point Register（第4軸的目標坐標寄存器）

以上各寄存器中，Denominator Pre-Buffer、Denominator Register、Denominator Counter為32Bit 二進制數形式。其它為二進制數補碼形式。

當以下條件成立時，芯片認為直線補間操作完成，BUSY_IP高為低電平。

.Denominator Counter=Denominator Register

.N1 Now Point Register=N1 Object Point Register

.N2 Now Point Register=N2 Object Point Register

.N3 Now Point Register=N3 Object Point Register

.N4 Now Point Register=N4 Object Point Register

3-2 Endless直線補間 (Endless Staight Drive)

單個MCD451芯片，可最多進行4軸間的Endless直線補間驅動。

Endless直線補間和3-1中所述的 Straight Drive補間驅動相同，只是補間操作可以越過設定的目標點坐標，成爲一條射線。直到接收到Interoperation Stop命令，Endless直線補間過程不會停止[譯注：當然是指沒有遇到物理界限或軟體界限的情況]。

執行Endless直線補間驅動時，將認爲當前位置坐標爲(0, 0, 0, 0) [譯注：4個軸的情況]，並指定目標位置的坐標(N1, N2, N3, N4) [或稱爲N1 Obj, 諸如此例]。驅動開始前應當在程序中分別指定N1, N2, N3, N4。

Endless直線補間驅動使用其同的分母D進行驅動運算，使用Denominator Pre-Buffer寄存器可設定該分母值。

Endless直線補間沒有終點。故所指定的“終點”只是用來確定各補間脈衝的頻率值的比[譯注：在坐標上表現爲直線的斜率]。

進行以上設定後，各軸輸出的補間脈衝如下（根據脈衝發生器算出）：

第1軸的補間驅動脈衝＝脈衝發生器的當前輸出頻率×|N1|÷D

第2軸的補間驅動脈衝＝脈衝發生器的當前輸出頻率×|N2|÷D

第3軸的補間驅動脈衝＝脈衝發生器的當前輸出頻率×|N3|÷D

第4軸的補間驅動脈衝＝脈衝發生器的當前輸出頻率×|N4|÷D

注2：

. 不輸出補間脈衝的軸，及指定爲“獨立模式”的軸，目標位置設定爲0

. 分母D應當滿足以下條件

$D \geq |N1|、|N2|、|N3|、|N4|$ 的最大值

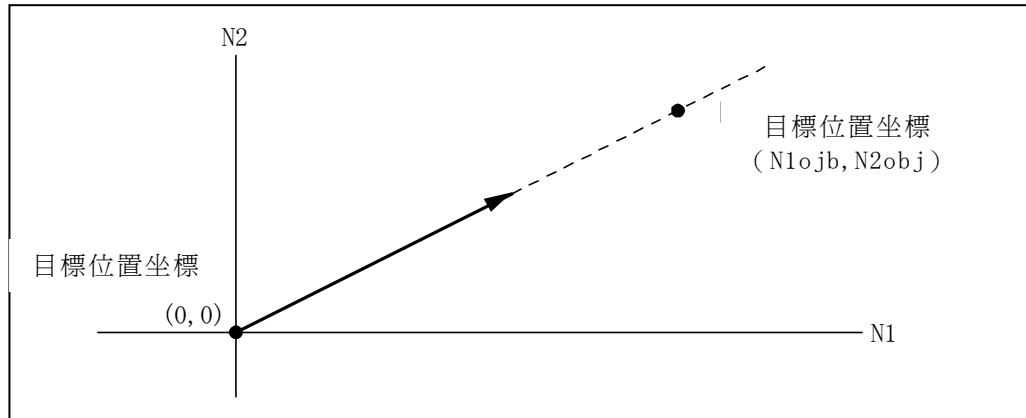
. 要使補間各軸的合成速度最大，可設定D如下

$D = |N1|、|N2|、|N3|、|N4|$ 的最大值

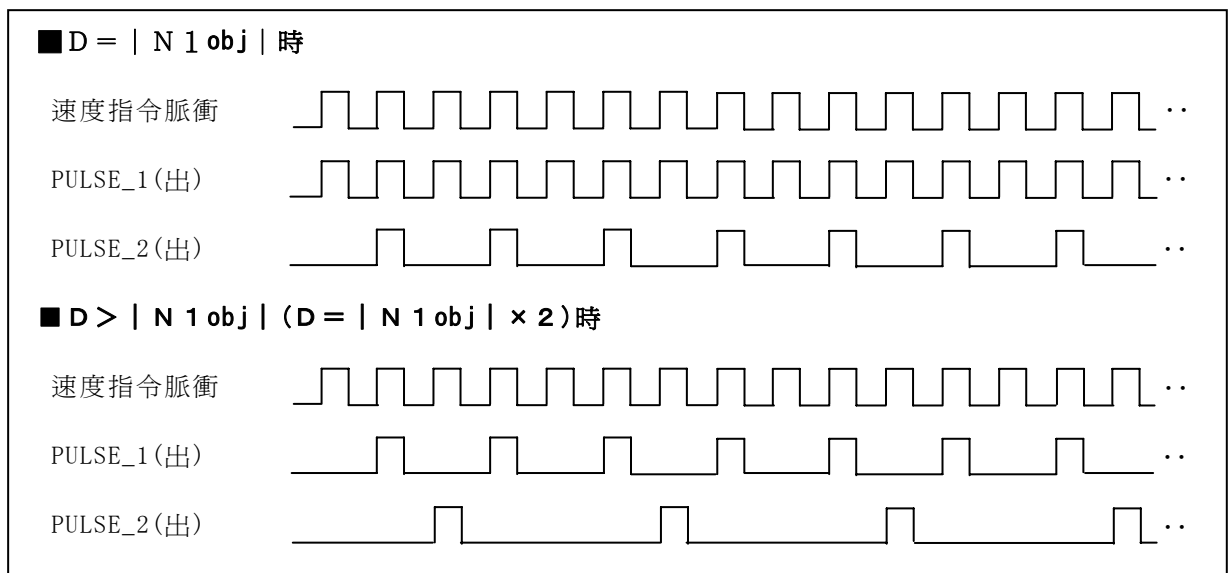
. 不管補間方向如何，全成速度一定的情況下，分母D應滿足以下式子（僅適於2軸或3軸正交坐標）

$$D = \sqrt{N1\text{ obj}^2 + N2\text{ obj}^2 + N3\text{ obj}^2 + N4\text{ obj}^2}$$

■ Endless 直線補間示例圖



該例為第 1 軸和第 2 軸進行 2 軸間補間的情形（ $N3=N4=0$ ）。脈衝發生器的頻率和補間脈衝頻率的關係如下圖所示：



. 本例中 $N1>0$ ， $N2>0$ 。因此執行的驅動是平面坐標的第1象限的補間驅動。依設定值的符號組合，可進行第2、3、4象限的補間驅動；

. 本例為單脈衝方式（有效電平為高電平[譯注：是指電平和脈衝發生器的電平相位相同。反之，日語中的有效電平為低電平，有修飾脈衝時，有時指反相脈衝]）的情形；

. 脈衝發生器產生的脈衝的當前頻率，和補間運算輸出的當前脈衝頻率關係如下：

脈衝發生器產生的脈衝的當前頻率： $Pulse_1（輸出）：Pulse_2（輸出）=D：|N1|：|N2|$

. 補間運算和指軸的“獨立模式”、“補間模式”的指定值間沒有直接關係，但補間回路，只對設定為“補間模式”的軸輸出補間脈衝。

直線補間驅動開始後，以下值被裝載到補間回路的相關寄存器中。以後每接收到一個脈衝發生器送來的脈衝，Denominator Counter將加1[本寄存器和Denominator寄存器為不同的寄存器，譯注]，各個

軸每輸出一個補間脈衝，對應軸的N1/N2/N3/N4 Now Point Register將增1或減1。

Denominator Pre-Buffer→Denominator Register（輸出比的分母值）

0（00000000H）→Denominator Counter（分母計數器）

N1 Object Point Pre-Buffer→N1 Object Point Register（第1軸的目標坐標寄存器）

N2 Object Point Pre-Buffer→N2 Object Point Register（第2軸的目標坐標寄存器）

N3 Object Point Pre-Buffer→N3 Object Point Register（第3軸的目標坐標寄存器）

N4 Object Point Pre-Buffer→N4 Object Point Register（第4軸的目標坐標寄存器）

以上各寄存器中，Denominator Pre-Buffer、Denominator Register、Denominator Counter為32Bit 二進制數形式。其它為二進制數補碼形式。

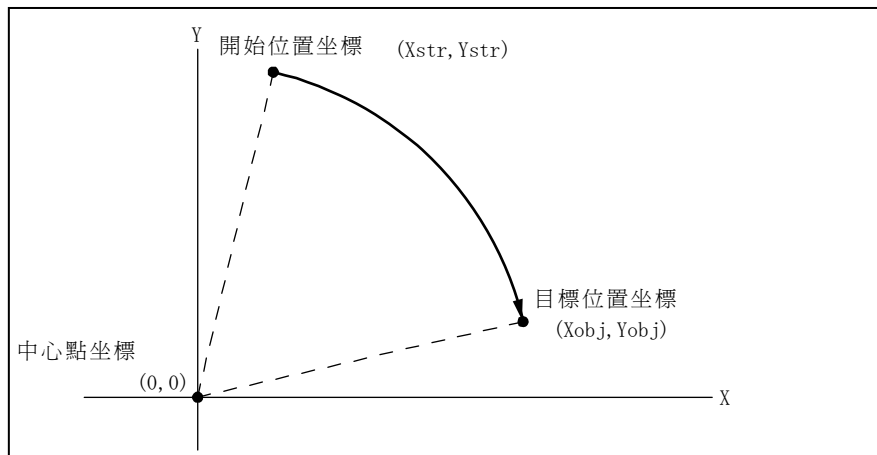
Endless直線補間沒有終點，也就不會停止。要想停止Endless直線補間（使得BUSY_IP回到低電平），須向芯片（補間回路）發送Interoperation Stop命令。

3-3 順時針圓弧補間 (Right Circular Drive)

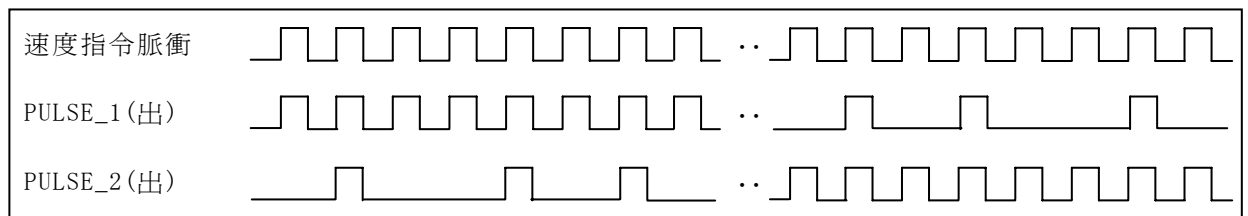
可以將本MCD451芯片中的4個軸任意組合指定為X或Y軸。並基於此設定進行順時針圓弧補間驅動。

執行順時針圓弧補間時，所畫的圓的圓心坐標被假設為坐標原點（0，0），並基於此假設，在程序中應先指定圓弧開始點坐標（Xstr,Ystr），及圓弧終點坐標（Xobj,Yobj）。X/Y坐標可以通過X/Y Start Point Pre-Buffer和X/Y Object Point Pre-Buffer寄存器指定。

■ 順時針圓弧補間的例



此例中，脈衝發生器的脈衝頻率和補間脈衝的頻率關係，如下所示：



. 本例中 $N1 > 0$ ， $N2 > 0$ 。因此執行的驅動是平面坐標的第1象限的補間驅動。依設定值的符號組合，可進行第2、3、4象限的補間驅動；

. 本例為單脈衝方式（有效電平為高電平[譯注：是指電平和脈衝發生器的電平相位相同。反之，日語中的有效電平為低電平，有修飾脈衝時，有時指反相脈衝]）的情形；

. 補間運算和指軸的“獨立模式”、“補間模式”的指定值間沒有直接關係，但補間回路，只對設定為“補間模式”的軸輸出補間脈衝。

順時針補間驅動開始後，以下值被裝載到補間回路的相關寄存器中。以後各個軸每輸出一個補間脈衝，對應軸的X/Y Now Point Register將增1或減1。

X Object Point Pre-Buffer → X Object Point Register

X Start Point Pre-Buffer → X Start Point Register

X Start Point Pre-Buffer → X Now Point Register

Y Object Point Pre-Buffer→Y Object Point Register

Y Start Point Pre-Buffer→Y Start Point Pre-Buffer

Y Start Point Pre-Buffer→Y Now Point Register

以上各寄存器中的值都為2進制補碼形式。順時針圓弧補間在以下兩個條件同時成立時，認為補間操作完成，BUSY_IP回到低電平：

.X now Point Register=X Object Point Register

.Y Now Point Register=Y Object Point Register

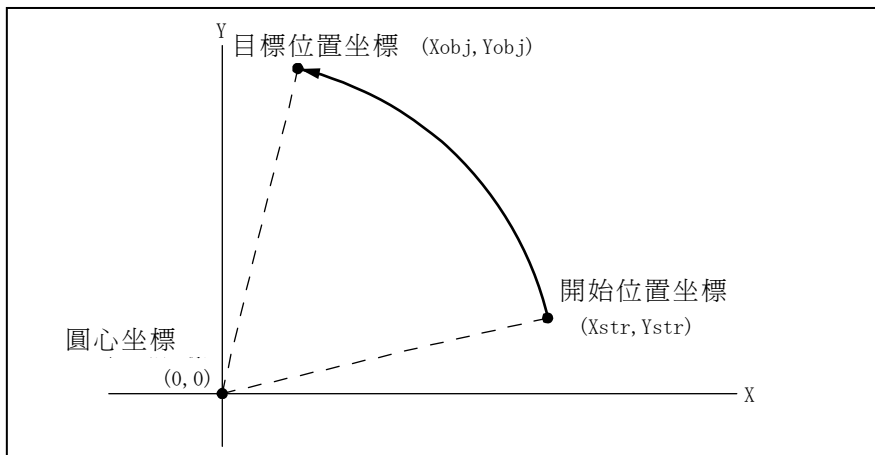
但是，進行360°圓弧補間時〔也即圓周〕，因為“開始點坐標”等於“終點坐標”，補間操作剛啟動，以上條件即成立。此時，MCD451會將其自動識別為360°圓周補間，等畫完一個圓時，再行判斷終止條件。

3-4 逆時針圓弧補間 (Left Circular Drive)

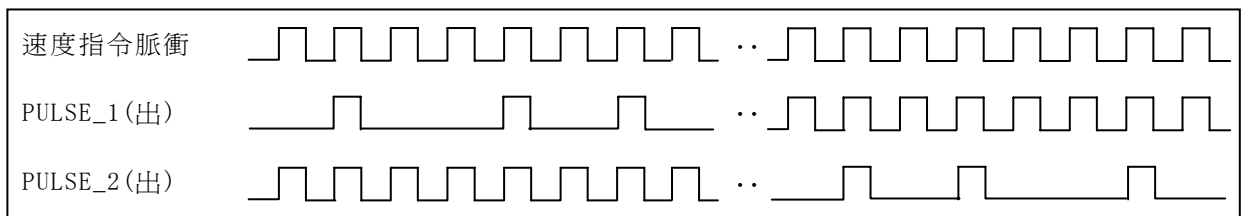
可以將本MCD451芯片中的4個軸任意組合指定為X或Y軸。並基於此設定進行逆時針圓弧補間驅動。

執行逆時針圓弧補間時，所畫的圓的圓心坐標被假設為坐標原點（0，0），並基於此假設，在程序中應先指定圓弧開始點坐標（Xstr,Ystr），及圓弧終點坐標（Xobj,Yobj）。X/Y坐標可以通過X/Y Start Point Pre-Buffer和X/Y Object Point Pre-Buffer寄存器指定。

■逆時針圓弧補間的例



此例中，脈衝發生器的脈衝頻率和補間脈衝的頻率關係，如下所示：



. 本例中 $N1 > 0$ ， $N2 > 0$ 。因此執行的驅動是平面坐標的第1象限的補間驅動。依設定值的符號組合，可進行第2、3、4象限的補間驅動；

. 本例為單脈衝方式（有效電平為高電平[譯注：是指電平和脈衝發生器的電平相位相同。反之，日語中的有效電平為低電平，有修飾脈衝時，有時指反相脈衝]）的情形；

. 補間運算和指軸的“獨立模式”、“補間模式”的指定值間沒有直接關係，但補間回路，只對設定為“補間模式”的軸輸出補間脈衝。

逆時針補間驅動開始後，以下值被裝載到補間回路的相關寄存器中。以後各個軸每輸出一個補間脈衝，對應軸的X/Y Now Point Register將增1或減1。

X Object Point Pre-Buffer → X Object Point Register

X Start Point Pre-Buffer → X Start Point Register

X Start Point Pre-Buffer → X Now Point Register

Y Object Point Pre-Buffer→Y Object Point Register

Y Start Point Pre-Buffer→Y Start Point Pre-Buffer

Y Start Point Pre-Buffer→Y Now Point Register

以上各寄存器中的值都為2進制補碼形式。逆時針圓弧補間在以下兩個條件同時成立時，認為補間操作完成，BUSY_IP回到低電平：

.X now Point Register=X Object Point Register

.Y Now Point Register=Y Object Point Register

但是，進行360°圓弧補間時〔也即圓周〕，因為“開始點坐標”等於“終點坐標”，補間操作剛啟動，以上條件即成立。此時，MCD451會將其自動識別為360°圓周補間，等畫完一個圓時，再行判斷終止條件。

4. 其他功能

4-1 圓周速補償功能

相對於獨立軸驅動而言，多軸補間時，合成速度最大可為單軸的 $\sqrt{2}$ （2軸補間時）、 $\sqrt{3}$ （3軸補間時）倍。

如果僅僅要求軌跡正確，這不會有什麼問題。但實際情況下，可以對加工的速度有一定的要求。

在進行直線補間或Endless直線補間時，可以設定MCD451的分母D寄存器，任意地調節合成速度。

在圓弧補間中，2軸同時輸出驅動脈衝時，MCD451能夠自己丟棄2/7的脈衝發生器脈衝[譯注：在某段圓弧上]，將速度降為原來的5/7。也即周速補正功能。

這樣[譯注：圓弧補間時]，就可以將最後的合成速度控制在單軸驅動速度的0.93~1.01間（注3）。

注3：

將和X軸或Y軸垂直時的合成速度做為1[譯注：此時，和單軸驅動的速度比相同，也為1]，中心點和即時位置所在的點組成的線段和X軸或Y軸成21.8°時合成速度比最小為0.93，成45°時最小，為1.01。

4-2 連續補間

MCD451可以進行連續補間，如：直線補間→圓弧補間→...→直線補間...。

在當前補間運算尚未完成時，往芯片中寫入下一個補間命令，即被認為是連續補間請求。只要遵守此規則，即可無限地發出連續補間請求。（參見11章11-1小節）

4-3 補間用的脈衝發生器自動停止功能

補間驅動根據脈衝發生器的頻率計算當前參與補間驅動的各軸的速度頻率。當補間運算已完成時，如果作為脈衝發生器的軸還沒有停止，則須在程序中手工發送停止命令。

MCD451中可以設定補間運算停止時，自動停止脈衝發生器，省卻以上麻煩。

可在程序中指定是否使用該機能。

4-4 軟件界限越界判定功能

MCD451中可以設定軟件界限。當參與補間的某個軸越過設定的軟件界限時，IP Status-3(B)寄存器內的E_SLMSTS或S_SLMSTS Bit被置為1。用於補間運算的脈衝發生器的軸也將判定為越界停止。

但是，此功能無效時，芯片將忽略軟件界限的設置。

4-5 物理界限越界判定功能

MCD451芯片的每個軸的正負方向上分別擁有一個用於輸出物理界限的信號引腳。當參與補間的某個

軸，在補間驅動對應的方向上，收到物理界限越界信號時，IP Status-3(B) Read Port內的E_LMSSTS或S_LMSSTS Bit將被置為1。用於補間運算的脈衝發生器的軸也將被判定為越界停止。

但是，可以在程序中設定忽略物理界限信號。則補間驅動將不受此類信號的影響。

4-6 馬達反饋的狀態判定功能

MCD451補間回路，可處理從馬達的反饋信號。參與補間的某個軸，接收到報警信號時，IP Status-3(B) Read Port寄存器的ALMSTS Bit被置為1，用於補間運算的脈衝發生器的軸也將因為此報警信號而緊急停止。

但是，可以參程序中設定忽略此信號。則補間驅動將不受此類信號的影響。

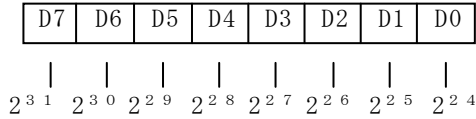
另，參與補間運算的所有軸收到馬達到位信號時，或是所有的軸被指定為“獨立模式”時，IP Status-3(B) Read Port中的INPSTS Bit被置為1。用於補間運算的脈衝發生器的軸也將被認為已經到位。

但是，芯片不會確認“依據INP_n信號（或ALM_n信號）保持BUSY_n為高電平”功能設置為設置為無效的軸。

5. 使用 8Bit 數據總線時的各種端口說明

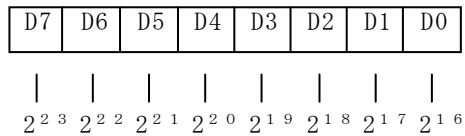
5-1 IP DATA1 WRITE PORT

用于写入补间电路回路部分的各种数据的 $2^{31} \sim 2^{24}$ Bit 部分。詳見第 7 章。



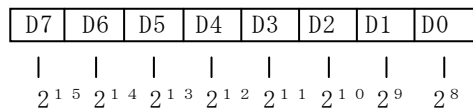
5-2 IP DATA2 WRITE PORT

用于写入补间电路部分的各种数据的 $2^{23} \sim 2^{16}$ Bit 部分。詳見第 7 章。



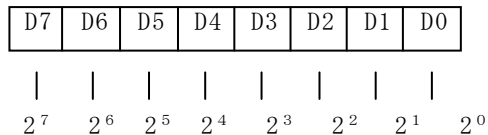
5-3 IP DATA3 WRITE PORT

用于写入补间电路部分的各种数据的 $2^{15} \sim 2^8$ Bit 部分。詳見第 7 章。



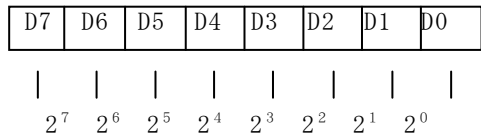
5-4 IP DATA4 WRITE PORT

用于写入补间电路部分的各种数据的 $2^{23} \sim 2^{16}$ Bit 部分。詳見第 7 章。



5-5 IP COMMAND WRITE PORT

用於寫入補間回路各種命令。當補間回路正在發出中斷請求時（INT_OR*為低電平），在本端口寫入補間命令會產生清除中斷效果（INT_OR*變回高阻態）。詳見第 7 章。

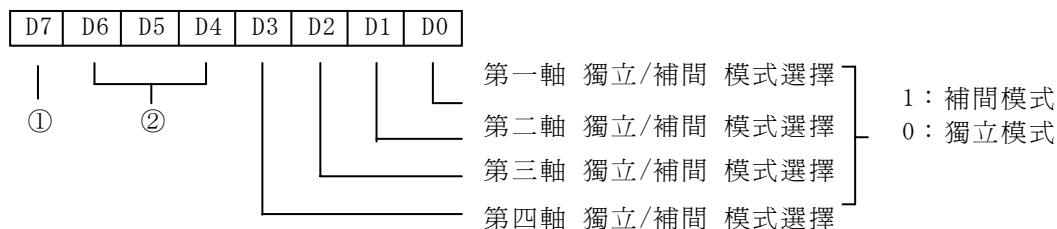


5-6 IP MODE-A WRITE PORT

該寄存器用於指定“單片模式”、“多片模式”、補間驅動電路的脈衝發生器。及決定 1~4 軸工作於獨立模式，還是補間模式。

該寄存器只能在 BUSY_IP 為低電平時設定，BUSY_IP 為高電平（芯片正在進行補間運算）更改該寄存器，將無法保證此後芯片的動作。

芯片重置後，該寄存器被設定為 0。可以讀取 IP Mode-A Read Port 讀出當前的設定值。



①單片模式／多片模式的指定

D7	模式
0	單片模式： 不使用補間機能，或只在 MC 8841P 片內 4 軸範圍內使用補間功能時。可指定為該模式。 （就算是物理上和其他的 MC 8841P 芯片通過 MCB9~MCB0 總線相聯，設置為單片模式時，MCB9~MCB0 的接線信號在內部被屏蔽）
1	多片模式： 和其他的 MC 8841P 通过 MCB9~MCB0 总线相联，可以进行 4×n 轴间的补间运算。

②指定补间运算使用的脉冲发生器（注 4）

D6	D5	D4	脈衝發生器
0	—	—	已和其它 MC 8841P 芯片通過 MCB9~MCB0 互聯，由互聯芯片產生
1	0	0	本芯片第 1 軸
1	0	1	本芯片第 2 軸
1	1	0	本芯片第 3 軸
1	1	1	本芯片第 4 軸

注 4.

補間運算所用的脈衝發生器，通常為參加補間運算的軸，或其它 MC 8841P 芯片中指定的軸。

也可以使用指定了“獨立模式”的軸，但此時，如果中途由於某種原因補間運算停止了，停止原因並不會反饋給脈衝發生器。（如有必要，）此時需要手工停止脈衝發生器。

不使用補間機能時，D6 一定要設為 0。

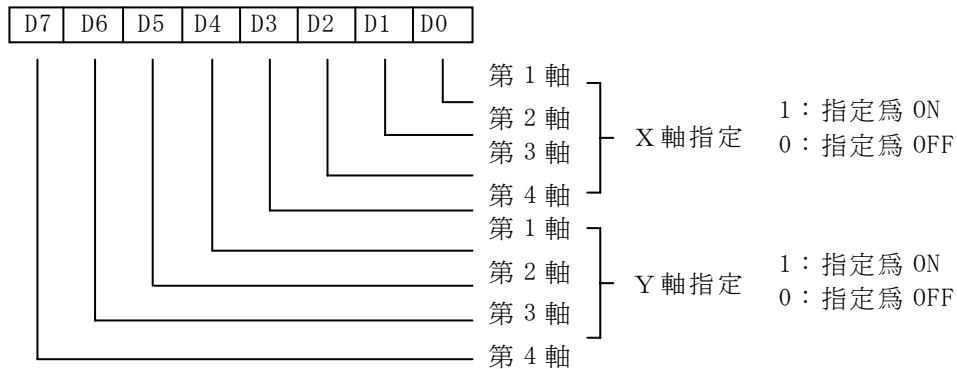
5-7 IP MODE-B WRITE PORT

指定補間運算的 X/Y 軸。

本寄存器隨時可寫入新數據。當前正在進行補間運算時，修改的值要到下一個補間運算命令提交時才生效。

可以同時指定 X/Y 軸，或指定多個 X/Y 軸，或僅指定 X 或 Y 值，但不能將一個軸同時指定了 X 和 Y 軸，否則將無法保證芯片能正常工作。

芯片重置後，該寄存器被設定為 0。當前的設定值可以從 IP Mode-B Read Port 中讀出。

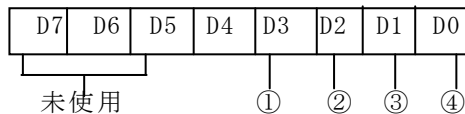


5-8 IP MODE-C WRITE PORT

該寄存器用於控制補間運算的各種補間動作。

芯片正在進行補間運算時（BUSY_IP 為高電平），更改該寄存器的設定（⑤除外），此後芯片的動作將無法保證。芯片重置後，該寄存器被清 0。

該寄存器當前的設定值可以通過 IP Mode-C Read Port 讀出。



①PG AUTO STOP MODE 指定

將 D3 設定為 1 時，補間運算完成時，補間脈衝發生器將會自動停止（限於使用本芯片的軸做脈衝發生器時）。

這個功能在連續補間操作時也可以使用。

②INTERRUPT OUT ENABLE MODE 指定

將 D2 設定為 1 後，補間驅動脈衝輸出完成後（BUSY_IP 從高電平變為低電平），INT_OR*變成低電平。完成輸出以驅動脈衝後且 INT_OR*為低電平時，設置 D2 為 0，INT_OR 將變高阻態。

D2 設定為 0 後，輸出補間脈衝完成時，不會將 INT_OR*設為低電平。

③SIGNAL INDEX DRIVE USE MODE 指定

當連續補間速度脈衝數大於 32Bit 時，或連續補間驅動所需的脈衝數難以計算時，將 D1 設定為 1，就可以使用 Signal Index Drive 來產生速度脈衝。此時，Signal Index Drive 設定的脈衝數，為最後執行的補間命令的脈衝數。（詳見第 9 章 9-4 小節）

連續補間完成後，一定要將 D1 重設為 0。

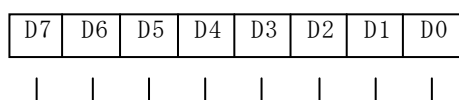
④LAST DRIVE REQUEST

使用④所描述的使用 Signal Index Drive 功能時，寫入最後的補間命令後，將 D0 設定為 1，即開始執行最後的補間運算。運算完成後自動減速停止。（見第 9 章 9-4 小節）

連續補間完成後，必須將 D0 重設為 0。

5-9 IP DATA1 READ PORT

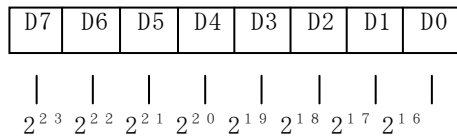
用於讀出補間運算回路中各種數據的 $2^{31} \sim 2^{24}$ Bit 部分的内容。詳見第七章。



$$2^{31} 2^{30} 2^{29} 2^{28} 2^{27} 2^{26} 2^{25} 2^{24}$$

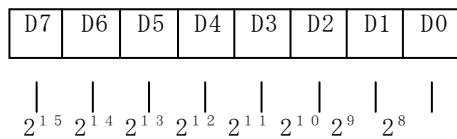
5-10 IP DATA2 READ PORT

用於讀出補間運算回路中各種數據的 $2^{23} \sim 2^{16}$ Bit 部分的内容。詳見第七章。



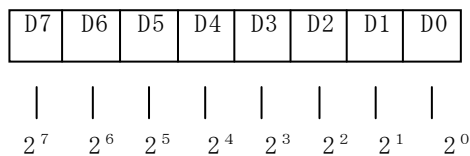
5-11 IP DATA3 READ PORT

用於讀出補間運算回路中各種數據的 $2^{15} \sim 2^8$ Bit 部分的内容。詳見第七章。



5-12 IP DATA4 READ PORT

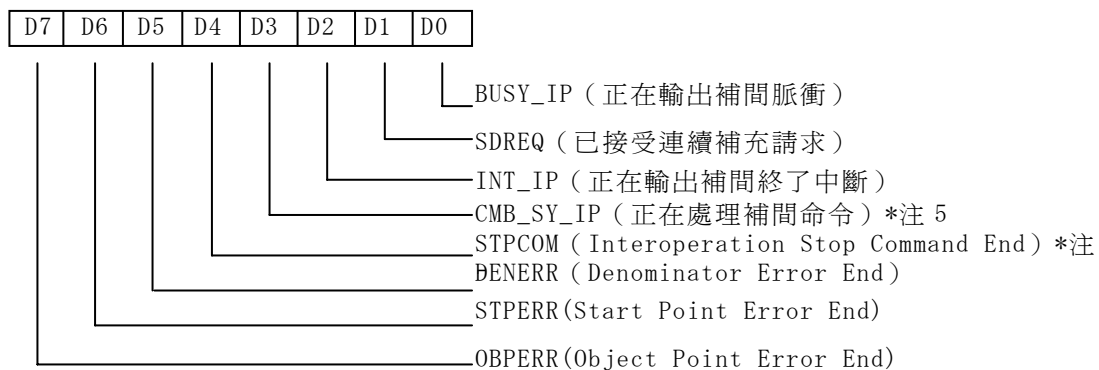
用於讀出補間運算回路中各種數據的 $2^7 \sim 2^0$ Bit 部分的内容。詳見第七章。



5-13 IP STATUS-1 READ PORT

用於讀出補間運算回路的各種狀態，及運算終了原因。該端口隨時可讀。

D3~D0 Bit 的數值反映當前各種輸出引腳的狀態。引腳名後帶有*的低電平有效的電平，該類有效時，對應的 Bit 也將被設為 1 而不是 0。



注 5. INT_OR*, CMB_SY_OR* 信号是指第 1~第 4 轴 PG 回路及补间运算回路产生的中断信号的逻辑與的結果。INT_IP, CMB_SY_IP 將實時反應補間運算回路的當前狀態。

■ STPCOM Bit

補間運算被 Interoperation Stop 命令停止時，STPCOM Bit 被置為 1。芯片開始新的補間運算時 (BUSY_IP 從低電平變為高電平)，該 Bit 自動回復為 0。

■ DENERR Bit

因為補間運算設定的分母值不符合要求，無法進行補間運算時，DENERR Bit 被設定為 1。在進行直線補間或 Endless 直線補間運算時，如果分母和目標位置的關係為：

$$D < \text{Max}(|N1_{obj}|, |N2_{obj}|, |N3_{obj}|, |N4_{obj}|) \text{ 時，}$$

被認定為分母設定值不符合要求 (錯誤)。

接收到下一個補間運算命令後，DENERR Bit 被回復為 0。

■ STPERR Bit

指定的開始位置設置錯誤，從而無法進行補間運算時，STPERR 被設置為 1。將位置設置為：
 (80000000H,Y)、(X,80000000H) 或 (0,0) 時，被認為開始位置設置錯誤。
 接收到下一個補間命令後，STPERR Bit 被回復為 0。

■OBPERR Bit

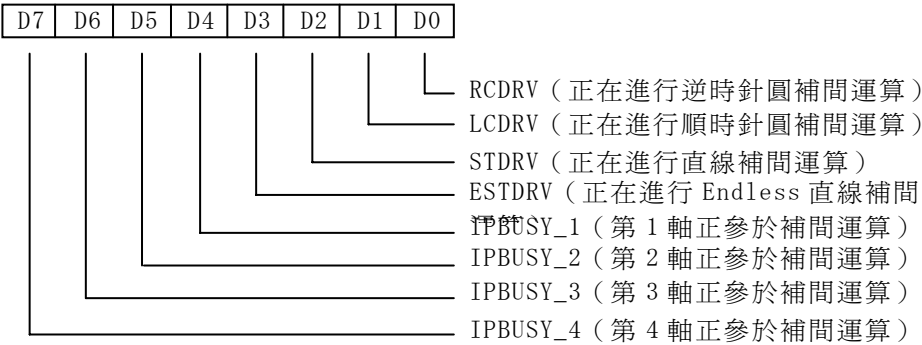
指定的目標位置錯誤，從而無法進行補間運算時，OBPERR 被設置為 1。目標位置錯誤，有以下幾種情形：

- Endless 直線補間時，(N1 Object,N2 Object, N3 Object,N4 Object) 其中之一被設置為 80000000H。
- 圓補間時，目標位置被設置為像 (80000000H,y)，(x,80000000H) 或 (0,0)。

接收到下一個補間命令後，OBPERR 被回復為 0。

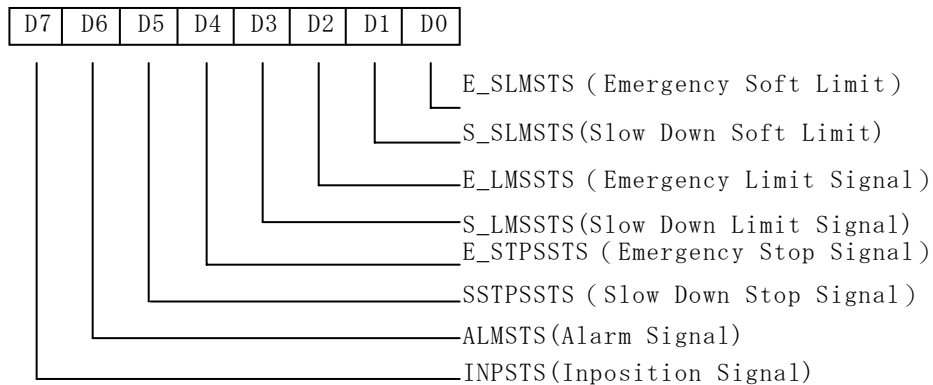
5-14 IP STATUS-2 READ PORT

用於讀出補間驅動脈衝的運算狀態。本端口隨時可讀。
 D7~D4 Bit 實時反應各種輸出引腳的狀態。



5-15 IP STATUS-3 READ PORT

用於讀出補間運算回路的當前補間運算狀態。本端口隨時可讀。這個端口的反應的結果，將用於會影響被指定為補間運算用的脈衝發生器的軸（控制其動作）。



■ E_SLMSTS Bit

參加補間運算的某個軸，還處於設定的已遇到軟件界限，且該軸被設置到達軟件界限則急停止模式時，E_SLMSTS Bit 被設置為 1。

■ S_SLMSTS Bit

參加補間運算的某個軸，還處於設定的已遇到軟件界限，且該軸被設置到達軟件界限則減速停止模式時，S_SLMSTS Bit 被設置為 1。

■ E_LMSSTS Bit

參加補間運算的某個軸，還處於設定的已遇到物理界限，且該軸被設置到達物理界限則急停止模式時，E_LMSSTS Bit 被設置為 1。

■ S_LMSSTS Bit

參加補間運算的某個軸，還處於設定的已遇到物理界限，且該軸被設置到達物理界限則減速停止模式時，S_LMSSTS Bit 被設置為 1。

■ ESTPSSTS Bit

參加補間運算的某個軸，被急停止輸入信號停止時，ESTPSSTS Bit 被設定為 1。

■ SSTPSSTS Bit

參加補間運算的某個軸，被減速停止輸入信號停止時，ESTPSSTS Bit 被設定為 1。

■ ALMSTS Bit

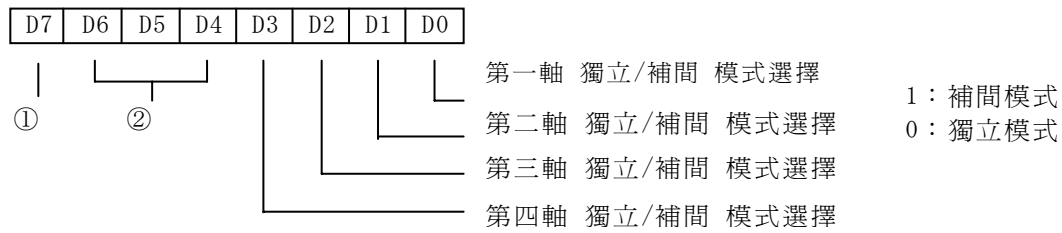
參加補間運算的某個軸，接收到警報輸入信號，且其模式被設定為遇此信號急停止時，ALMSTS Bit 被設置為 1。

■ INPSTS Bit

在補間模式中，指定到達位置確認模式為“等待 INP_n 信號（或 ALM_n 信號）”模式的所有軸，接收到 INP_n 輸入信號時，INPSTS Bit 被設置為 1。所有軸被指定為獨立模式（沒有任何軸被指定為補間模式）時，該 Bit 也被設為 1。

5-16 IP MODE-A READ PORT

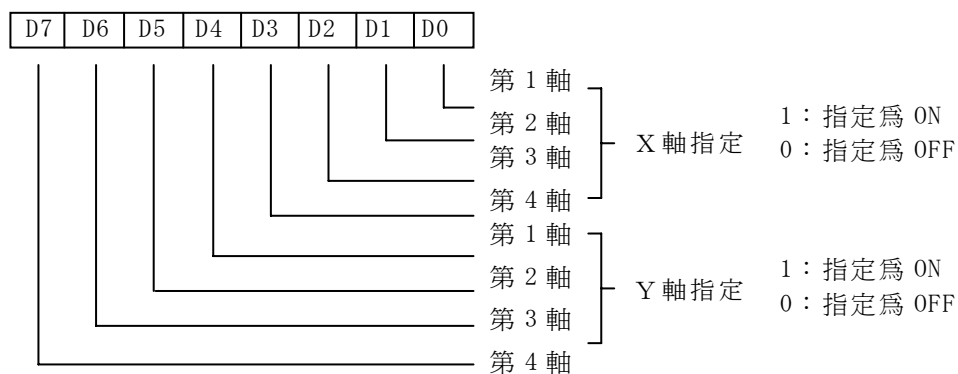
用於讀取以前寫入 IP Mode-A 的數據。該端口隨時可讀。



其中各個 Bit 的意義，請參見 IP Mode-A Write Port 所述。

5-17 IP MODE-B READ PORT

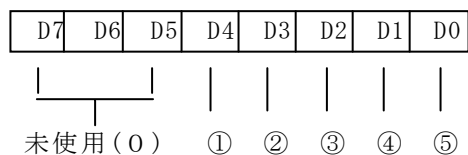
用於讀取以前寫入 IP Mode-B Write Port 的數據。該端口隨時可讀。但該端口的更新，只在圓補間驅動命令開始後進行，而不是反映當前寫入的 IP Mode-B Write Port 的值。



其中各個 Bit 的含義，請見 IP Mode-B Write Port。

5-18 IP MODE-C READ PORT

用於讀出以前寫入 IP Mode-C Write Port 的內容。

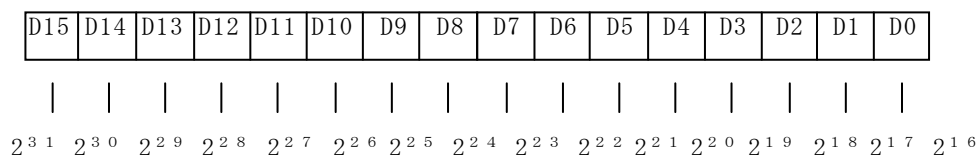


各個 Bit 的意義，詳見 IP Mode-C Write Port。

6. 16Bit 數據總線時的補間回路 IO 端口

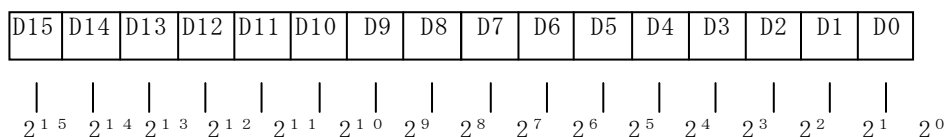
6-1 IP DATA-H WRITE PORT

用於寫入補間回路的各種數據的 $2^{31} \sim 2^{16}$ Bit 部分的內容。詳見第 7 章。



6-2 IP DATA-L WRITE PORT

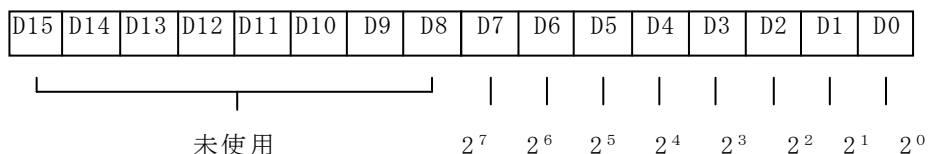
用於寫入補間回路的各種數據的 $2^{15} \sim 2^0$ Bit 部分的內容。詳見第 7 章。



6-3 IP COMMAND WRITE PORT

用於寫入補間回路的各種命令。

補間回路的中斷請求引腳 INT_OR* 為低電平時，在該端口中寫入補間驅動命令，將會使得 INT_OR* 回復到開路狀態（高阻態）。詳見第 7 章。

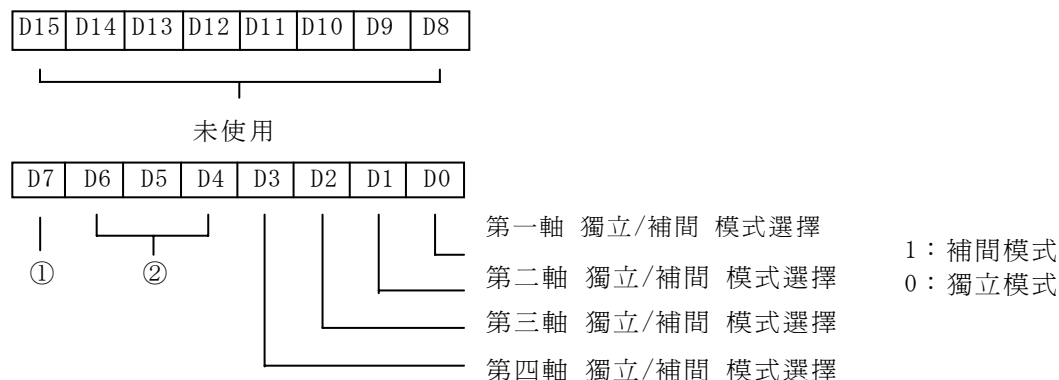


6-4 IP MODE-A WRITE PORT

補間回路的各種模式，如“單片模式” / “多片級聯模式”，以及指定哪個軸做為補間運算的脈衝發生器。還有第 1~4 軸的“獨立模式” / “補間模式”的指定。

該端口只能在補間驅動命令執行前（BUSY_IP 為低電平）時設定。正在輸出補間驅動脈衝時（BUSY_IP 為高電平），更改該端口的設定值，將無法保證此後芯片的動作。

芯片重置後，該端口被重置為 0。當前的設定值，可以從 IP Mode-A Read Port 中讀出。



①單片模式／多片模式的指定

D7	模式
0	單片模式： 不使用補間機能，或只在 MC 8841P 片內 4 軸範圍內使用補間功能時。可指定為該模式。 (就算是物理上和其他的 MC 8841P 芯片通過 MCB9~MCB0 總線相聯，設置為單片模式時，MCB9~MCB0 的接線信號在內部被屏蔽)
1	多片模式： 和其他的 MC 8841P 通过 MCB9~MCB0 总线相联，可以进行 4×n 轴间的补间运算。

②指定补间运算使用的脉冲发生器（注 6）

D6	D5	D4	脈衝發生器
0	—	—	已和其它 MC 8841P 芯片通過 MCB9~MCB0 互聯，由互聯芯片產生
1	0	0	本芯片第 1 軸
1	0	1	本芯片第 2 軸
1	1	0	本芯片第 3 軸
1	1	1	本芯片第 4 軸

注 6.

補間運算所用的脈衝發生器，通常為參加補間運算的軸，或其它 MC 8841P 芯片中指定的軸。

也可以使用指定了“獨立模式”的軸，但此時，如果中途由於某種原因補間運算停止了，停止原因並不會反饋給脈衝發生器。（如有必要，）此時需要手工停止脈衝發生器。

不使用補間機能時，D6 一定要設為 0。

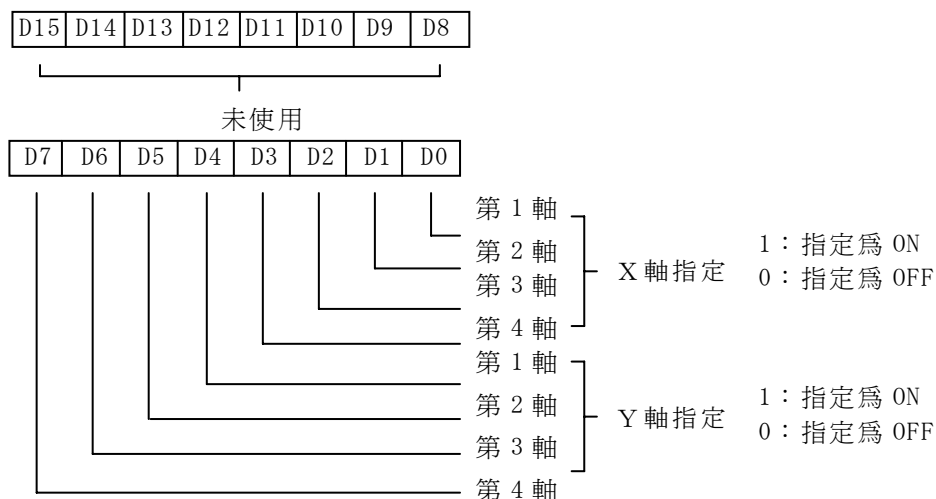
6-5 IP MODE-B WRITE PORT

指定補間運算的 X/Y 軸。

本寄存器隨時可寫入新數據。當前正在進行補間運算時，修改的值要到下一個補間運算命令提交時才生效。

可以同時指定 X/Y 軸，或指定多個 X/Y 軸，或僅指定 X 或 Y 值，但不能將一個軸同時指定了 X 和 Y 軸，否則將無法保證芯片能正常工作。

芯片重置後，該寄存器被設定為 0。當前的設定值可以從 IP Mode-B Read Port 中讀出。

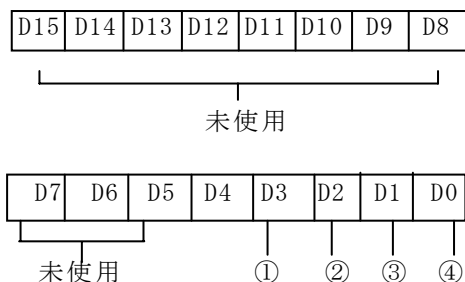


6-6 IP MODE-C WRITE PORT

該寄存器用於控制補間運算的各種補間動作。

芯片正在進行補間運算時（BUSY_IP 為高電平），更改該寄存器的設定（⑤除外），此後芯片的動作將無法保證。芯片重置後，該寄存器被清 0。

該寄存器當前的設定值可以通過 IP Mode-C Read Port 讀出。



① PG AUTO STOP MODE 指定

將 D3 設定為 1 時，補間運算完成時，補間脈衝發生器將會自動停止（限於使用本芯片的軸做脈衝發生器時）。

這個功能在連續補間操作時也可以使用。

② INTERRUPT OUT ENABLE MODE 指定

將 D2 設定為 1 後，補間驅動脈衝輸出完成後（BUSY_IP 從高電平變為低電平），INT_OR*變成低電平。完成輸出以驅動脈衝後且 INT_OR*為低電平時，設置 D2 為 0，INT_OR 將變高阻態。

D2 設定為 0 後，輸出補間脈衝完成時，不會將 INT_OR*設為低電平。

③ SIGNAL INDEX DRIVE USE MODE 指定

當連續補間速度脈衝數大於 32Bit 時，或連續補間驅動所需的脈衝數難以計算時，將 D1 設定為 1，就可以使用 Signal Index Drive 來產生速度脈衝。此時，Signal Index Drive 設定的脈衝數，為最後執行的補間命令的脈衝數。（詳見第 9 章 9-4 小節）
連續補間完成後，一定要將 D1 重設為 0。

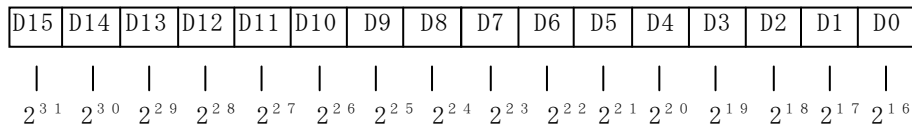
④ LAST DRIVE REQUEST

使用④所描述的使用 Signal Index Drive 功能時，寫入最後的補間命令後，將 D0 設定為 1，即開始執行最後的補間運算。運算完成後自動減速停止。（見第 9 章 9-4 小節）

連續補間完成後，必須將 D0 重設為 0。

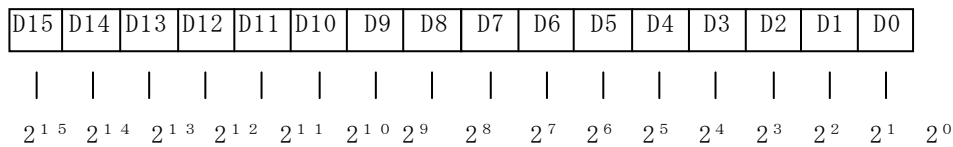
6-7 IP DATA-H READ PORT

用於寫入補間運算回路的各種數據的 $2^{31} \sim 2^{16}$ Bit 部分的內容。詳見第 7 章。



6-8 IP DATA-L READ PORT

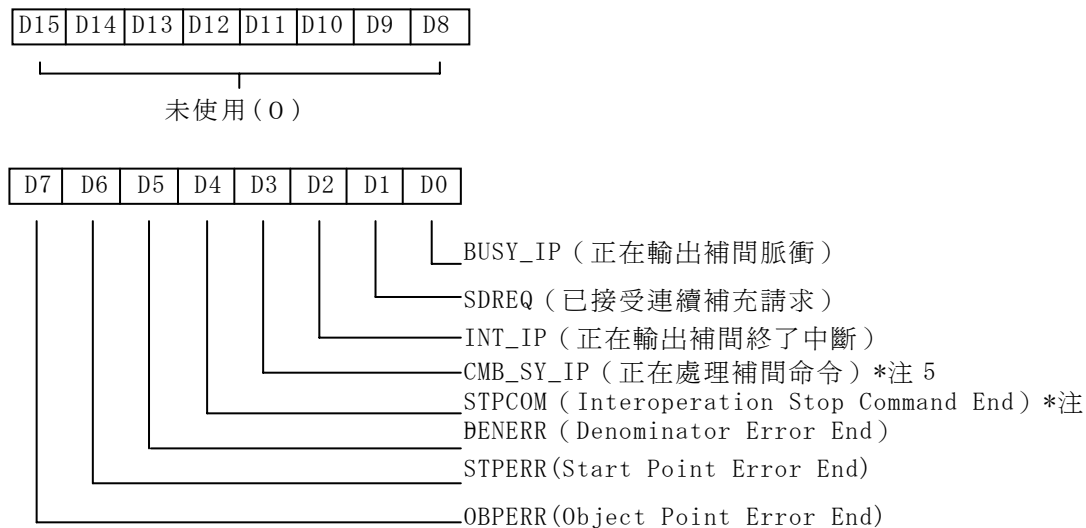
用於寫入補間運算回路的各種數據的 $2^{15} \sim 2^0$ Bit 部分的內容。詳見第 7 章。



6-9 IP STATUS-A READ PORT

用於讀出補間運算回路的各種狀態，及運算終了原因。該端口隨時可讀。

D3~D0 Bit 的數值反映當前各種輸出引腳的狀態。引腳名後帶有*的低電平有效的電平，該類有效時，對應的 Bit 也將被設為 1 而不是 0。



注 5. INT_OR*, CMB_SY_OR* 信号是指第 1~第 4 轴 PG 回路及补间运算回路产生的中断信号的逻辑與的結果。INT_IP, CMB_SY_IP 將實時反應補間運算回路的當前狀態。

■ STPCOM Bit

補間運算被 Interoperation Stop 命令停止時，STPCOM Bit 被置為 1。芯片開始新的補間運算時 (BUSY_IP 從低電平變為高電平)，該 Bit 自動回復為 0。

■ DENERR Bit

因為補間運算設定的分母值不符合要求，無法進行補間運算時，DENERR Bit 被設定為 1。在進行直線補間或 Endless 直線補間運算時，如果分母和目標位置的關係為：

$D < \max(|N1_{Obj}|, |N2_{Obj}|, |N3_{Obj}|, |N4_{Obj}|)$ 時，

被認定為分母設定值不符合要求 (錯誤)。

接收到下一個補間運算命令後，DENERR Bit 被回復為 0。

■ STPERR Bit

指定的開始位置設置錯誤，從而無法進行補間運算時，STPERR 被設置為 1。將位置設置為：

(80000000H, Y)、(X, 80000000H) 或 (0, 0) 時，被認為開始位置設置錯誤。

接收到下一個補間命令後，STPERR Bit 被回復為 0。

■ OBPERR Bit

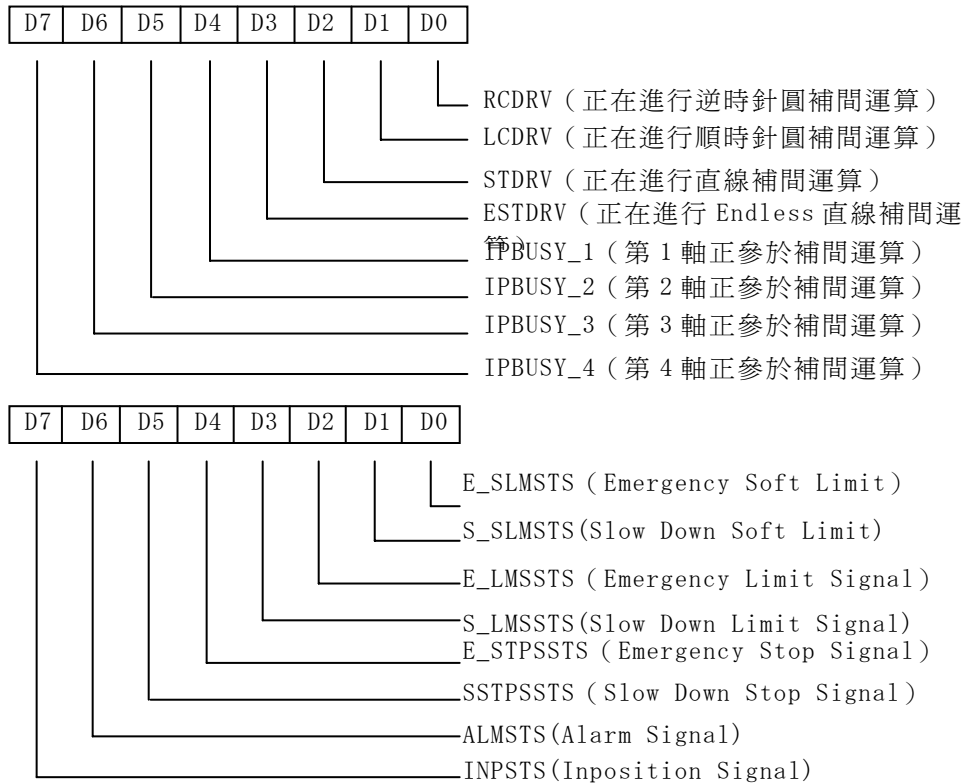
指定的目標位置錯誤，從而無法進行補間運算時，OBPERR 被設置為 1。目標位置錯誤，有以下幾種情形：

- Endless 直線補間時，（N1 Object,N2 Object， N3 Object,N4 Object）其中之一被設置為 80 000000H。
 - 圓補間時，目標位置被設置為像（80000000H,y），（x,80000000H）或（0，0）。
- 接收到下一個補間命令後，OBPERR 被回復為 0。

6-10 IP STATUS-B READ PORT

用於讀取當前補間驅動命令的執行狀態，及反映參與補間操作的各軸的各引腳的狀態。本端口的內容會影響被指定為補間運算脈衝發生器的軸。

D15~D12 實時反映了對應的輸出引腳的當前狀態。



■ E_SLMSTS Bit

參加補間運算的某個軸，還處於設定的已遇到軟件界限，且該軸被設置到達軟件界限則急停止模式時，E_SLMSTS Bit 被設置為 1。

■ S_SLMSTS Bit

參加補間運算的某個軸，還處於設定的已遇到軟件界限，且該軸被設置到達軟件界限則減速停止模式時，S_SLMSTS Bit 被設置為 1。

■ E_LMSSTS Bit

參加補間運算的某個軸，還處於設定的已遇到物理界限，且該軸被設置到達物理界限則急停止模式時，E_LMSSTS Bit 被設置為 1。

■ S_LMSSTS Bit

參加補間運算的某個軸，還處於設定的已遇到物理界限，且該軸被設置到達物理界限則減速停止模式時，S_LMSSTS Bit 被設置為 1。

■ ESTPSSTS Bit

參加補間運算的某個軸，被急停止輸入信號停止時，ESTPSSTS Bit 被設定為 1。

■ SSTPSSTS Bit

參加補間運算的某個軸，被減速停止輸入信號停止時，ESTPSSTS Bit 被設定為 1。

■ ALMSTS Bit

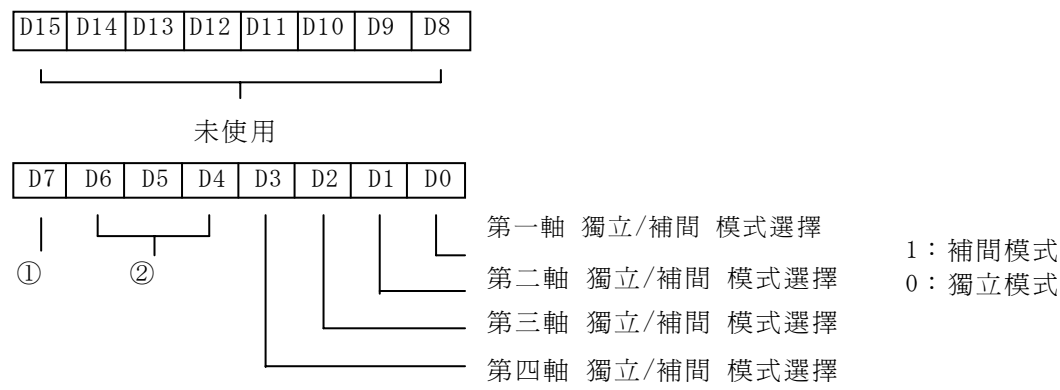
參加補間運算的某個軸，接收到警報輸入信號，且其模式被設定為遇此信號急停止時，ALMSTS Bit 被設置為 1。

■ INPSTS Bit

在補間模式中，指定到達位置確認模式為“等待 INP_n 信號（或 ALM_n 信號）”模式的所有軸，接收到 INP_n 輸入信號時，INPSTS Bit 被設置為 1。所有軸被指定為獨立模式（沒有任何軸被指定為補間模式）時，該 Bit 也被設為 1。

6-11 IP MODE-A READ PORT

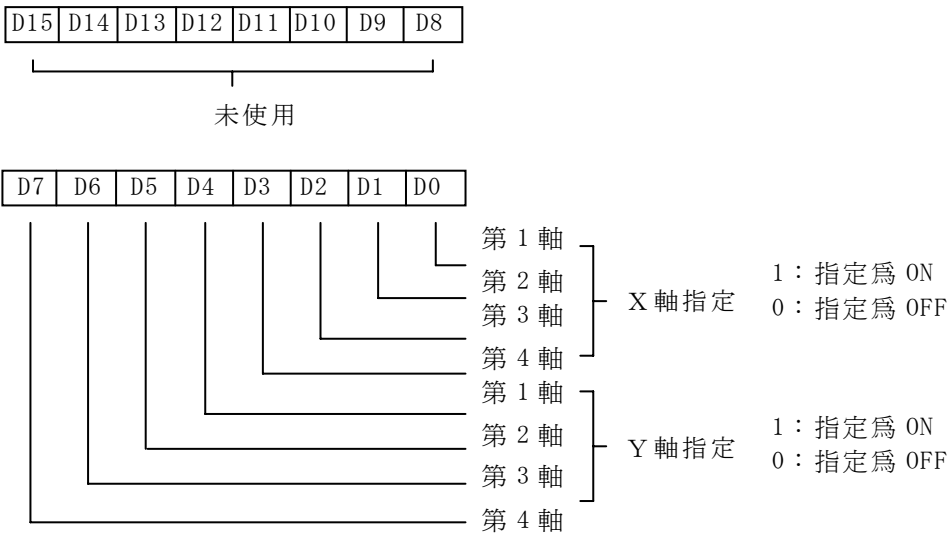
用於讀出先前寫入 IP Mode-A Write Port 的數據。該端口隨時可讀。



其中各個 Bit 的含義，請見 IP Mode-A Write Port 所述。

6-12 IP Mode-B READ PORT

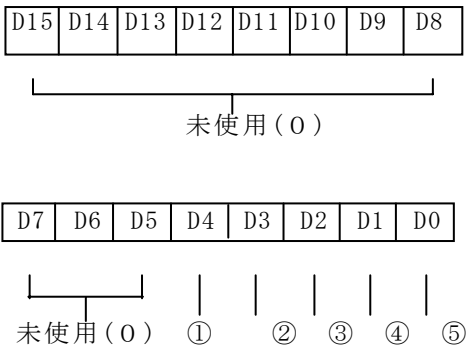
該端口用於讀出先前設定的 IP Mode-B Write Port 的內容。該端口隨時可讀。讀出的數據，要等實際執行了圓補間驅動命令後才會更新。而不是實時反映 IP Mode-B Write Port 的當前數據。



其中各個 Bit 的含義請見 IP Mode-B Write Port 中所述。

6-13 IP Mode-C READ PORT

用於讀出 IP Mode-C Write Port 中寫入的數據。該端口隨時可讀。



讀出的各 Bit 的含義，請見 IP Mode-C Write Port。

7. 補間运算回路命令及功能说明

7-1 補間回路命令码表

本芯片搭載的補間回路，共有 43 個命令。所有命令隨時可執行。特別地，芯片正在輸出補間驅動脈衝時（BUSY_IP 為高電平時），寫入的下一個補間驅動命令，將被當作連續補間驅動請求。

HEX CODE	命令名稱	命令類別		
		寫入動作	讀出動作	補間驅動命令
0 0	INTERPOLATION STOP			
0 1	SEQUENTIAL DRIVE REQUEST CLEAR			
0 2	STRAIGHT DRIVE			○
0 3	ENDLESS STRAIGHT DRIVE			○
0 4	RIGHT CIRCULAR DRIVE A			○
0 5	RIGHT CIRCULAR DRIVE B			○
0 6	LEFT CIRCULAR DRIVE A			○
0 7	LEFT CIRCULAR DRIVE B			○
0 8	INITIAL CLEAR			
0 9	NO OPERATION			
0 A	NO OPERATION			
0 B	NO OPERATION			
0 C	DENOMINATOR PRE-BUFFER WRITE	○		
0 D	DENOMINATOR PRE-BUFFER READ		○	
0 E	DENOMINATOR REGISTER READ		○	
0 F	DENOMINATOR COUNTER READ		○	
1 0	X OBJECT POINT PRE-BUFFER WRITE	○		
1 1	X OBJECT POINT PRE-BUFFER READ		○	
1 2	X OBJECT POINT REGISTER READ		○	
1 3	X NOW POINT REGISTER READ		○	
1 4	X START POINT PRE-BUFFER WRITE	○		
1 5	X START POINT PRE-BUFFER READ		○	
1 6	X START POINT REGISTER READ		○	
1 7	NO OPERATION			
1 8	Y OBJECT POINT PRE-BUFFER WRITE	○		
1 9	Y OBJECT POINT PRE-BUFFER READ		○	
1 A	Y OBJECT POINT REGISTER READ		○	
1 B	Y NOW POINT REGISTER READ		○	
1 C	Y START POINT PRE-BUFFER WRITE	○		
1 D	Y START POINT PRE-BUFFER READ		○	
1 E	Y START POINT REGISTER READ		○	
1 F	NO OPERATION			
2 0	N1 OBJECT POINT PRE-BUFFER WRITE	○		
2 1	N1 OBJECT POINT PRE-BUFFER READ		○	
2 2	N1 OBJECT POINT REGISTER READ		○	
2 3	N1 NOW POINT REGISTER READ		○	
2 4	N2 OBJECT POINT PRE-BUFFER WRITE	○		
2 5	N2 OBJECT POINT PRE-BUFFER READ		○	
2 6	N2 OBJECT POINT REGISTER READ		○	
2 7	N2 NOW POINT REGISTER READ		○	

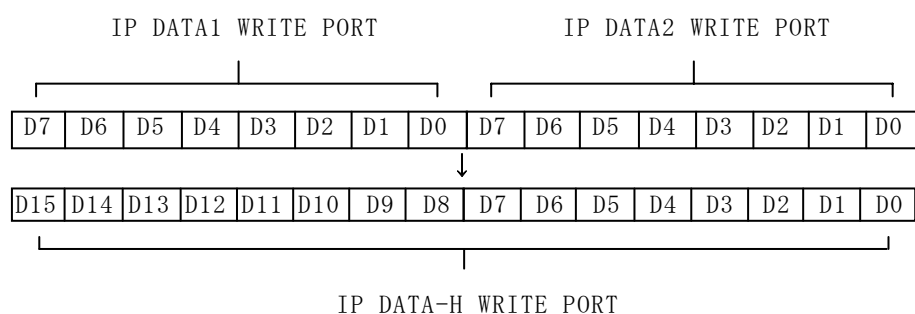
下頁繼續……

HEX CODE	命令名稱	類別		
		寫入動作	讀取動作	補間驅動命令
2 8	N3 OBJECT POINT PRE-BUFFER WRITE	○		
2 9	N3 OBJECT POINT PRE-BUFFER READ		○	
2 A	N3 OBJECT POINT REGISTER READ		○	
2 B	N3 NOW POINT REGISTER READ		○	
2 C	N4 OBJECT POINT PRE-BUFFER WRITE	○		
2 D	N4 OBJECT POINT PRE-BUFFER READ		○	
2 E	N4 OBJECT POINT REGISTER READ		○	
2 F	N4 NOW POINT REGISTER READ		○	
3 0	NO OPERATION			
:	:			
7 F	NO OPERATION			
8 0	実行禁止			
:	:			
F F	実行禁止			

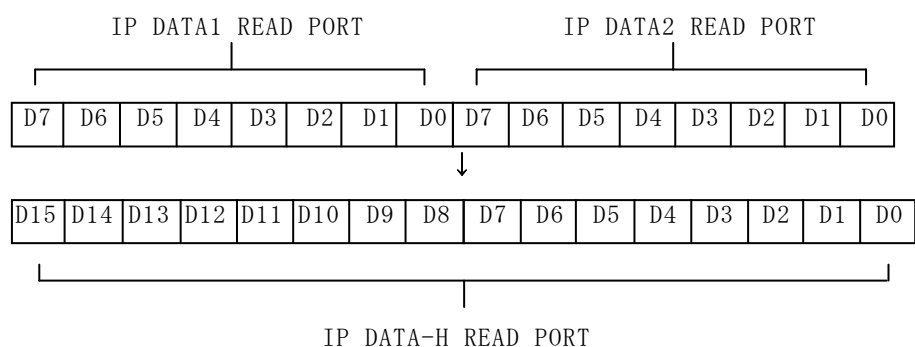
7-2 數據總線寬度不同時存取方法的不同

本章讲述 8 位数据总线时的存取方法。使用 16 位数据总线时。可以将（8Bit 总线下的）2 个存取操作合并为 1 个。

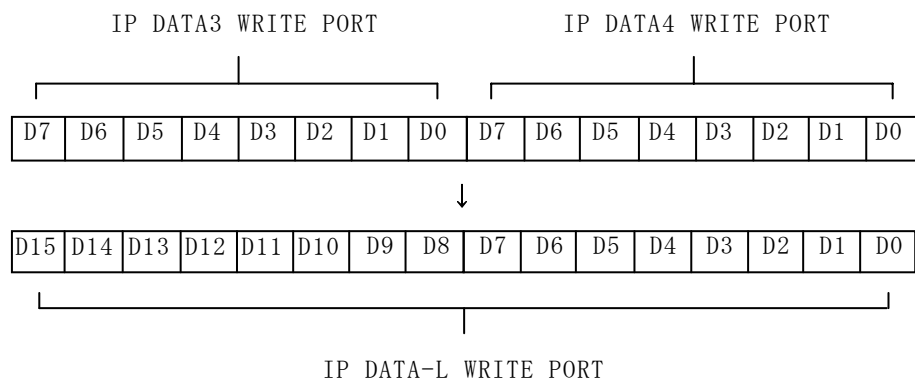
■写入 IP Data1,2 Write Port 的操作，可合并为对 IP Data-H 的一次写操作



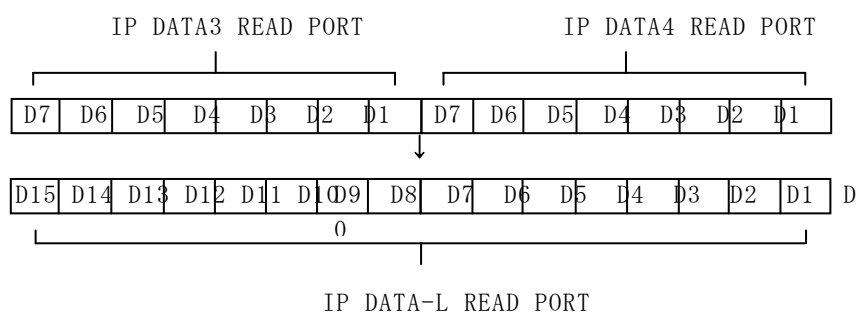
■讀取 IP DATA1,2 READ PORT 的操作、可合併為對 IP Data-H Read Port 的一次讀操作



■写入 IP Data3,4 Write Port 的操作，可合并为一次对 IP Data-L Write Port 的写操作



■讀取 IP Data3,4 Read Port 的操作，可合并为一次对 IP Data-L Read Port 的读操作



7-3 補間回路命令及功能說明

INTERPOLATION STOP 命令	CODE=00H
------------------------------	-----------------

停止當前正在進行的補間運算。執行該命令，往 IP Command Write Port 中寫入命令碼 00H 即可。命令執行後，補間運算停止，BUSY_IP 從高電平變為低電平。

補間運算由於執行該命令停止時，IP Status-1(A) Read Port 的 STPCOM Bit 被置為 1。已接受了連續補間請求（SDREQ*為低電平，IP Status-1(A) Read Port 內的 SDREQ 為 1）也將被解除。

該命令隨時可執行。當前並未進行任何補間驅動運算時，該命令無效。

SEQUENTIAL DRIVE REQUEST CLEAR 命令	CODE=01H
--	-----------------

取消已受理連續補間請求（SDREQ*為低電平，IP Status-1(A) Read Port 的 SDREQ Bit 為 1 時）。往 IP Command Write Port 中寫入命令碼 01H 即可執行該命令。

該命令隨時可執行。未受理任何補間請求時（除 SDREQ*為低電平，IP Status-1(A) Read Port 的 SDREQ Bit 為 1 以外）該命令無效。

STRAIGHT DRIVE 命令	CODE=02H
--------------------------	-----------------

執行直線補間驅動的命令。往 IP Command Write Port 中寫入命令碼 02H 即可提交該命令。

該命令隨時可執行，在當前不在執行任何補間驅動命令中時，直接啟動直線補間運算，BUSY_IP 從低電平變為高電平。當前正在執行補間驅動命令時，SDREQ*引腳變為低電平，IP Status-1(A) 的 SDREQ Bit 被設置為 1，本次驅動請求被當作連續補間請求處理。

連續補間受理後，正在執行的補間驅動完成後，馬上就處理本次請求的補間驅動運算。

執行直線補間驅動命令時，要事先設定輸出比的分母 D，及目標位置 N1 Obj, N2 Obj, N3 Obj, N4 Obj。請參照第 3 章 3-1 小節。

需要事先時行的操作：

- 使用 Denominator Pre-Buffer Write 命令 (Code =0CH)，設定輸出比的分母；
- 使用 N1, N2, N3, N4 Object Point Pre-Buffer Write 命令 (Code=20H, 24H, 28H, 2CH)，設定目標位置。

ENDLESS STRAIGHT DRIVE 命令	CODE=03H
----------------------------------	-----------------

Endless 直線補間驅動命令。往 IP Command Write Port 中寫入命令碼 03H，即可提交該命令。

該命令隨時可執行，在當前不在執行任何補間驅動命令中時，直接啟動 Endless 直線補間運算，BUSY_IP 從低電平變為高電平。當前正在執行補間驅動命令時，SDREQ*引腳變為低電平，IP Status-1(A) 的 SDREQ Bit 被設置為 1，本次驅動請求被當作連續補間請求處理。

連續補間受理後，正在執行的補間驅動完成後，馬上就處理本次請求的補間驅動運算。

執行直線補間驅動命令時，要事先設定輸出比的分母 D，及目標位置 N1 Obj, N2 Obj, N3 Obj, N4 Obj。請參照第 3 章 3-2 小節。

需要事先時行的操作：

- 使用 Denominator Pre-Buffer Write 命令 (Code =0CH)，設定輸出比的分母；
- 使用 N1, N2, N3, N4 Object Point Pre-Buffer Write 命令 (Code=20H, 24H, 28H, 2CH)，設定目標位置。

RIGHT CIRCULAR DRIVE A/B 命令	CODE=04H/05H
------------------------------------	---------------------

執行順時針圓弧補間驅動的命令。往 IP Command Write Port 中寫入命令碼 04H 或 05H 即可啟動該命令。

該命令隨時可執行，在當前不在執行任何補間驅動命令中時，直接啟動圓弧補間運算，BUSY_IP 從低電平變為高電平。當前正在執行補間驅動命令時，SDREQ*引腳變為低電平，IP Status-1(A) 的 SDREQ Bit 被設置為 1，本次驅動請求被當作連續補間請求處理。

連續補間受理後，正在執行的補間驅動完成後，馬上就處理本次請求的補間驅動運算。

順時針圓弧補間有兩種類型，Right Circular Drive A 命令（命令碼為 04H）為未進行圓周速補正的圓弧補間驅動；Right Circular Drive B（命令碼 05H）為有圓周速補正的圓弧補間驅動。

關於圓周速補正的內容，請參見第 4 章 4-1 小節所述。

圓弧補間時，要以圓心為座標 (0, 0) 時，指定弧的開始點位置座標 (Xstr, Ystr)，目標點位置座標

(Xobj,Yobj)，並設定必要的 X/Y 軸。該命令執行前，要事先設定以下參數：

- 使用 X,Y Object Point Pre-Buffer Write 命令（命令碼 10H,18H），設定 X 軸，Y 軸對應的（Xobj,Yobj）座標。
- 使用 X,Y Start Point Pre-Buffer Write 命令（命令碼 14H,1CH），設定 X 軸，軸對應的（Xstr,Ystr）座標。
- 將 X 軸/Y 軸的設定寫入 IP Mode-B Write Port。

LEFT CIRCULAR DRIVE A/B 命令

CODE=06H/07H

提交逆時針圓弧補間驅動的命令。往 IP Command Write Port 中寫入 06 或 07H 時，即可提交該命令。

該命令隨時可執行，在當前不在執行任何補間驅動命令中時，直接啟動圓弧補間運算，BUSY_IP 從低電平變為高電平。當前正在執行補間驅動命令時，SDREQ*引腳變為低電平，IP Status-1(A) 的 SDREQ Bit 被設置為 1，本次驅動請求被當作連續補間請求處理。

連續補間受理後，正在執行的補間驅動完成後，馬上就處理本次請求的補間驅動運算。

逆時針圓弧補間有兩種形式，Left Circular Drive A 命令（命令碼 06H）為未進行圓周速補正的圓弧補正命令，Left Circular Drive B（命令碼 07H）為已進行圓周速補正的圓弧補正命令。

關於圓周速補正的內容，請參見第 4 章 4-1 小節。

圓弧補間時，要以圓心為座標（0，0）時，指定弧的開始點位置座標（Xstr,Ystr），目標點位置座標（Xobj,Yobj），並設定必要的 X/Y 軸。該命令執行前，要事先設定以下參數：

- 使用 X,Y Object Point Pre-Buffer Write 命令（命令碼 10H,18H），設定 X 軸，Y 軸對應的（Xobj,Yobj）座標。
- 使用 X,Y Start Point Pre-Buffer Write 命令（命令碼 14H,1CH），設定 X 軸，軸對應的（Xstr,Ystr）座標。
- 將 X 軸/Y 軸的設定寫入 IP Mode-B Write Port。

INITIAL CLEAR 命令

CODE=08H

將補間回路部分的電路的全部設定，包括設定的模式，寫入的數據，輸出引腳的電平全部重置的命令。往 IP Command Write Port 中寫入命令碼 08H 即可執行該命令。

該命令隨時可執行。芯片正在執行補間驅動命令時（BUSY_IP 為高電平），會停止該命令。並使 BUSY_IP 恢復為低電平。

芯片未進行正常的上電重置時，包括本命令在內的所有命令不能正常執行，故本命令不能代替 Reset 輸入引腳的功能。

DENOMINATOR PRE-BUFFER WRITE 命令

CODE=0CH

設定 Denominator Pre-Buffer 的命令。往 IP Data1,2,3,4 中寫入要設定的數據後，往 IP Command Write Port 端口中寫入命令字 0CH 即可。

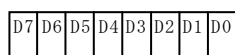
在 Denominator Pre-Buffer 中設定的值，將會在以下的兩種補間驅動命令啟動時，被裝載到 Denominator Register：

* Straight Drive----- 直線補間驅動命令

* Endless Straight Drive ----- Endless 直線補間驅動命令

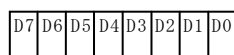
寫入時使用 2 進制補碼形式，可設定的範圍是 0~4294967295（00000000H~FFFFFFFH）。芯片重置後，Denominator Pre-Buffer 被置為 0（00000000H）。該命令隨時可執行。

IP DATA1 WRITE PORT



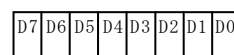
2³¹ 2²⁴

IP DATA2 WRITE PORT



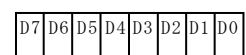
2²³ 2¹⁶

IP DATA3 WRITE PORT



2¹⁵ 2⁸

IP DATA4 WRITE PORT



2⁷ 2⁰

DENOMINATOR PRE-BUFFER READ 命令**CODE=0DH**

讀出 Denominator Pre-Buffer 的當前設定值的命令。往 IP Command Write Port 中寫入命令字 0DH 後，即可從 IP Data1, 2, 3, 4 Read Port 中讀出當前的設定值。該命令隨時可執行。

IP DATA1 READ PORT

D7	D6	D5	D4	D3	D2	D1	D0
----	----	----	----	----	----	----	----

--	--	--	--	--	--	--	--

$2^{31} \dots \dots \dots 2^{24}$

IP DATA2 READ PORT

D7	D6	D5	D4	D3	D2	D1	D0
----	----	----	----	----	----	----	----

--	--	--	--	--	--	--	--

$2^{23} \dots \dots \dots 2^{16}$

IP DATA3 READ PORT

D7	D6	D5	D4	D3	D2	D1	D0
----	----	----	----	----	----	----	----

--	--	--	--	--	--	--	--

$2^{15} \dots \dots \dots 2^8$

IP DATA4 READ PORT

D7	D6	D5	D4	D3	D2	D1	D0
----	----	----	----	----	----	----	----

--	--	--	--	--	--	--	--

$2^7 \dots \dots \dots 2^0$

DENOMINATOR REGISTER READ 命令**CODE=0EH**

讀出 Denominator Register 的當前設定值。往 IP Command Write Port 中寫入命令字 0EH 後，即可從 IP Data1, 2, 3, 4 Read Port 中讀出該值。該命令隨時可執行。

IP DATA1 READ PORT

D7	D6	D5	D4	D3	D2	D1	D0
----	----	----	----	----	----	----	----

--	--	--	--	--	--	--	--

$2^{31} \dots \dots \dots 2^{24}$

IP DATA2 READ PORT

D7	D6	D5	D4	D3	D2	D1	D0
----	----	----	----	----	----	----	----

--	--	--	--	--	--	--	--

$2^{23} \dots \dots \dots 2^{16}$

IP DATA3 READ PORT

D7	D6	D5	D4	D3	D2	D1	D0
----	----	----	----	----	----	----	----

--	--	--	--	--	--	--	--

$2^{15} \dots \dots \dots 2^8$

IP DATA4 READ PORT

D7	D6	D5	D4	D3	D2	D1	D0
----	----	----	----	----	----	----	----

--	--	--	--	--	--	--	--

$2^7 \dots \dots \dots 2^0$

DENOMINATOR COUNTER READ 命令**CODE=0FH**

讀取 Denominator Counter 的當前設定值。往 IP Command Write Port 中寫入命令字 0FH 後，即可從 IP Data1, 2, 3, 4 Read Port 中讀出該值。讀出的數據為 2 進制形式。

該命令隨時可執行。芯片正在執行直線補間、Endless 直線補間驅動命令時，得到的是當前補間驅動的累計現時點的結果經。除此之外的情況下，得到的是最後一次執行直線補間驅動終了時的值。

IP DATA1 READ PORT

D7	D6	D5	D4	D3	D2	D1	D0
----	----	----	----	----	----	----	----

--	--	--	--	--	--	--	--

$2^{31} \dots \dots \dots 2^{24}$

IP DATA2 READ PORT

D7	D6	D5	D4	D3	D2	D1	D0
----	----	----	----	----	----	----	----

--	--	--	--	--	--	--	--

$2^{23} \dots \dots \dots 2^{16}$

IP DATA3 READ PORT

D7	D6	D5	D4	D3	D2	D1	D0
----	----	----	----	----	----	----	----

--	--	--	--	--	--	--	--

$2^{15} \dots \dots \dots 2^8$

IP DATA4 READ PORT

D7	D6	D5	D4	D3	D2	D1	D0
----	----	----	----	----	----	----	----

--	--	--	--	--	--	--	--

$2^7 \dots \dots \dots 2^0$

X/Y OBJECT POINT PRE-BUFFER WRITE 命令**CODE=10H/18H**

設定 X/Y Object Point Pre-Buffer 的命令。往 IP Data1, 2, 3, 4 Write Port 中寫入需設定的值後，往 IP Command Write Port 中寫入 10H 或 18H 即可完成此命令。

保存在 X/Y Object Pre-Buffer 中的值。在以下的兩種補間驅動啟動時，將會被裝載到 X/Y Object Point Register 中。

- Right Circular Drive -----順時針圓弧補間驅動
- Left Circular Drive -----逆時針圓弧補間驅動

設定提供的數據使用 2 進制數據補碼的形式，可設定的範圍為 -2147483647~+2147483647 (80000001H~7FFFFFFH)。芯片重置後，X/Y Object Point Pre-Buffer 被設定為 0 (00000000H)。該命令隨時可執行。

IP DATA1 WRITE PORT

D7	D6	D5	D4	D3	D2	D1	D0
----	----	----	----	----	----	----	----

--	--	--	--	--	--	--	--

$2^{31} \dots \dots \dots 2^{24}$

IP DATA2 WRITE PORT

D7	D6	D5	D4	D3	D2	D1	D0
----	----	----	----	----	----	----	----

--	--	--	--	--	--	--	--

$2^{23} \dots \dots \dots 2^{16}$

IP DATA3 WRITE PORT

D7	D6	D5	D4	D3	D2	D1	D0
----	----	----	----	----	----	----	----

--	--	--	--	--	--	--	--

$2^{15} \dots \dots \dots 2^8$

IP DATA4 WRITE PORT

D7	D6	D5	D4	D3	D2	D1	D0
----	----	----	----	----	----	----	----

--	--	--	--	--	--	--	--

$2^7 \dots \dots \dots 2^0$

X/Y OBJECT POINT PRE-BUFFER READ 命令**CODE=11H/19H**

讀取 X/Y Object Point Pre-Buffer 的當前設定值。往 IP Command Write Port 中寫入命令字 11H 或 19H 後，即可從 IP Data1, 2, 3, 4 Read Port 中讀出該值。該命令隨時可執行。

IP DATA1 READ PORT

D7	D6	D5	D4	D3	D2	D1	D0
----	----	----	----	----	----	----	----

--	--	--	--	--	--	--	--

$2^{31} \dots \dots \dots 2^{24}$

IP DATA2 READ PORT

D7	D6	D5	D4	D3	D2	D1	D0
----	----	----	----	----	----	----	----

--	--	--	--	--	--	--	--

$2^{23} \dots \dots \dots 2^{16}$

IP DATA3 READ PORT

D7	D6	D5	D4	D3	D2	D1	D0
----	----	----	----	----	----	----	----

--	--	--	--	--	--	--	--

$2^{15} \dots \dots \dots 2^8$

IP DATA4 READ PORT

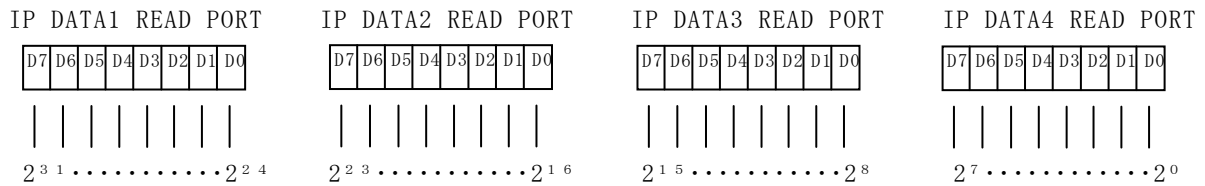
D7	D6	D5	D4	D3	D2	D1	D0
----	----	----	----	----	----	----	----

--	--	--	--	--	--	--	--

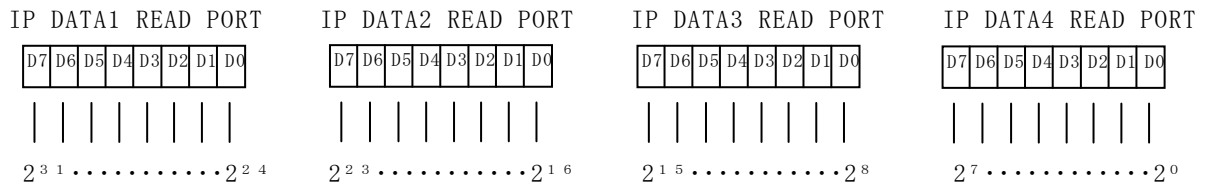
$2^7 \dots \dots \dots 2^0$

X/Y OBJECT POINT REGISTER READ 命令**CODE=12H/1AH**

讀取 X/Y Object Point Register 的當前設定值。往 IP Command Write Port 中寫入命令碼 12H 或 1AH 後，即可從 IP Data1, 2, 3, 4 Read Port 中讀出所需的值（如下所示）。該命令隨時可執行。

**X/Y NOW POINT REGISTER READ 命令****CODE=13H/1BH**

讀取 X/Y Now Point Register 的當前值。往 IP Command Write Port 中寫入命令碼 13H 或 1BH 後，即可從 IP Data 1, 2, 3, 4 Read Port 中讀出所需的值。該命令隨時可執行。驅動器正在進行圓補間運算時，讀出的是當前座標的值。否則為最后一次圓補間終了時的最終的值。

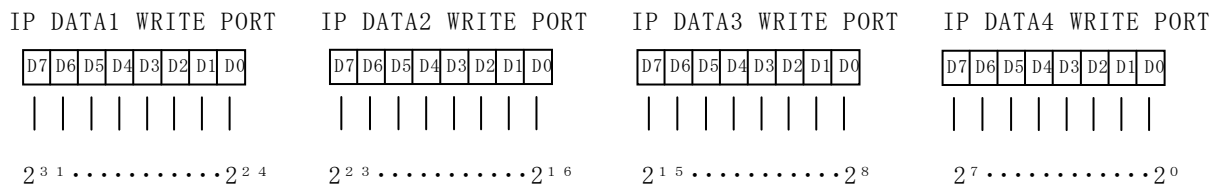
**X/Y START POINT PRE-BUFFER WRITE 命令****CODE=14H/1CH**

設定 X/Y Start Point Pre-Buffer 的命令。往 IP Data1, 2, 3, 4 Write Port 中寫入要設定的值後，往 IP Command Write Port 中寫入命令碼 14H 或 1CH 即可執行此命令。

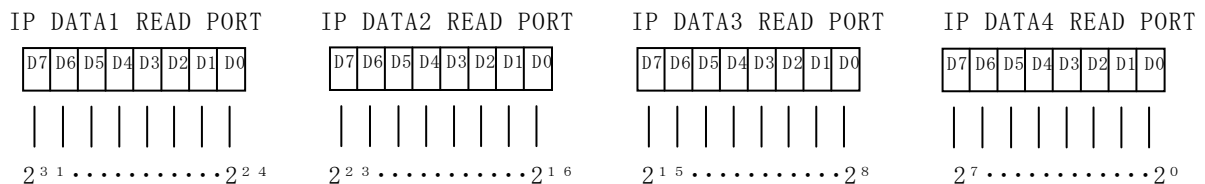
在 X/Y Start Point Pre-Buffer 中的值，在以下兩種補間驅動開始時，將會被裝載到 X/Y Start Point Register，X/Y Now Point Register 中。

- Right Circular Drive -----順時針圓補間驅動
- Left Circular Drive -----逆時值圓補間驅動

用於設定的值採用 2 進制數據補碼形式，可設定的範圍為 -2147483747~+2417483647（80000001H~7FFF FFFH）。芯片重置後，X/Y Start Point Pre-Buffer 被設定為 0（00000000H）該命令隨時可執行。

**X/Y START POINT PRE-BUFFER READ 命令****CODE=15H/1DH**

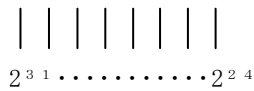
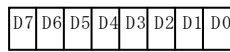
讀取 X/Y Start Point Pre-Buffer 中的當前設定值。往 IP Command Write Port 在寫入命令碼 15H 或 1DH 後，即可從 IP Data1, 2, 3, 4 Read Port 中讀出所需的值。



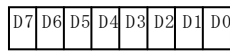
X/Y START POINT REGISTER READ 命令**CODE=16H/1EH**

讀取 X/Y Start Point Register 的當前設定值。往 IP Command Write port 中寫入命令碼 16H 或 1EH 後，即可從 IP Data1, 2, 3, 4 Read Port 中讀出所需的數據。該命令隨時可執行。

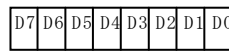
IP DATA1 READ PORT



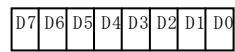
IP DATA2 READ PORT



IP DATA3 READ PORT



IP DATA4 READ PORT

**N1/N2/N3/N4 Object Point Pre-Buffer Write 命令****CODE=20H/24H/28H/2CH**

設定 N1/N2/N3/N4 Object Pre-Buffer 的命令。往 IP Data1, 2, 3, 4 Write Port 中寫入需要設定的值後，往 IP Command Write Port 中寫入 20H、24H、28H 或 2CH 中的一個命令碼即可執行。

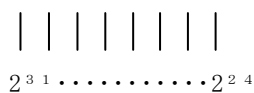
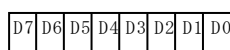
保存在 N1/N2/N3/N4 Object Point Pre-Buffer 中的值。當以下的補間驅動啟動時，會被裝載到 N1/N2/N3/N4 Object Point Register 中。

* Straight Drive----- 直線補間驅動

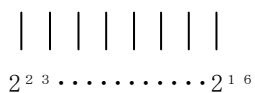
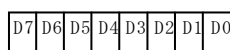
* Endless Straight Drive----- Endless 直線補間驅動

設定時採用的為 2 進制補碼形式。可設定的範圍為 -2147483647 ~ +2147483647 (80000001H ~ 7FFFFFFFH)。芯片重置後，N1/N2/N3/N4 被置為 0 (00000000H)。該命令隨時可執行。

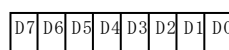
IP DATA1 WRITE PORT



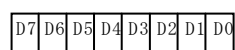
IP DATA2 WRITE PORT



IP DATA3 WRITE PORT

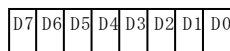


IP DATA4 WRITE PORT

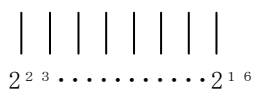
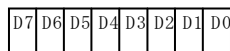
**N1/N2/N3/N4 Object Point Pre-Buffer Read 命令****CODE=21H/25H/29H/2DH**

讀取 N1/N2/N3/N4 Object Point Pre-Buffer 的當前設定值。往 IP Command Write Port 中寫入命令碼 21H、25H、28H 或 2DH 中的一個後，即可從 IP Data1, 2, 3, 4 Read Port 中讀出該值。該命令隨時可執行。

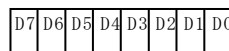
IP DATA1 READ PORT



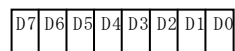
IP DATA2 READ PORT



IP DATA3 READ PORT

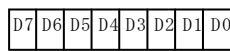


IP DATA4 READ PORT

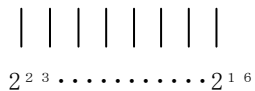
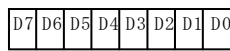
**N1/N2/N3/N4 Object Point Register Read 命令****CODE=22H/26H/2AH/2EH**

讀取 N1/N2/N3/N4 Object Point Register 的當前設定值的命令。往 IP Command Write Port 中寫入命令字 22H、26H、2AH 或 2EH 中的一個，即可從 IP Data1, 2, 3, 4 Read Port 中讀出該值。該命令隨時可執行。

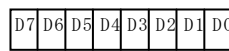
IP DATA1 READ PORT



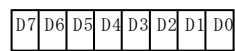
IP DATA2 READ PORT



IP DATA3 READ PORT



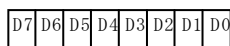
IP DATA4 READ PORT

**N1/N2/N3/N4 NOW POINT REGISTER READ 命令****CODE=23H/27H/2BH/2FH**

讀取 N1/N2/N3/N4 Now Point Register 當前值的命令。往 IP Command Write Port 中寫入 23H、27H、2BH 或 2FH 命令碼的其中一個後，即可從 IP Data1, 2, 3, 4 Read Port 中讀出所需的值。讀出的值為 2 進制補碼形式。

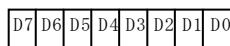
該命令隨時可執行。芯片正在執行直線補間驅動、Endless 直線補間驅動時，讀出的是正在執行的直線補間驅動的當前座標值。否則，讀出的是最後一次執行過的直線補間完成時的最終座標值。

IP DATA1 READ PORT



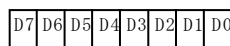
$2^{31} \dots \dots \dots 2^{24}$

IP DATA2 READ PORT



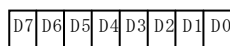
$2^{23} \dots \dots \dots 2^{16}$

IP DATA3 READ PORT



$2^{15} \dots \dots \dots 2^8$

IP DATA4 READ PORT



$2^7 \dots \dots \dots 2^0$

禁止執行的命令

CODE=80H~FFH

處於該範圍的命令碼保留為測試用。一旦執行。將會啟動測試電路。芯片將不會完成正常的功能。

8. 执行补间回路命令的步骤

本章中，就 MC 8841P 芯片的補間回路實行步驟（使用 8Bit 數據總線帶寬時）。

命令大致分為三種類型，“寫入了數據的命令”、“讀取了數據的命令”和“不進行數據存取的命令”。其中，“不進行數據存取的命令”執行時直接往 PG Command Write Port 中寫入命令碼即可。

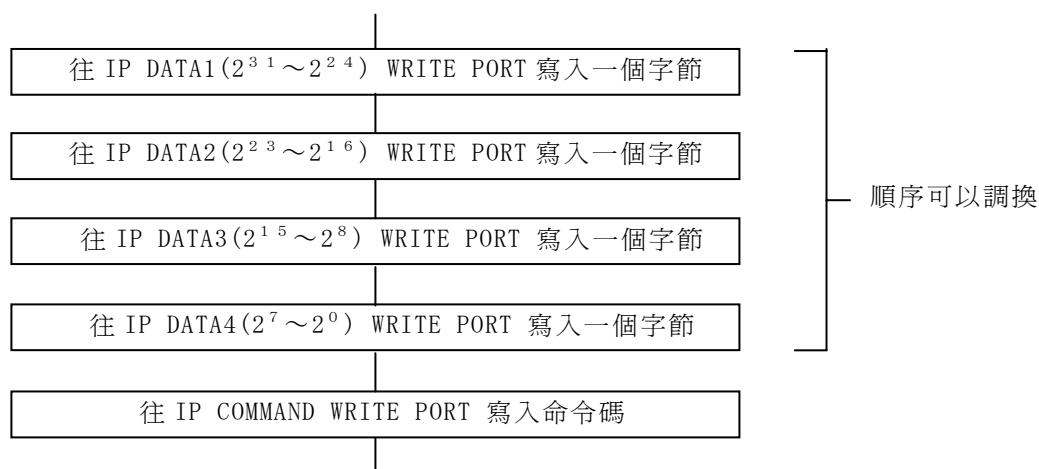
現在的 CPU 處理速度很快，I/O 命令執行時間小於 $T_{cyc} \times 8$ 時，命令執行前，應當先確認 CMBSY_OR* 處於開路（高阻態），或 IP Status-1(A) Read Port 的 CMBSY_IP Bit 確實為 0。詳見 13 章 13-1 小節。

8-1 寫入了 4 字節數據的命令

寫入 4 字節的命令為下表列出的 9 個命令。

HEX CODE	命令名稱
0 C	D E N O M I N A T O R P R E - B U F F E R W R I T E
1 0	X O B J E C T P O I N T P R E - B U F F E R W R I T E
1 4	X S T A R T P O I N T P R E - B U F F E R W R I T E
1 8	Y O B J E C T P O I N T P R E - B U F F E R W R I T E
1 C	Y S T A R T P O I N T P R E - B U F F E R W R I T E
2 0	N 1 O B J E C T P O I N T P R E - B U F F E R W R I T E
2 4	N 2 O B J E C T P O I N T P R E - B U F F E R W R I T E
2 8	N 3 O B J E C T P O I N T P R E - B U F F E R W R I T E
2 C	N 4 O B J E C T P O I N T P R E - B U F F E R W R I T E

執行步驟

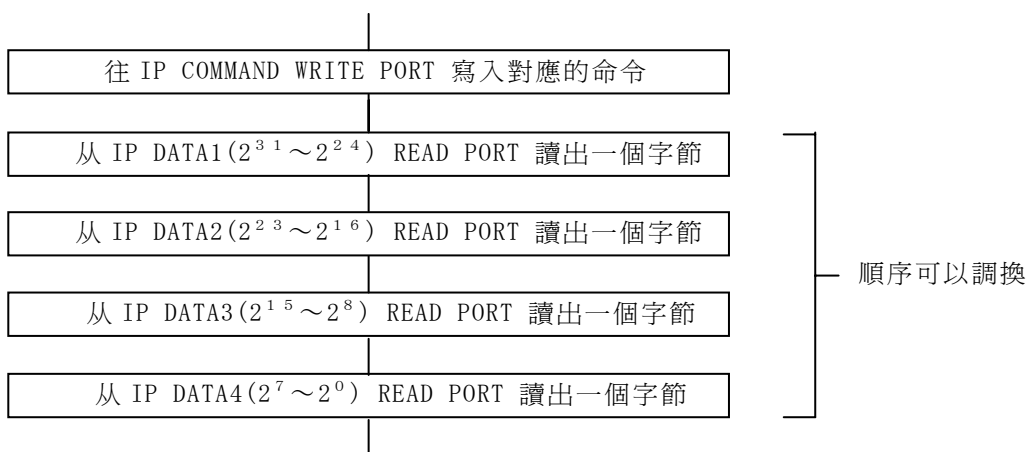


8-2 讀取了 4 個字節的命令

讀取了 4 個字節為下表列出的 25 個命令。

HEX CODE	コ マ ン ド 名 称
0D	DENOMINATOR PRE-BUFFER READ
0E	DENOMINATOR REGISTER READ
0F	DENOMINATOR COUNTER READ
11	X OBJECT POINT PRE-BUFFER READ
12	X OBJECT POINT REGISTER READ
13	X NOW POINT REGISTER READ
15	X START POINT PRE-BUFFER READ
16	X START POINT REGISTER READ
19	Y OBJECT POINT PRE-BUFFER READ
1A	Y OBJECT POINT REGISTER READ
1B	Y NOW POINT REGISTER READ
1D	Y START POINT PRE-BUFFER READ
1E	Y START POINT REGISTER READ
21	N1 OBJECT POINT PRE-BUFFER READ
22	N1 OBJECT POINT REGISTER READ
23	N1 NOW POINT REGISTER READ
25	N2 OBJECT POINT PRE-BUFFER READ
26	N2 OBJECT POINT REGISTER READ
27	N2 NOW POINT REGISTER READ
29	N3 OBJECT POINT PRE-BUFFER READ
2A	N3 OBJECT POINT REGISTER READ
2B	N3 NOW POINT REGISTER READ
2D	N4 OBJECT POINT PRE-BUFFER READ
2E	N4 OBJECT POINT REGISTER READ
2F	N4 NOW POINT REGISTER READ

执行步骤



9. 補間運算回路編程實例

為加深大家對補間回路的理解。本章闡述 8 Bit 數據總線下 C 語言、實現第 1 軸、第 2 軸補間驅動的編程例子。

基本軸的編程示例，請參見《MCD451 使用說明書（基本功能編）》。此處假定第 1 軸~第 4 軸都已檢出機械原點。

此處假定 MCD451 所佔用的 I/O 端口地址為 40H~7FH（第 1 軸為 40H~47H、第 2 軸為 48H~4FH、第 3 軸為 50H~57H、第 4 軸為 58H~5FH，補間回路為 60H~6FH。

此外，依賴於 MCD451 標準輸出時鐘信號的頻率和 CPU 處理速度，寫入命令後，在一定時間窗口內，可能不應對芯片進行存取操作。

本章中的程序，在執行命令前，都對前一個命令是否處理完畢進行了判斷。詳見 13 章 13-1 小節。

9-1 初始設定條件

本例中，假定程序開始前，補間回路部分的初始設定內容如下：

. 單芯片模式/多芯片模式..... 單芯片模式（不進行多個芯片間的補間）
. 第 1 軸 “補間模式”/“獨立模式”..... 補間模式
. 第 2 軸 “補間模式”/“獨立模式”..... 補間模式
. 第 3 軸 “補間模式”/“獨立模式”..... 獨立模式
. 第 4 軸 “補間模式”/“獨立模式”..... 獨立模式
. 圓弧補間時的 X 軸指定..... 第 1 軸
. 圓弧補間時的 Y 軸指定..... 第 2 軸
. PG Auto Stop Mode 設定..... 啓用

不過，此處的設定，根據實際情況可隨意指定。（見第 5 章 5-6、5-7、5-8 小節，第 6 章的 6-4、6-5、6-6 上節）

下面的程序沒有確認 PG 回路的輸出脈衝（譯注：指脈衝發生器）是否終了。因為此處指定了 PG Auto Stop Mode，故補間操作結束後，用做補間運算的脈衝發生器也自動停止。

如果沒有指定 PG Auto Stop Mode，補間運算結束後，還應向 PG 回路寫入停止命令，確認脈衝發生器是否終止。

9-2 動作例 1

本程序中，執行以下的無連續補間、無周速補正動作。

(0 , 0)

↓ 直線補間

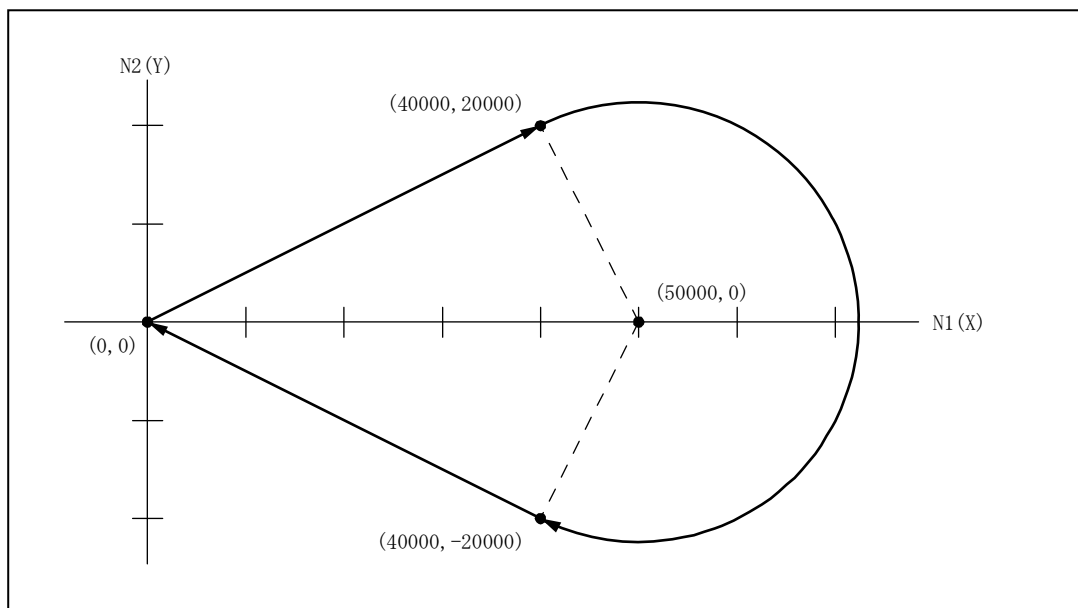
(40000 , 20000)

↓ 順時針補間 [中心坐標為 (50000 , 0)]

(40000 , -20000)

↓ 直線補間

(0 , 0)



■ C 語言源代碼

```
/* **** */
/* MCD451 第 1 轴 I/O 端口定义 */
/* **** */

#define DATA1_1 0x00 /* N1 DATA1 READ/WRITE */
#define DATA2_1 0x01 /* N1 DATA2 READ/WRITE */
#define DATA3_1 0x02 /* N1 DATA3 READ/WRITE */
#define DATA4_1 0x03 /* N1 DATA4 READ/WRITE */

#define COMM_1 0x04 /* N1 COMMAND WRITE */
#define STSSEL_1 0x05 /* N1 DRIVE STATUS SELECT WRITE */

#define DSTS_1 0x04 /* N1 DRIVE STATUS READ */
#define ESTS_1 0x05 /* N1 END STATUS READ */
#define MECS_1 0x06 /* N1 MECHANICAL SIGNAL READ */

#define UNIS_1 0x07 /* N1 UNIVERSAL SIGNAL READ/WRITE */
```

```

/*****
/* MCD451 补间运算 I/O 端口定义 */
*****/

#define DATA1_IP 0x20 /* IP DATA1 READ/WRITE */
#define DATA2_IP 0x21 /* IP DATA2 READ/WRITE */
#define DATA3_IP 0x22 /* IP DATA3 READ/WRITE */
#define DATA4_IP 0x23 /* IP DATA4 READ/WRITE */

#define COMM_IP 0x24 /* IP COMMAND WRITE */

#define STS1_IP 0x24 /* IP STATUS-1 READ */
#define STS2_IP 0x25 /* IP STATUS-2 READ */
#define STS3_IP 0x26 /* IP STATUS-3 READ */

#define MODEA_IP 0x28 /* IP MODE-A READ/WRITE */
#define MODEB_IP 0x29 /* IP MODE-B READ/WRITE */
#define MODEC_IP 0x2A /* IP MODE-C READ/WRITE */

/*****
/* MCD451 端口基址 */
*****/

Short HEAD_ADDR = 0x0040;

/*****
/* MCD451 第 1 轴 命令處理完了確認 */
*****/
void cmd_chk_n1(void)
{
    outp(HEAD_ADDR+STSSEL_1, 0x00); /* N1 DRIVE STATUS1 SELECT */
    while ((inp(HEAD_ADDR+DSTS_1) & 0x80) != 0x00); /* CMBSY_1=0 WAIT */
}

/*****
/* MCD451 第 1 轴 驅動命令完成確認 */
*****/
void end_chk_n1(void)
{
    outp(HEAD_ADDR+STSSEL_1, 0x00); /* N1 DRIVE STATUS1 SELECT */
    while ((inp(HEAD_ADDR+DSTS_1) & 0x01) != 0x00); /* BUSY_1=0 WAIT */
}

/*****
/* MCD451 第 1 轴 命令写入 */
*****/
void com_wrt_n1(short comm)
{
    cmd_chk_n1(); /* CMBSY_1=0 WAIT */
    outp(HEAD_ADDR+COMM_1, comm); /* COMMAND WRITE */
}

```

```

/*****
/* MCD451 第1轴 4Bytes 数据+命令写入 */
/*****/
void com_lwrt_n1(short comm, unsigned long data)
{
    cmd_chk_n1(); /* CMBSY_1=0 WAIT */
    outp(HEAD_ADDR+DATA1_1, (data >> 24) & 0xff); /* D31 - D24 WRITE */
    outp(HEAD_ADDR+DATA2_1, (data >> 16) & 0xff); /* D23 - D16 WRITE */
    outp(HEAD_ADDR+DATA3_1, (data >> 8) & 0xff); /* D15 - D8 WRITE */
    outp(HEAD_ADDR+DATA4_1, data & 0xff); /* D7 - D0 WRITE */
    outp(HEAD_ADDR+COMM_1, comm); /* COMMAND WRITE */
}

/*****
/* MCD451 第1轴 命令写入+4Bytes 数据读出 */
/*****/
unsigned long com_lrd_n1(short comm)
{
    unsigned long data idata;
    unsigned char bwork;

    cmd_chk_n1(); /* CMBSY_1=0 WAIT */
    outp(HEAD_ADDR+COMM_1, comm); /* COMMAND WRITE */
    cmd_chk_n1(); /* CMBSY_1=0 WAIT */

    bwork = inp(HEAD_ADDR+DATA1_1); /* D31 - D24 READ */
    idata = (unsigned long)bwork << 24;
    bwork = inp(HEAD_ADDR+DATA2_1); /* D23 - D16 READ */
    idata |= (unsigned long)bwork << 16;
    bwork = inp(HEAD_ADDR+DATA3_1); /* D15 - D8 READ */
    idata |= (unsigned long)bwork << 8;
    bwork = inp(HEAD_ADDR+DATA4_1); /* D7 - D0 READ */
    idata |= bwork;

    return idata;
}

/*****
/* MCD451 補間回路 命令處理終了確認 */
/*****/
void cmd_chk_ip(void)
{
    while ((inp(HEAD_ADDR+STS1_IP) & 0x08) != 0x00); /* CMBSY_IP=0 WAIT */
}

/*****
/* MCD451 補間驅動完了確認 */
/*****/
void end_chk_ip(void)
{
    while ((inp(HEAD_ADDR+STS1_IP) & 0x01) != 0x00); /* BUSY_IP=0 WAIT */
}

```

```

/*****
/* MCD451 補間運算回路 命令寫入 */
/*****/
void com_wrt_ip(short comm)
{
    cmd_chk_ip(); /* CMBSY_IP=0 WAIT */
    outp(HEAD_ADDR+COMM_IP, comm); /* COMMAND WRITE */
}

/*****
/* MCD451 補間運算回路 4Bytes 数据+命令碼写入 */
/*****/
void com_lwrt_ip(short comm, unsigned long data)
{
    cmd_chk_ip(); /* CMBSY_IP=0 WAIT */
    outp(HEAD_ADDR+DATA1_IP, (data >> 24) & 0xff); /* D31 - D24 WRITE */
    outp(HEAD_ADDR+DATA2_IP, (data >> 16) & 0xff); /* D23 - D16 WRITE */
    outp(HEAD_ADDR+DATA3_IP, (data >> 8) & 0xff); /* D15 - D8 WRITE */
    outp(HEAD_ADDR+DATA4_IP, data & 0xff); /* D7 - D0 WRITE */
    outp(HEAD_ADDR+COMM_IP, comm); /* COMMAND WRITE */
}

/*****
/* MCD451 補間運算回路 初始化設定 */
/*****/
void hmc_init_ip(void)
{
    outp(HEAD_ADDR+MODEA_IP, 0x43); /* 单芯片模式 */
    /* 補間用的脈衝發生器 = 第 1 軸 */
    /* 第 1、2 軸 = 補間模式 */
    /* 第 3、4 軸 = 獨立模式 */
    outp(HEAD_ADDR+MODEB_IP, 0x21); /* X 軸 = 第 1 軸, Y 軸 = 第 2 軸 */
    outp(HEAD_ADDR+MODEC_IP, 0x08); /* PG AUTO STOP MODE */
}

/*****
/* 一系列的補間驅動 */
/*****/
void ip_drive(void)
{
    com_lwrt_ip(0x20, 40000); /* N1 OBJECT POINT PRE-BUFFER = 40000 WRITE */
    /* 將現在位置坐标认定为 (0,0) 時 N1 軸的坐标 40,000, 即相对坐标 40,000 */
    com_lwrt_ip(0x24, 20000); /* N2 OBJECT POINT PRE-BUFFER = 20000 WRITE */
    /* 將現在坐标認定為 (0,0) 時 N2 軸的目標位置坐标 20,000, 即相對坐标 20,000 */
    com_lwrt_ip(0x28, 0); /* N3 OBJECT POINT PRE-BUFFER = 0 WRITE */
    /* N3 軸不進行補間操作 */
    com_lwrt_ip(0x2c, 0); /* N4 OBJECT POINT PRE-BUFFER = 0 WRITE */
    /* N4 軸不進行補間操作 */
    com_lwrt_ip(0x0c, 40000); /* DENOMINATOR PRE-BUFFER = 40000 WRITE */
    /* 分母值的算出方法, 見 10 章 10-1 a) 小节 */
    com_wrt_ip(0x02); /* 开始直线补間运算 */
    com_lwrt_ip(0x20, 40000); /* N1 40000 PULSE INDEX DRIVE START */
    /* 做為補間脈衝發生器, 啟動第 1 軸 */

    end_chk_ip(); /* 直線補間完了確認 */
}

```

```

com_lwrt_ip(0x14, -10000);          /* X START POINT PRE-BUFFER = -10000 WRITE */
/*將圓心坐标(50000,0)作為原點(0,0)的X軸開始坐標40,000,也即-10,000 */
com_lwrt_ip(0x1c, 20000);          /* Y START POINT PRE-BUFFER = 20000 WRITE */
/*將圓心坐标(50000,0)作為原點(0,0)的Y軸開始點坐标20,000即20,000 */
com_lwrt_ip(0x10, -10000);          /* X OBJECT POINT PRE-BUFFER = -10000 WRITE */
/*將圓心坐标(50000,0)作為原點(0,0)的X軸目標位置坐標40,000,即-10,000 */
com_lwrt_ip(0x18, -20000);          /* Y OBJECT POINT PRE-BUFFER = -20000 WRITE */
/*將圓心坐标(50000,0)作為原點(0,0)的Y軸目標位置坐标-20,000,即-20,000 */
com_wrt_ip(0x04);                  /*開始順時針不圓周速補正補間*/
com_lwrt_n1(0x20, 83248);           /* N1 83248 PULSE INDEX DRIVE START */
/* 做為補間脈衝發生器，啟動第1軸 */
/* 速度指令的算出方法，請參照10章10-2小節 */

end_chk_ip();                      /* 圓弧補間完了確認 */

com_lwrt_ip(0x20, -40000);          /* N1 OBJECT POINT PRE-BUFFER = -40000 WRITE */
/* 相对于现在位置坐标(40000,-20000)的N1轴目标位置坐标0,即-40,000 */
com_lwrt_ip(0x24, 20000);          /* N2 OBJECT POINT PRE-BUFFER = 20000 WRITE */
/* 相对于现在位置坐标(40000,-20000)的N2轴目标位置坐标0,即20,000 */
com_lwrt_ip(0x0c, 40000);          /* DENOMINATOR PRE-BUFFER = 40000 WRITE */
/* 分母D的计算方法，請見第10章10-1 a)小節 */
com_wrt_ip(0x02);                  /* 开始直线補間運算 */
com_lwrt_n1(0x20, 40000);           /* N1 40000 PULSE INDEX DRIVE START */
/* 做為補間運算的脈衝發生器，啟動第1軸 */

end_chk_ip();                      /* 確認補間操作完了 */
}

/*****/
/* MCD451 補間編程例 */
/*****/
void main(void)
{
    /* PG 回路的初始化設定，原點檢出等處理 */

    hmc_init_ip();                  /* 初始化 MCD451 補間回路 */

    ip_drive();                     /* 執行一系列的補間操作 */
}

```

9-3 動作例2

本程序執行和 9-2 相同的動作。但展示的是有周速補正連續補間的情形。
此處只展示和 9-2 例中不同的 ip_drive 函數。並省略了一部分注釋。

```
/* **** */
/* 一系列的補間驅動 */
/* **** */
void ip_drive(void)
{
    com_lwrt_ip(0x20, 40000);          /* N1 OBJECT POINT PRE-BUFFER = 40000 WRITE */
    com_lwrt_ip(0x24, 20000);          /* N2 OBJECT POINT PRE-BUFFER = 20000 WRITE */
    com_lwrt_ip(0x28, 0);              /* N3 OBJECT POINT PRE-BUFFER = 0 WRITE */
    com_lwrt_ip(0x2c, 0);              /* N4 OBJECT POINT PRE-BUFFER = 0 WRITE */
    com_lwrt_ip(0x0c, 48000);          /* DENOMINATOR PRE-BUFFER = 48000 WRITE */
    /* 分母值 D 的計算方法，請見第 10 章 10-1 c) 小節 */
    com_wrt_ip(0x02);                  /* 啟動直線補間運算 */
    com_lwrt_nl(0x20, 191618);          /* N1 191618 PULSE INDEX DRIVE START */
    /* 做為補間驅動的脈沖發生器，啟動第 1 軸 */
    /* 191, 618 可解譯為 48,000 + 95,618 + 48,000 */
    /* 速度指令脈沖數的計算方法，詳見第 10 章 10-1 c)、第 10 章 10-3 小節 */

    while ((inp(HEAD_ADDR+STS1_IP) & 0x01) != 0x01); /* BUSY_IP=1 WAIT */

    com_lwrt_ip(0x14, -10000);          /* X START POINT PRE-BUFFER = -10000 WRITE */
    com_lwrt_ip(0x1c, 20000);          /* Y START POINT PRE-BUFFER = 20000 WRITE */
    com_lwrt_ip(0x10, -10000);          /* X OBJECT POINT PRE-BUFFER = -10000 WRITE */
    com_lwrt_ip(0x18, -20000);          /* Y OBJECT POINT PRE-BUFFER = -20000 WRITE */
    com_wrt_ip(0x05);                  /* 順時針有周速補正請求 */
    /* 將被視為連續補間請求 */

    while ((inp(HEAD_ADDR+STS1_IP) & 0x02) != 0x02); /* SDREQ=1 WAIT */
    while ((inp(HEAD_ADDR+STS1_IP) & 0x02) != 0x00); /* SDREQ=0 WAIT */
    /* 等待直線補間驅動完成后，開始順時針圓弧補間 */

    com_lwrt_ip(0x20, -40000);          /* N1 OBJECT POINT PRE-BUFFER = -40000 WRITE */
    com_lwrt_ip(0x24, 20000);          /* N2 OBJECT POINT PRE-BUFFER = 20000 WRITE */
    com_lwrt_ip(0x0c, 48000);          /* DENOMINATOR PRE-BUFFER = 48000 WRITE */
    /* 分母 D 的算出方法，請見第 10 章 10-1 c) 小節 */
    com_wrt_ip(0x02);                  /* 直線補間請求 */
    /* 將被視為連續補間請求 */

    end_chk_ip();                      /* 確認連續補間完成 */
}
```

9-4 動作例3

和 9-3 中的例子相同。但此處展示執行 Signal Index Drive 時，如何指定最後的速度指令脈衝。
此外，此處僅展示和 9-3 中不同的函數 ip_drive。

```

/*****
/* 一系列的补间驱动 */
*****/
void ip_drive(void)
{
    outp(HEAD_ADDR+MODEC_IP, 0x0a); /* PG AUTO STOP MODE,SIGNAL INDEX DRIVE USE MODE */

    com_lwrt_ip(0x20, 40000); /* N1 OBJECT POINT PRE-BUFFER = 40000 WRITE */
    com_lwrt_ip(0x24, 20000); /* N2 OBJECT POINT PRE-BUFFER = 20000 WRITE */
    com_lwrt_ip(0x28, 0); /* N3 OBJECT POINT PRE-BUFFER = 0 WRITE */
    com_lwrt_ip(0x2c, 0); /* N4 OBJECT POINT PRE-BUFFER = 0 WRITE */
    com_lwrt_ip(0x0c, 48000); /* DENOMINATOR PRE-BUFFER = 48000 WRITE */
    /* 关于分母值 D 的计算方法，详见第 10 章 10-1 c) 小节 */
    com_wrt_ip(0x02); /* 启动直线补间 */
    com_lwrt_nl(0x28, 48000); /* N1 4800 PULSE SIGNAL INDEX DRIVE START */
    /* 作为补间驱动的脉冲发生器轴，启动第 1 轴 */
    /* 此处的 48,000，仅为最后的直线补间驱动所需的脉冲数 */
    /* 要保证此脉冲的数字足够大，足够用于减速停止 */
    /* 速度指令脉冲数的计算方法，详见 10 章 10-1 小节 */

    while ((inp(HEAD_ADDR+STS1_IP) & 0x01) != 0x01); /* BUSY_IP=1 WAIT */

    com_lwrt_ip(0x14, -10000); /* X START POINT PRE-BUFFER = -10000 WRITE */
    com_lwrt_ip(0x1c, 20000); /* Y START POINT PRE-BUFFER = 20000 WRITE */
    com_lwrt_ip(0x10, -10000); /* X OBJECT POINT PRE-BUFFER = -10000 WRITE */
    com_lwrt_ip(0x18, -20000); /* Y OBJECT POINT PRE-BUFFER = -20000 WRITE */
    com_wrt_ip(0x05); /* 顺时针有周速补正圆弧补间请求 */
    /* 連続補間ドライブ要求となる */

    while ((inp(HEAD_ADDR+STS1_IP) & 0x02) != 0x02); /* SDREQ=1 WAIT */
    while ((inp(HEAD_ADDR+STS1_IP) & 0x02) != 0x00); /* SDREQ=0 WAIT */
    /* 等待直线补间驱动完成后，启动顺时针圆弧补间 */

    com_lwrt_ip(0x20, -40000); /* N1 OBJECT POINT PRE-BUFFER = -40000 WRITE */
    com_lwrt_ip(0x24, 20000); /* N2 OBJECT POINT PRE-BUFFER = 20000 WRITE */
    com_lwrt_ip(0x0c, 48000); /* DENOMINATOR PRE-BUFFER = 48000 WRITE */
    /* 分母 D 的算出方法，详见第 10 章 10-1 c) 小节 */
    com_wrt_ip(0x02); /* 直线补间请求 */
    /* 将被视为连续补间驱动请求 */

    outp(HEAD_ADDR+MODEC_IP, 0x0b); /* PG AUTO STOP MODE,SIGNAL INDEX DRIVE USE MODE */
    /* LAST DRIVE REQ ON */

    end_chk_ip(); /* 确认连续补间驱动完成 */

    outp(HEAD_ADDR+MODEC_IP, 0x08); /* SIGNAL INDEX DRIVE USE MODE, LAST DRIVE REQ OFF */
}

```

10.（驅動所需的）脈衝數的算出方法

執行直線補間、順時針圓弧補間、逆時針圓弧補間時，如果脈衝發生器產後的脈衝數不足，則馬達將無法到達預期位置。

本章說明所需的脈衝數的算出方法。

10-1 直線補間驅動的脈衝數

執行直線補間操作時，應設定目標位置（N1 Obj、N2 Obj、N3 Obj、N4 Obj），並設定輸出比的分母D。D必須滿足以下條件：

$D \geq |N1\ Obj|、|N2\ Obj|、|N3\ Obj|、|N4\ Obj|$ 的最大值

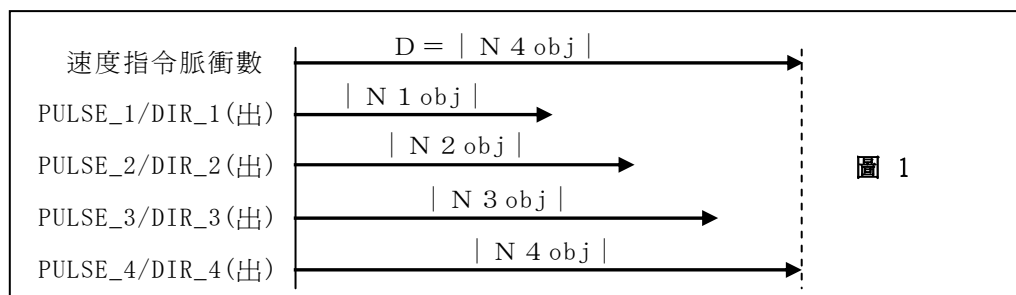
直線補間所需的脈衝數，即是分母D的值。（見第3章3-1，12章12-3小節）

另，分母D的設定，可根據實際情況做如下例子所示的設定：

a)使得補間速度最大化的場合

分母D設定為 $|N1\ Obj|、|N2\ Obj|、|N3\ Obj|、|N4\ Obj|$ 中的最大值。

例. $|N4\ Obj| > |N3\ Obj| > |N2\ Obj| > |N1\ Obj|$ 時：

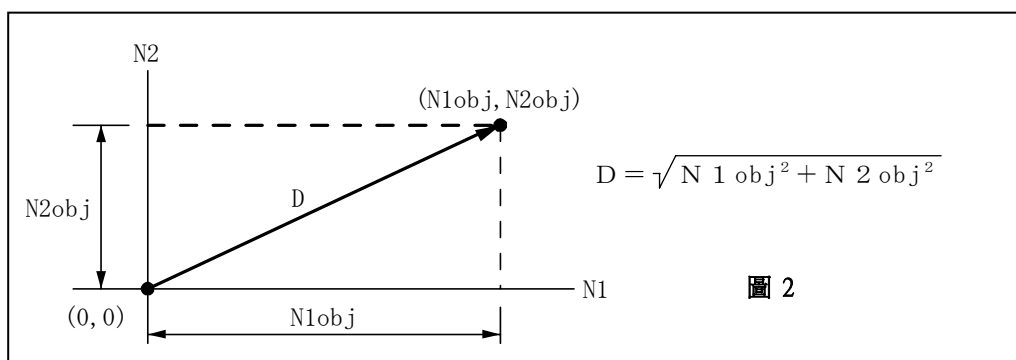


b)無論補間驅動為哪個方向，合成速度都相同的情況（譯注：指命令中指定的速度頻率數據也一樣的情況）

可將分母D設為如下的值。（僅在2軸、3軸直角坐標補間時）

$$D = \sqrt{N1\ Obj^2 + N2\ Obj^2 + N3\ Obj^2 + N4\ Obj^2}$$

例. 第1軸、第2軸直角坐標的情況下：



c) 從圓弧補間過渡到直線補間的連續補間（或相反），實現合成速度的連續性時：

無圓周速補償的情形，將分母D設定為 $|N1_obj|$ 、 $|N2_obj|$ 、 $|N3_obj|$ 、 $|N4_obj|$ 的最大值。

有圓周速補償時，分母D應設定為以下值：

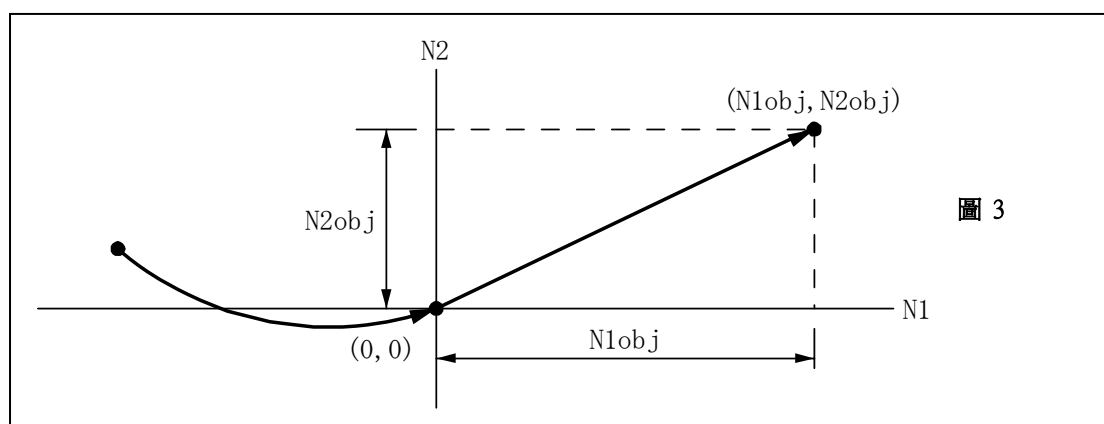
$$D = |NLong_obj| + |NShort_obj| \times 0.4$$

此處的NLong_obj和NShort_obj指的是：

NLong_obj：長軸的目標位置坐標

NShort_obj：短軸的目標位置坐標

例. 第1軸、第2軸成直角坐標，從逆時針圓弧補間過渡到直線補間時（當 $N1_obj > N2_obj$ 時）



第1軸和第2軸成直角坐標，從直線補間過渡到逆時針圓弧補間時（當 $N1_obj > N2_obj$ 時）

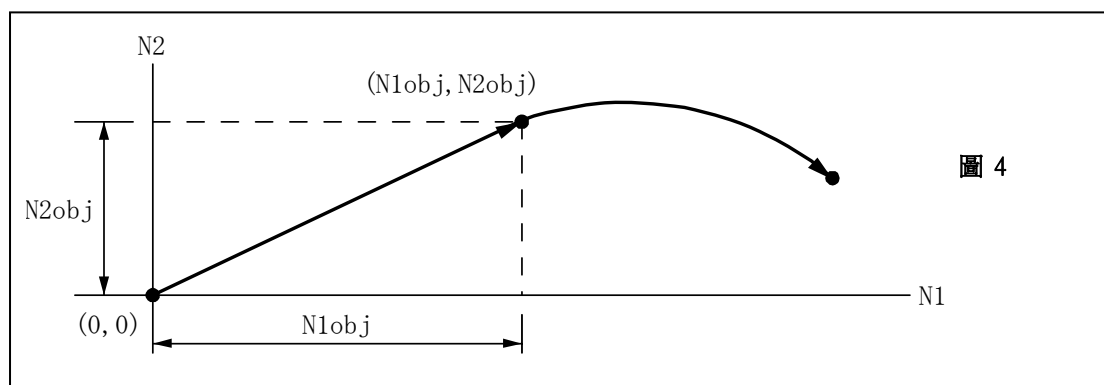


圖 3、4 中，圓弧補間為無圓周速補償時，D 設為 $N1obj$ 、圓弧補間為圓周速補償圓弧補間時，將 D 設定為 $N1obj + N2obj \times 0.4$ ，即可保證合成速度的連續性。

10-2 圓弧補間驅動的脈衝數（無圓周速補償時）

圓弧補間的脈衝發生器所需的脈衝數，為將各象限劃為 45°區間，並計算完成各個區間所需的脈衝數，再將這些脈衝數相加所得到的值。

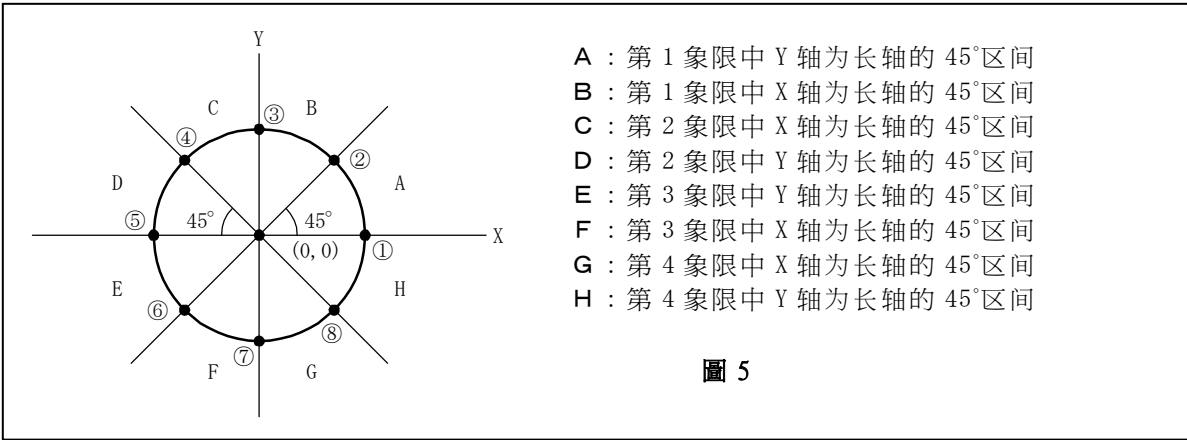


圖 5

圖5中①~⑧各點的坐標如下。此處所謂的r為，中心坐標為（0，0），開始坐標為（Xstr, Ystr）時的圓的半徑。

$$r = \sqrt{X_{str}^2 + Y_{str}^2}$$

$$\begin{aligned} \textcircled{1} &= (r, 0) \\ \textcircled{2} &= (r / \sqrt{2}, r / \sqrt{2}) \\ \textcircled{3} &= (0, r) \\ \textcircled{4} &= (-r / \sqrt{2}, r / \sqrt{2}) \\ \textcircled{5} &= (-r, 0) \\ \textcircled{6} &= (-r / \sqrt{2}, -r / \sqrt{2}) \\ \textcircled{7} &= (0, -r) \\ \textcircled{8} &= (r / \sqrt{2}, -r / \sqrt{2}) \end{aligned}$$

各個45°區間所需的脈衝數，如表1所示。（參見圖6）

表 1

區間	X 軸	Y 軸
A, D, E, H	$(1 - 1 / \sqrt{2}) r$ [短軸]	$r / \sqrt{2}$ [長軸]
B, C, F, G	$r / \sqrt{2}$ [長軸]	$(1 - 1 / \sqrt{2}) r$ [短軸]

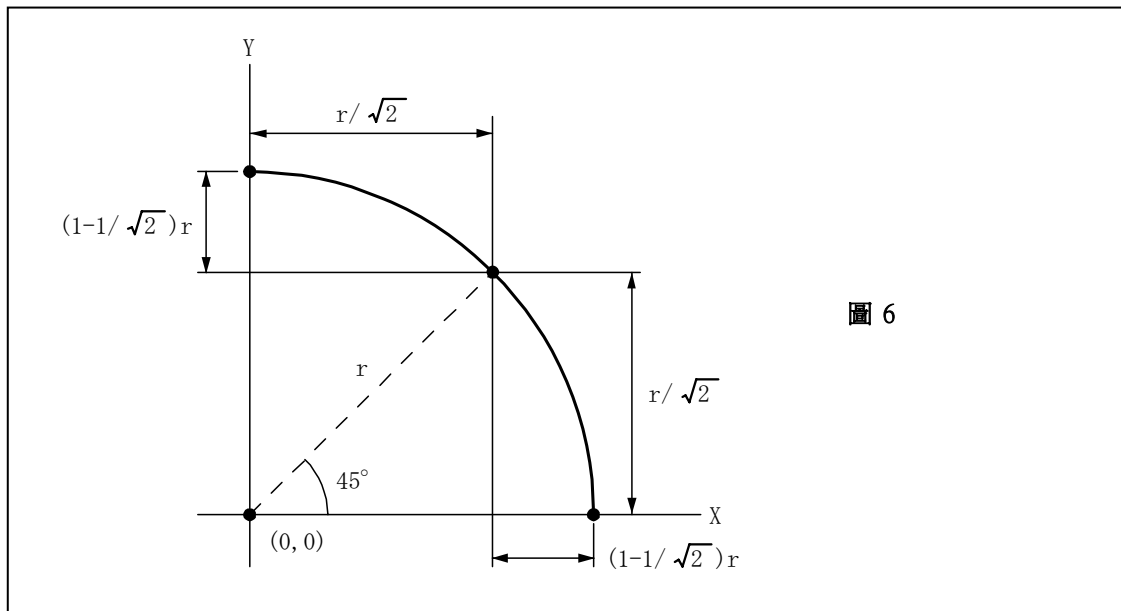


圖 6

小於 45° 的區間所需的脈衝數，則為表2所示。此處的2點的坐標，各為 $P_1=(X_1, Y_1)$ ， $P_2=(X_2, Y_2)$ 。

(見圖7)

表 2

區間	X 軸	Y 軸
A, D, E, H	$ X_1 - X_2 $ [短軸]	$ Y_1 - Y_2 $ [長軸]
B, C, F, G	$ X_1 - X_2 $ [長軸]	$ Y_1 - Y_2 $ [短軸]

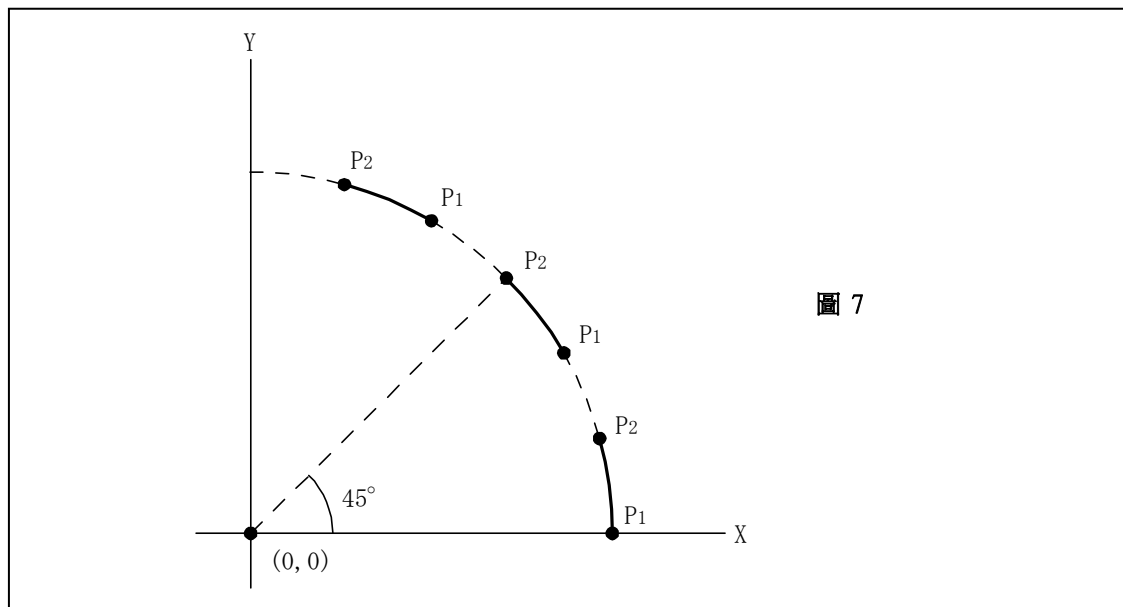


圖 7

最後，將A~H區間中的所有長軸所需脈衝數相加，即可得到總的所需脈衝數。

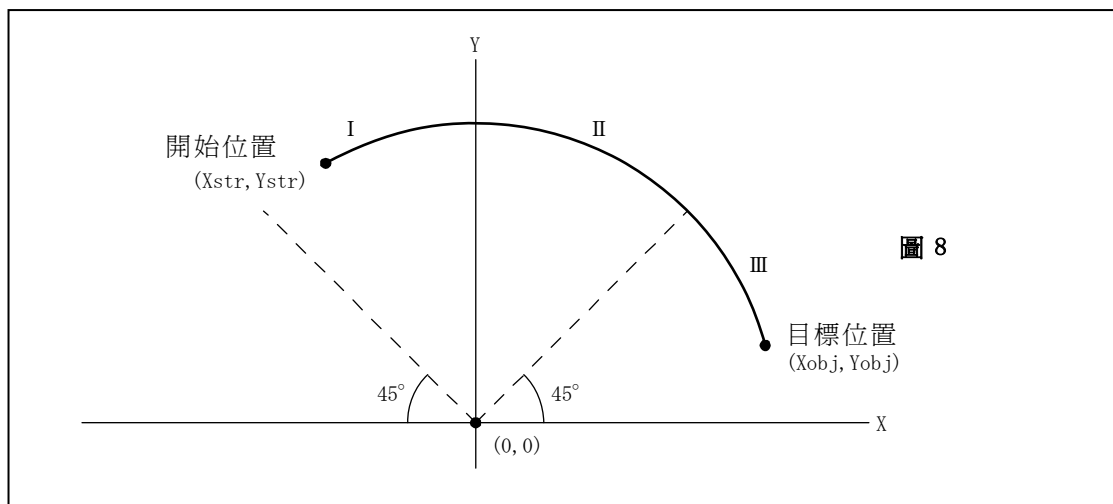
例. 圖8所示的順時針圓弧補間操作中，進行無圓周速補償圓弧補間時，所需的脈衝數為：

- I X軸為長軸，所需的脈衝數： $P_I = 0 - X_{str}$
- II X軸為長軸，所需的脈衝數： $P_{II} = r / \sqrt{2}$
- III Y軸為長軸，所需的脈衝數： $P_{III} = r / \sqrt{2} - Y_{obj}$

總的所需脈衝數則爲：

$$P_I + P_{II} + P_{III} = \sqrt{2} r - X_{str} - Y_{obj}$$

此處的 r 爲，所畫的圓弧的半徑， $r = \sqrt{X_{str}^2 + Y_{str}^2}$



10-3 圓弧補間驅動的脈衝數（有圓周速補償時）

進行圓周速補償圓弧補間，且兩軸同時輸出脈衝時，輸出的脈衝頻率為脈衝發生器的輸入速度的5/7。所以，所需的脈衝數必須為實際所需的脈衝的7/5倍（1.4倍）。

也就是10-2圖5中所示的那樣，所需的脈衝數為僅長軸輸出脈衝時所需脈衝數加上兩軸同時輸出的脈衝數的5/7。如下式所示（參見表1，圖6）

$$\text{僅長軸輸出脈衝時所需的脈衝數（短軸不動）} = r / \sqrt{2} - (1 - 1 / \sqrt{2}) r$$

$$\text{兩軸同時動作時所需的脈衝數} = (1 - 1 / \sqrt{2}) r$$

$$\begin{aligned} \text{得到的總的脈衝數爲（最後結果）} &= (r / \sqrt{2} - (1 - 1 / \sqrt{2}) r) + (1 - 1 / \sqrt{2}) r \times 1.4 \\ &= (0.3 \sqrt{2} + 0.4) r \end{aligned}$$

此理論也適用於小於45°的區間。設2點的坐標為P1（X1,Y1），P2（X2,Y2）。得到的所需脈衝數為：
（參見表2，圖7）

. A、D、E、H區間（Y軸為長軸時）

$$\text{僅長軸輸出的脈衝數：} |Y1-Y2| - |X1-X2|$$

$$\text{2軸同時輸出的脈衝數：} |X1-X2|$$

$$\text{所需的總脈衝數：} |Y1-Y2| - |X1-X2| + |X1-X2| \times 1.4 = |Y1-Y2| + 0.4 |X1-X2|$$

. B、C、F、G區間（X軸為長軸時）

$$\text{僅長軸輸出的脈衝數：} |X1-X2| - |Y1-Y2|$$

$$\text{2軸同時輸出的脈衝數：} |Y1-Y2|$$

$$\text{所需的總脈衝數：} |X1-X2| - |Y1-Y2| + |Y1-Y2| \times 1.4 = |X1-X2| + 0.4 |Y1-Y2|$$

使用以上方法，分別計算A~H區間所需的脈衝數，最後再相加合計即得所需的總脈衝數。

例. 10-2圖8所示的順時針圓弧補間中，所需的總的脈衝數為：

$$\text{I} \quad \dots\dots\dots \text{X軸爲長軸，所需的脈衝數：} -X_{str} + (r - Y_{str}) \times 0.4$$

$$\text{II} \quad \dots\dots\dots \text{X軸爲長軸，所需的脈衝數：} P_{II} = (0.3 \sqrt{2} + 0.4) r$$

$$\begin{aligned} \text{III} \quad \dots\dots\dots \text{Y軸爲長軸，所需的脈衝數：} \\ P_{III} = (r / \sqrt{2} - Y_{obj}) + (X_{obj} - r / \sqrt{2}) \times 0.4 \end{aligned}$$

總的所需的脈衝數為：

$$P_I + P_{II} + P_{III} = (0.6 \sqrt{2} + 0.8) r - (X_{str} + Y_{obj}) + (X_{obj} - Y_{str}) \times 0.4$$

此處的 r 爲，所畫的圓弧的半徑 $r = \sqrt{X_{str}^2 + Y_{str}^2}$

注9.

此外所示的計算方法，為目標點落在圓弧上的情形。目標點偏離圓弧時，要注意，（芯片）會對實際的目標坐標進行調整。

算出的半徑 r 及 $A^{\sim}H$ 的各個 45° 區間所需的脈衝數，產生餘數時，要將結果向上除整。

這樣可以防止，脈衝發生器產生的脈衝不足，而致使芯片無法驅動到預定的目標位置，BUSY_IP凍結為高電平狀態。也就是說，為防止芯片凍結，可適當多設這點脈衝數。

這個情況下，可設定為PG Auto Stop Mode，這樣，補間操作完成時，脈衝發生器也將被自動停止，減輕了用戶程序的負擔。（見第9章9-1，13章13-4小節）

11. 連續補間時的注意事項

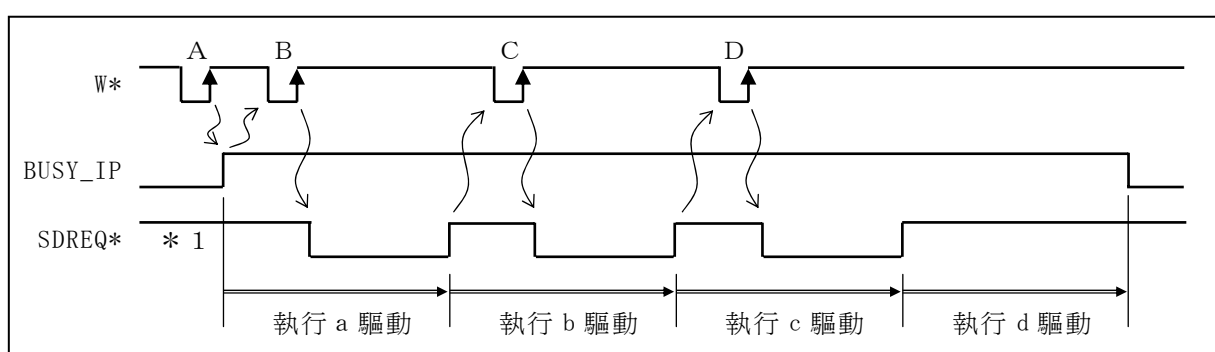
使用MCD451時，如果在芯片正在執行一個補間驅動時，往芯片中寫入下一個補間命令，則該操作將被識別為連續補間請求。

本章說明連續補間中所需注意的事項。

11-1 連續補間驅動的執行步驟

MCD451中，使用連續補間功能時，必須在補間驅動正在執行中（BUSY_IP為高電平，BUSY_IP Bit為1）時才有效，故應當遵守一定的握手操作，即在根據狀態把握讀寫命令的時機。

以下、闡述連續補間的實行步驟。（詳見第9章9-3，4，5小節）



A：設定執行a驅動所需的各種數據及寫入補間命令（立即開始補間驅動）

B：確認a驅動正在執行（通過讀取IP Status-1(A) Read Port並確認其中的BUSY_IP Bit為1，或確認BUSY_IP為高電平）後，設定b驅動所需的各種數據及寫入補間命令（芯片識別為連續補間請求，IP Status-1(A) Read Port中的Bit SDREQ被置為1，引腳SDREQ*變為低電平）；

C：確認a驅動已完成，b驅動已開始（IP Status-1(A) Read Port中的Bit SDREQ回復到0，或SDREQ*引腳為高阻態）後，設定執行c驅動所需的各種數據及寫入驅動命令（芯片識別為連續補間請求，IP Status-1(A) Read Port中的Bit SDREQ重新被置為1，引腳SDREQ*被也置為低電平）

D：b驅動完成後，重複C的步驟，啟動d連續補間驅動。

*1. 此處高電平，是指輸出為OFF的三態狀態。

11-2 驅動類型引起的限制

MCD451的連續補間功能，是通過在執行補間操作時寫入下一個補間命令完成的，故一個連續補間操作中，可進行的補間驅動的數量沒有限制。

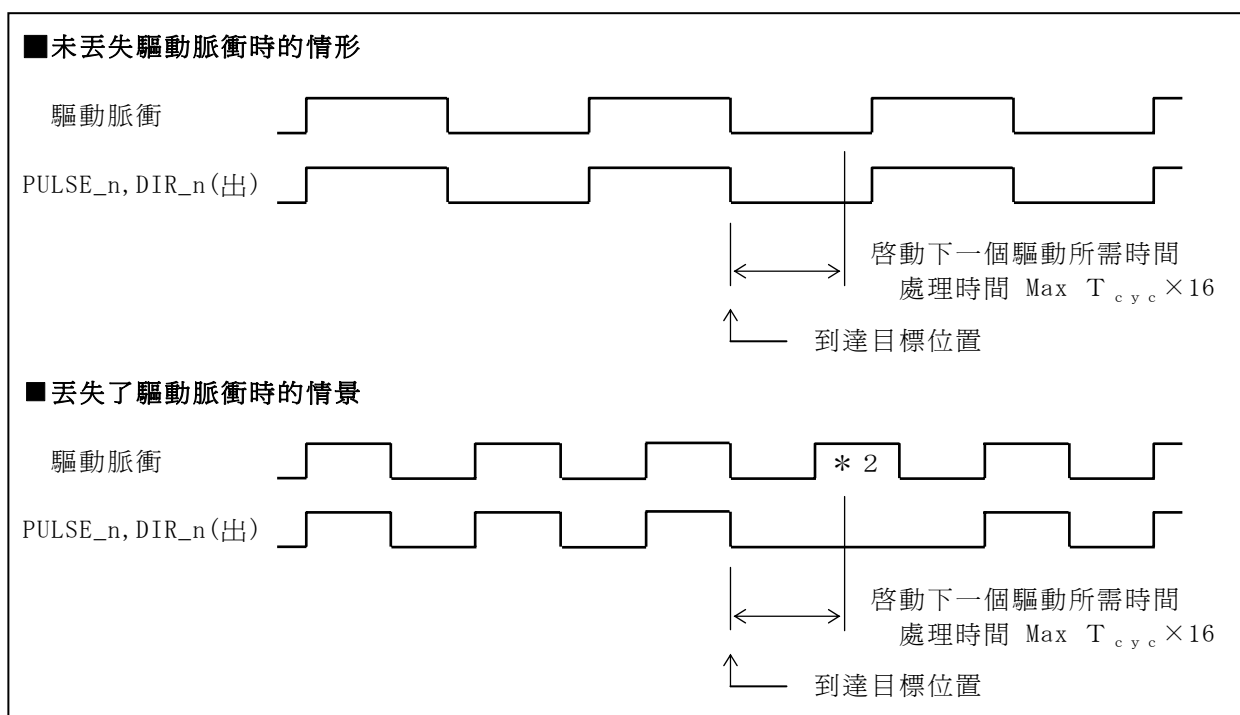
但是，Endless直線補間時，因為此補間操作不會自動停止，故必須使用命令Interoperation Stop命令停止此操作。補間操作停止後，已接受的連續補間請求變為無效。

故必須使用Interoperation Stop停止的Endless直線補間除了作為最後一個連續補間命令外，無法參與連續補間操作，故實際上在連續補間中可用的補間驅動類型為：

- . 直線補間
- . 順時針圓弧補間
- . 逆時針圓弧補間

11-3 丟失的驅動脈衝

MCD451在連續補間操作時，完成前一次補間驅動後、開始後一個補間驅動前，有一個最大 $T_{cyc} \times 16$ 的時間窗口。是故如果速度指令很高，它的一個脈衝週期中的低電平部分的時間小於 $T_{cyc} \times 16$ 時，啟動下一個驅動前，可能脈衝發生器輸出的脈衝已處於下一個脈衝的高電平部分。這時，脈衝發生器產生的脈衝將會有1個脈衝丟失。



*2. 在上述的 $T_{cyc} \times 16$ 時間內處於高電平的脈衝，將會丟失掉，不會為補間運算所用。

12. 補間驅動時的注意事項

本章講述MCD451在執行直線補間驅動、Endless直線補間驅動、順時針補間、逆時針圓弧補間驅動時的注意事項。

12-1 Range Data的設定範圍的限制

MCD451能輸出的最高驅動脈衝頻率，獨立模式為 $F_{clk}/4$ ，補間模式為 $F_{clk}/8$ 。

是故，如果脈衝發生器產生的脈衝的頻率大於 $F_{clk}/8$ ，則芯片發生誤動作，將無法正常工作。

要使速度頻率不超過 $F_{clk}/8$ ，必須保證Range Data ≥ 2 。

12-2 關於直線補間時目標位置的設定

執行直線補間驅動、Endless直線補間驅動，須以當前位置為（0，0，0，0），設定第1軸~第4軸的現在點的相對坐標，即目標位置坐標（N1 Obj, N2 Obj, N3 Obj, N4 Obj）。將目標位置設定為80000000H，則超出設定範圍，從而無法進行有效的補間運算（譯注：也就是說，最小的目標位置為80000001H，也就是7FFFFFFFH的負值，80000000H為7FFFFFFFH的負值再減1）

12-3 關於直線補間時的分母值

直線補間和Endless直線補間時，要預先第1軸~第4軸的目標坐標（N1 Obj, N2 Obj, N3 Obj, N4 Obj），及分母D。

將目標位置設為（N1 Obj, N2 Obj, N3 Obj, N4 Obj）時，分母D的設定條件為：

$D \geq |N1\ Obj|、|N2\ Obj|、|N3\ Obj|、|N4\ Obj|$ 的最大值

當 $D < |N1\ Obj|、|N2\ Obj|、|N3\ Obj|、|N4\ Obj|$ 時，因為有的軸的分子比分母D大，故理論上將不會執行補間操作。此時IP Status-1(A) Read Port中的DENERR Bit被設定為1，芯片不會開始補間操作。

12-4 關於圓弧補間時的開始位置的設定

圓弧補間操作時，須以圓心以（0，0）點，設定開始點的坐標（Xstr, Ystr）[譯注：一旦開始點確定，則圓心坐標也就確定了；開始點也即現在所在位置，相對於圓心的坐標；故圓心相對於開始點的坐標為（-Xstr, -Ystr）]。如果將開始點設定為（0，0），則出現圓心坐標等於目標點的情況，理論上將無法畫出一個圓來。如果將開始點設類似開（80000000H, y）或（x, 80000000H）的點，則開始點的坐標超出設定範圍，芯片將無法開始圓弧補間運算。

此時，IP Status-1(A) Read Port中的STPERR Bit被置為1，芯片不會執行補間操作。[譯注：BUSY_IP*將為低電平，馬上又因數據錯回到低電平]

12-5 關於圓弧補間時的目標位置的設定

圓弧補間操作時，須以圓心[譯注：圓心位置只決定於開始點坐標]為 $(0, 0)$ ，設定目標位置的坐標 (X_{obj}, Y_{obj}) 。當目標位置的坐標為 $(0, 0)$ 時，理論上將無法畫出一個圓來。如果將目標位置的坐標設定為類似於 $(80000000H, y)$ 或 $(x, 80000000H)$ 的點，則將因為目標點坐標超出芯片的設定範圍，無法開始實際的圓補間操作。

此時，IP Status-1(A) Read Port中的OBPERR Bit將被置為1，芯片不會執行補間操作。[譯注：BUSY_IP*將為低電平，馬上又因數據錯回到低電平]

12-6 關於360°圓弧補間（也就是畫一個圓）

做360°圓補間時，因為開始點坐標＝目標點坐標，圓弧補間操作將啟動，即符合“現在點坐標即＝目標點坐標”這個驅動終了條件。

MCD451會自動將這種情況識別為360°圓弧也即圓補間操作請求。故，需做360°圓補間時，無須人為將終點坐標做一定的挪移，或將其人為拆分為兩段圓弧。

12-7 關於圓補間時，指定的目標位置無法檢測到的情形

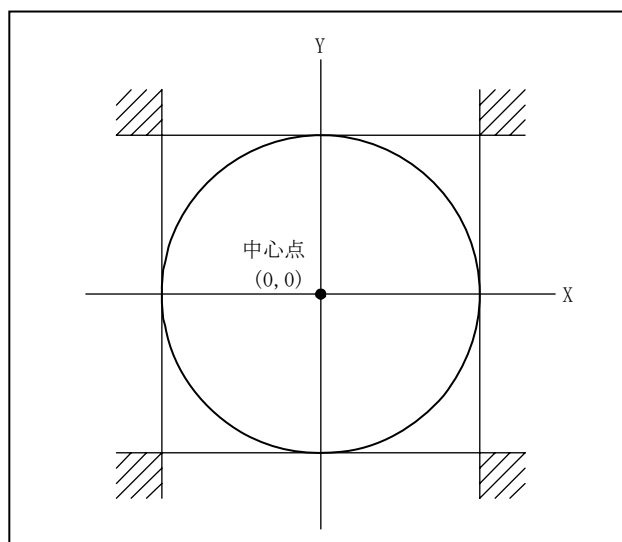
圓弧補間時，判定是否到達目標位置的依據是，以中心點坐標為 $(0, 0)$ 時，現在點的坐標和目標點的坐標在同一個象限上，且符合以下兩個條件中的一個：

.X Now Point Register (現在點的X軸坐標) = X Object Point Register (目標點位置的X軸坐標)

.Y Now Point Register (現在點的Y軸坐標) = Y Object Point Register (目標點位置的Y軸坐標)

但是，如果將目標點坐標設置為下圖陰影部分中的點時，上述條件將永遠無法達到。芯片將無法自動停止圓弧補間操作。

停止此補間操作，必須使用Interoperation Stop命令中止當前補間驅動。

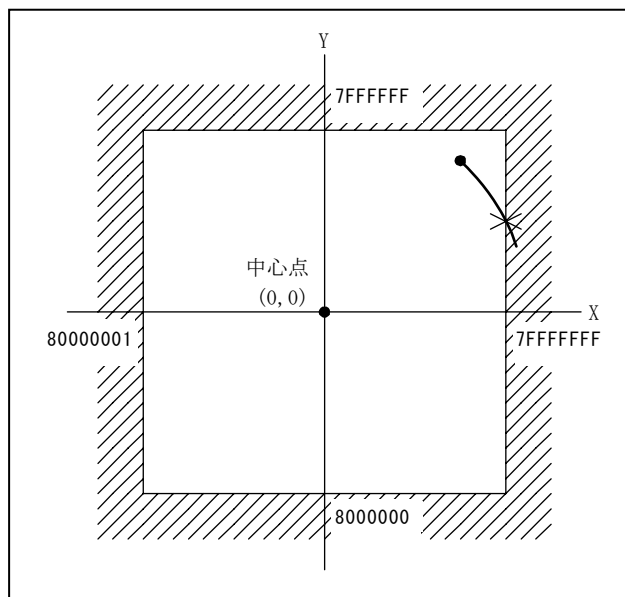


12-8 關於當前點坐標溢出時的情形

圓弧補間時，要以要畫的圓弧的圓心為中心，設定開始位置坐標 (Xstr, Ystr) 和目標位置 (Xobj, Yobj)。

圓弧補間開始時，會將開始位置的坐標值 (Xstr, Ystr) 加載到 X/Y Now Point Register 中，此後，每輸出一個脈衝 [譯注：在相應的坐標軸上的脈衝]，X/Y Now Point Register 的值將做 +1 或 -1 調整。

如果為做的補間操作的圓弧軌跡進入了以下的陰影部分，則 X/Y Now Point Register 將會超出 -2147783647 ~ +214783647 (800000001H ~ 7FFFFFFFH)。也即現在點位置坐標溢出，此後芯片的動作將無法保證。



12-9 關於圓補間時目標位置偏移（有誤差）時的處理

圓弧補間的圓弧半徑 r ，是以圓心為參照原點 $(0, 0)$ 時，設定的開始點坐標 (X_{str}, Y_{str}) 決定的， $r = \sqrt{X_{str}^2 + Y_{str}^2}$

也就是說，MCD451在進行圓弧補間時，就算是目標位置不落在圓周上，也會開始圓弧補間。

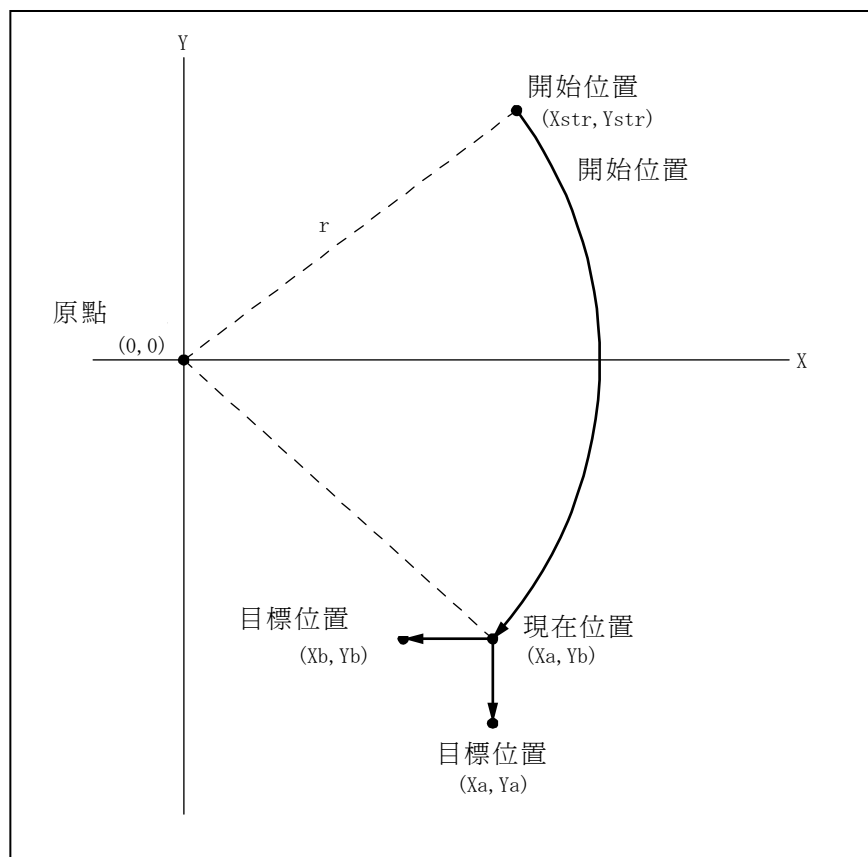
當目標點偏離圓周時，MCD451根據以下的原則將目標點坐標“挪到”“正確的位置”上。

①以圓心為坐標 $(0, 0)$ ，現在位置的坐標和目標位置的坐標在同一個象限，且符合以下條件之一時：

. $X_{Now\ Point\ Register}$ （現在位置的X軸坐標）= $X_{Object\ Point\ Register}$ （目標點的X軸坐標）

. $Y_{Now\ Point\ Register}$ （現在位置的Y軸坐標）= $Y_{Object\ Point\ Register}$ （目標點的Y軸坐標）

②到達以上條件時，將目標坐標點“挪到”當前位置。



此處的例子為順時針圓弧補間，目標位置在第4象限的情形。此處要注意的是，如果開始位置的坐標和目標位置的坐標在同一象限內，且X軸或Y軸的開始位置坐標和目標位置的X或Y軸坐標相等的情況。

13. 補充說明

13-1 標準輸入時鐘脈衝的選擇

ICLK引腳輸入的基準時針信號（Fclk）將會影響MCD451補間回路的以下內容。本手冊中，提到的Tcyc是指標準輸入時針信號（Fclk）的週期；

- ①補間脈衝的高電平的部分和低電平部分的時間長度都要大於Tcycx4。使用同一芯片中軸做為脈衝發生器時，為了使處理能夠同步，就當設定Range Data ≥ 2 ；此外，和其他的通過MCB9~MCB0互聯的MCD451間做補間操作時，就算是基準時鐘頻率一致，因為輸出緩衝延遲關係，也芯片將可能無法達到以上處理性能。此時，應當保證Range Data ≥ 3 ；
- ②寫入數據的命令，從Command Write開始，到寫入的數據裝載到內部寄存器所花費的時間最多為Tcycx8；故在執行了Command Write動作後，應當在Tcycx8長度的時間窗口內，避免再次往IP Data1, 2, 3, 4 (IP Data-H, L)寫入數據。這個時間窗口中往IP Data1, 2, 3, 4 (IP data H, L)中寫入數據，芯片此後的動作將無法保證；
- ③從芯片中讀出數據的命令，從Command Write開始，到內部數據裝載到讀緩衝所需要的時間最多為Tcycx8；故在執行了Command Write動作後，應當避免在Tcycx8長度的時間窗口內，讀出數據。否則讀出的數據的正確性無法保證；
- ④所有的命令，都需要最大Tcycx8的處理時間，故往IP Command Write Port中寫入命令後，在此後的Tcycx8的時間窗口內，應避免再次寫入命令。否則此後芯片的動作將無法保證。

以上的②③④的意思就是，“和CPU的I/O存取時間相比，如果HCM341的命令處理時間較長，則必須在程序[譯注：指不間斷對MCD451進行存取的情況]中加入人工的延遲”。要避免這個問題，可以選擇合適的Fclk，使得CPU的I/O存取時間 $\geq$ Tcycx8。

如果無法做到上述條件，則應當在程序中加入適當的延遲、或在寫入命令前確認IP Status-1(A) Read Port中的CMBSY_IP Bit為0，或輸出引CMBSY_OR*為低電平。

綜上的述，我們總是期望Fclk能夠盡可能設置得大一些（但最大只能為40MHz）。

13-2 收到RESET信號時的芯片初始化

本小節講述芯片收到RESET*信號時的動作。要使MCD451正常開始工作，應當在上電時，在芯片的RESET*引腳上輸入持續時間最少為400ns的低電平信號。

在接受到RESET信號前，MCD451的各輸出引腳的信號的輸出水平都是不確定的。

a) 收到有效的RESET*低電平信號後，補間回路中的各個寫端口（Write Port，用於寫的端口），都被預設為00H。

. IP Mode-A Write Port.....單芯片模式/補間用脈衝發生器：其他軸的MCD451的軸/所有軸為“獨立模式”

. IP Mode-B Write Port.....X軸指定：未指定/Y軸指定：未指定

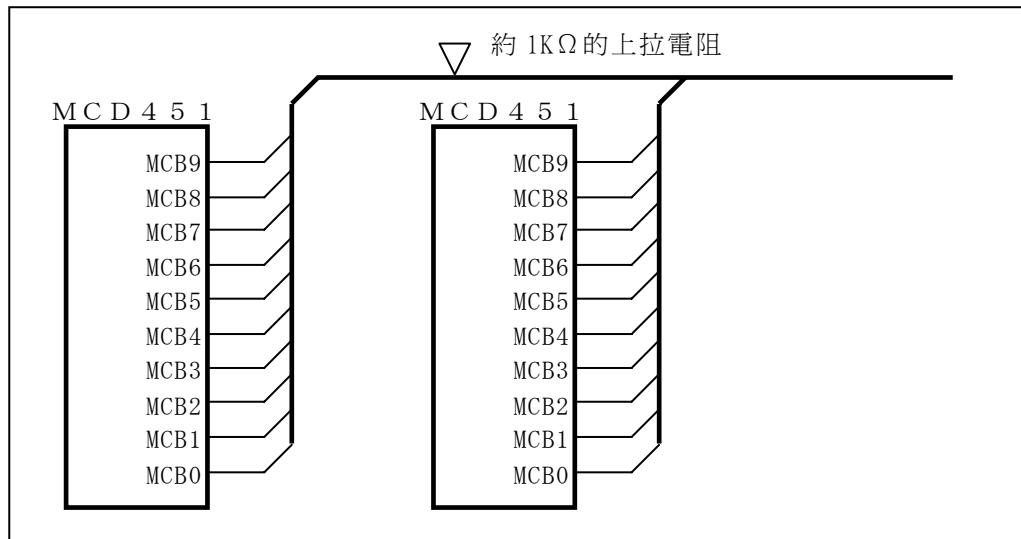
. IP Mode-C Write Port.....PG Auto Stop Mode：禁用/Interrupt Out Enable Mode：禁用/Signal Index Drive Mode：禁用/Last Drive Request：禁用

b) 收到有效的RESET*低電平信號後，補間加路中的各種參數

• DENOMINATOR PRE-BUFFER	0	(00000000H)
• DENOMINATOR REGISTER	0	(00000000H)
• DENOMINATOR COUNTER	0	(00000000H)
• X OBJECT POINT PRE-BUFFER	0	(00000000H)
• X OBJECT POINT REGISTER	0	(00000000H)
• X NOW POINT REGISTER	0	(00000000H)
• X START POINT PRE-BUFFER	0	(00000000H)
• X START POINT REGISTER	0	(00000000H)
• Y OBJECT POINT PRE-BUFFER	0	(00000000H)
• Y OBJECT POINT REGISTER	0	(00000000H)
• Y NOW POINT REGISTER	0	(00000000H)
• Y START POINT PRE-BUFFER	0	(00000000H)
• Y START POINT REGISTER	0	(00000000H)
• N1 OBJECT POINT PRE-BUFFER	0	(00000000H)
• N1 OBJECT POINT REGISTER	0	(00000000H)
• N1 NOW POINT REGISTER	0	(00000000H)
• N2 OBJECT POINT PRE-BUFFER	0	(00000000H)
• N2 OBJECT POINT REGISTER	0	(00000000H)
• N2 NOW POINT REGISTER	0	(00000000H)
• N3 OBJECT POINT PRE-BUFFER	0	(00000000H)
• N3 OBJECT POINT REGISTER	0	(00000000H)
• N3 NOW POINT REGISTER	0	(00000000H)
• N4 OBJECT POINT PRE-BUFFER	0	(00000000H)
• N4 OBJECT POINT REGISTER	0	(00000000H)
• N4 NOW POINT REGISTER	0	(00000000H)

13-3 5軸以上的補間操作

多個芯片通過MCB9~MCB0互聯時，MCD451可實現最大4xn軸間的直線補間及4xn軸中任意2軸間的圓弧補間驅動。



同一MCD451內的軸，自然使用同一個基準時鐘，和其他MCD451互聯時，原則上也應保證所有互聯的芯片使用的基準時鐘的頻率相同。

此外，MCB9信號用於輸出脈衝信號。當輸出頻率很高時，可能將無法保證輸出的波形為矩形波。此時，接在MCB9信號引腳上的下拉電阻應比其它引線的小（但最小為290 Ω）。

以下為，在多個互聯的芯片間實現補間時，應遵守的規則：

- 1) 補間用的脈衝發生器的軸，應保證Range Data ≥ 3 ；
- 2) 直線補間時，所有協作芯片的Denominator Pre-Buffer（分母D）的值為相同的值；
- 3) 圓弧補間時，所有協作芯片的X/Y Object Point Pre-Buffer和X/Y Start Point Pre-Buffer都有相同的設置值。

13-4 关于不进行加减速操作的补间驱动

圓弧補間驅動或包含了圓弧補間的連續補間驅動中，必須先求得圓弧補間所需的脈衝數。

關於圓弧補間的算出方法，可參見第10章10-2，10-3小節；不過，但無須加減速驅動時，因為可以將High-Speed和Low-Speed設為相同後，直接可以利用Scan Drive驅動脈衝發生器，也就無須預先算出所須的脈衝數。

另，可將IP Mode-C Write Port中的“PG Auto Stop Mode”指定為有效，這樣，補間操作完成後，當BUSY_IP變為低電平，BUSY_IP Bit變為0時，芯片可自動停止補間脈衝發生器。

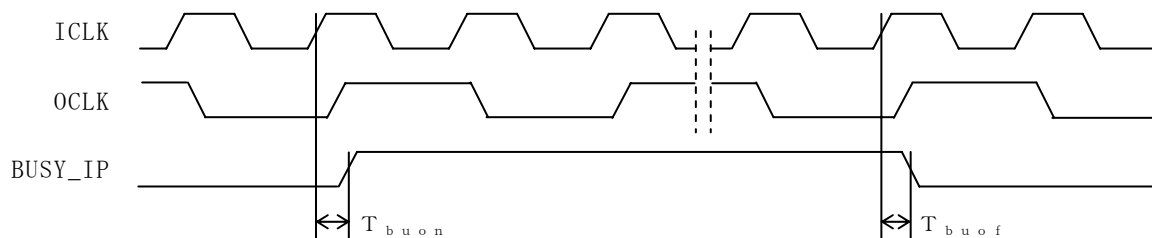
此方法對於直線補間、Endless直線補間，及含用各種補間的連續補間都有效。

14. 输入输出信号的时序

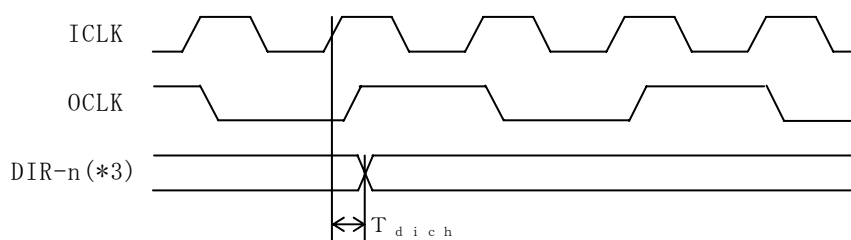
14-1 同步信号的输出时序

此處講述和時鐘信號（ICLK 基準輸入信號）同步的各種輸出信號和其相於基準輸入時鐘信號的時延。

• BUSY_IP 输出时序

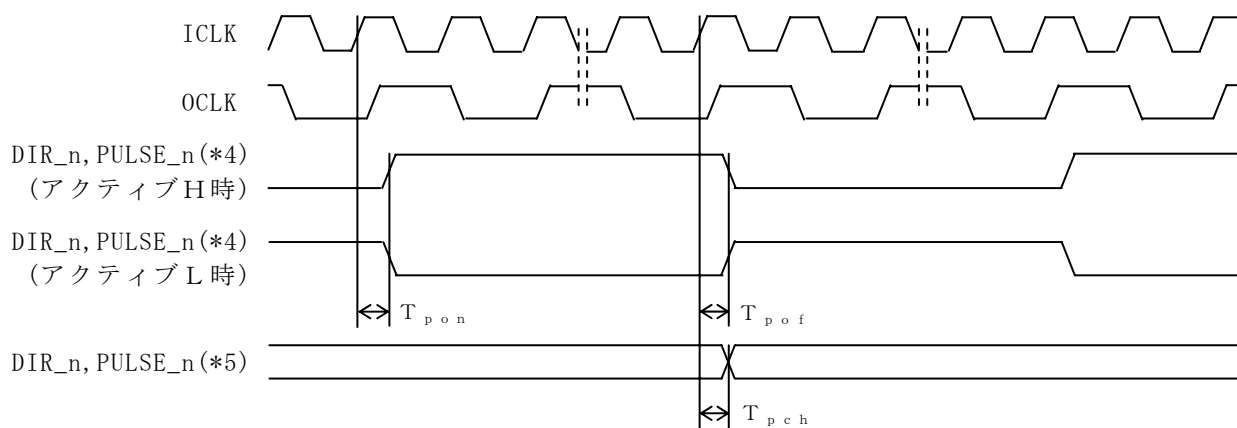


• DIR_n 信号的输出时序（補间驱动时）



* 3. DIR_n 只用来指示驱动方向的情景

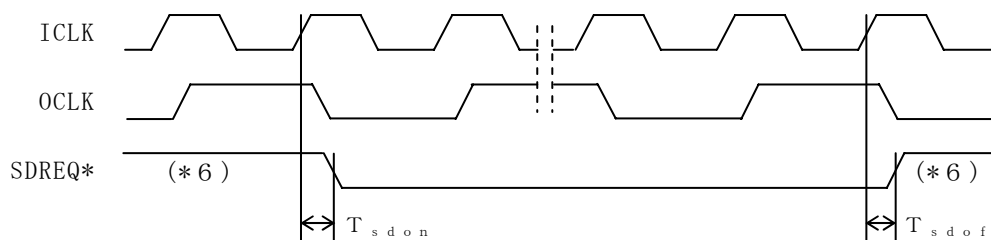
• DIR_n, PULSE_n 信号的输出时序（補间驱动时）



* 4. DIR_n 信號工作于雙脈衝方式時。

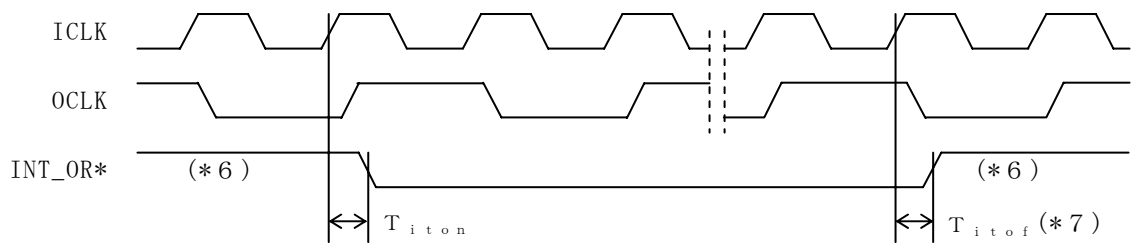
* 5. 輸出 2 相脈衝的情景

• SDREQ* 的输出时序



* 6. 此处所谓的高电平，指输出被关闭的高阻态

・INT_OR* 输出时序（补间驱动时）



- * 6. 此处所谓的高电平，指输出被关闭的高阻态
- * 7. 往 IP COMMAND WRITE PORT 中入补间命令时的时序

同步信号输出时序表

項目	記號	M I N	M A X
ICLK → BUSY_IP O N 輸出時延 *8	T_{buon}	3	19
ICLK → BUSY_IP O F F 輸出時延 *8	T_{buof}	3	20
ICLK → DIR_n 信號翻轉時延 *3	T_{dich}	4	21
ICLK → DIR_n, PULSE_n O N 輸出時延 *4	T_{pon}	5	24
ICLK → DIR_n, PULSE_n O F F 輸出時延 *4	T_{pof}	5	24
ICLK → DIR_n, PULSE_n 翻轉時延 *5	T_{pch}	5	24
ICLK → SDREQ* O N 輸出時延	T_{sdon}	4	16
ICLK → SDREQ* O F F 輸出時延	T_{sdoof}	4	21
ICLK → INT_OR* O N 輸出時延	T_{iton}	5	18
ICLK → INT_OR* O F F 輸出時延	T_{itof}	3	17

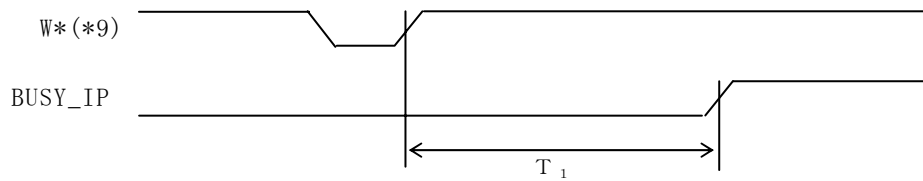
單位は nSec

- * 3. DIR_n 做為指示驅動方向的信號的情景；
- * 4. DIR_n 做为双脉冲输出的一个驱动脉冲的情景；
- * 5. 2 相脉冲输出方向时的时序；
- * 8. BUSY_IP 信号，也包括了补间模式中指定了的 IPBUSY_n 信号。

14-2 实动作时序

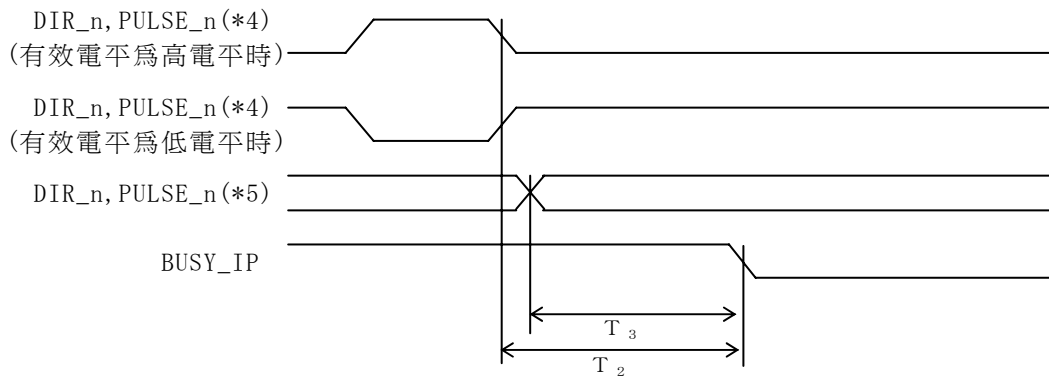
此處說明各種動作的輸入信號和輸出信號間的時序關係。

・開始補間驅動的時序



* 9. 往 IP COMMAND WRITE PORT 中写入补间驱动命令时。

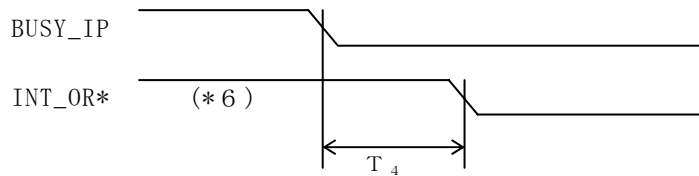
・補間驅動終了時序



* 4. DIR_n 做为双脉冲输出方式，输出的为其中一个脉冲时；

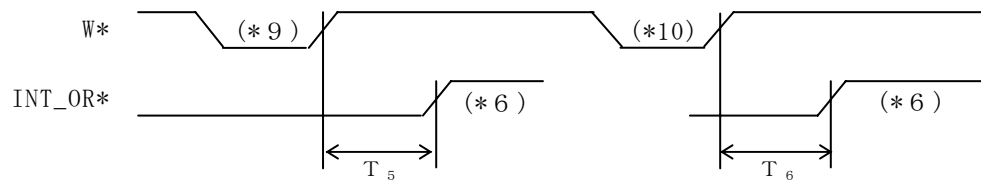
* 5. 2 相脈衝方式的情景

・INT_OR* 输出结果为 ON 的时序（補間驅動時）



* 6. 此处的高電平，是指輸出被關閉的高阻態。

・INT_OR* 输出结果为 OFF 的时序（補間驅動時）

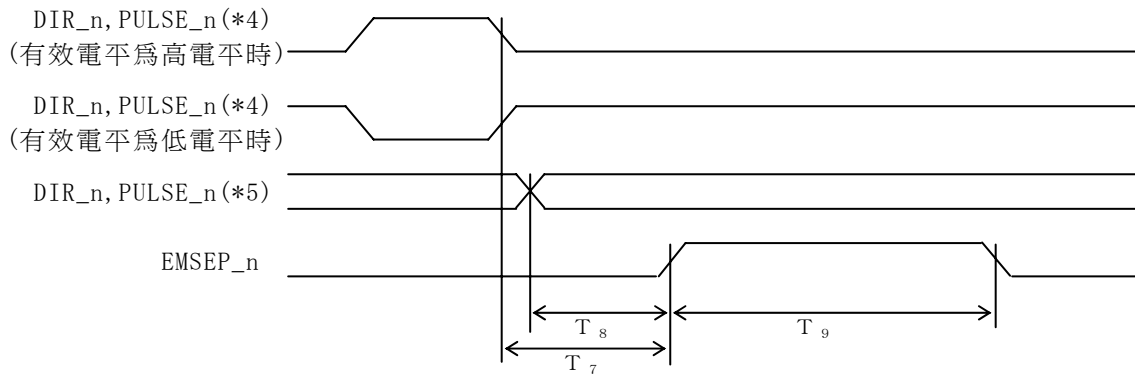


* 6. 此处的高电平，指输出被关闭的高阻态；

* 9. 往 IP COMMAND WRITE PORT 寫入補間驅動命令時；

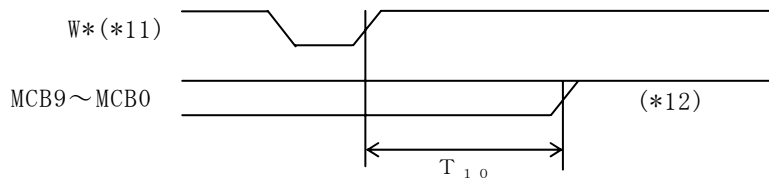
* 10. 写入 IP MODE-C WRITE PORT 解除 INTERRUPT OUT ENABLE MODE 時的情景。

• **EMSEP_n 输出时序**（补间驱动时）



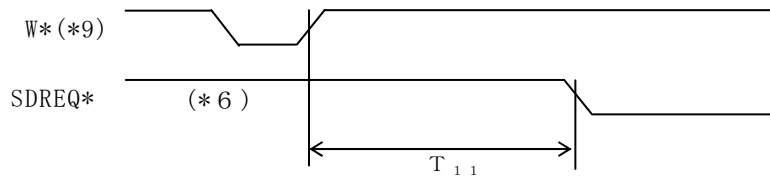
- * 4. 双脉冲输出方式中，D I R_n 做为其中的一路脉衝時；
- * 5. 2 相脉冲输出方式时的时序。

• **INITIAL CLEAR 命令执行时序**



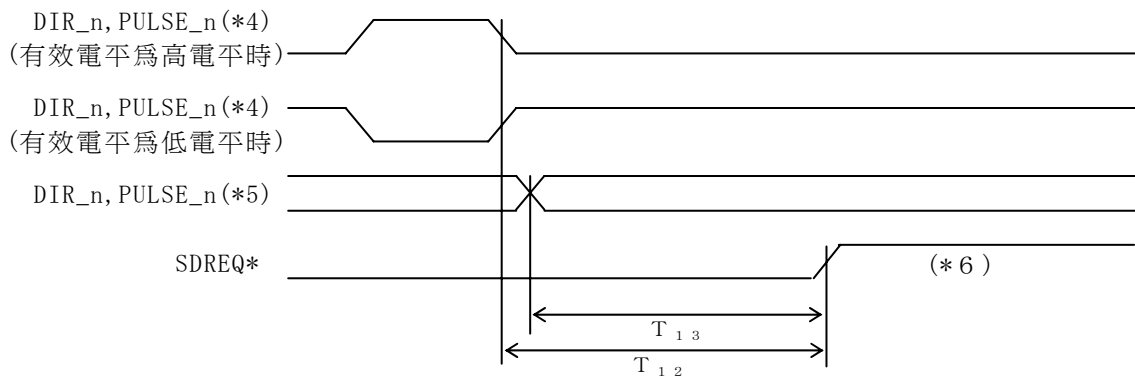
- * 11. 往 IP COMMAND WRITE PORT 中写入 INITIAL CLEAR 命令碼時；
- * 12. 此处的高电平，是指高阻态；

• **接受连续补间请求的时序**（只适用于 B U S Y_{I P} = H 時）



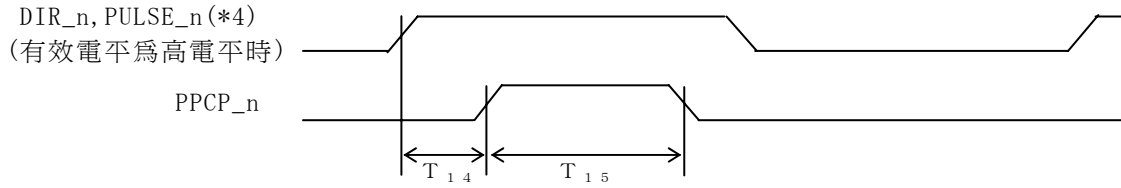
- * 6. 此处所谓的高电平，是指输出被关闭的高阻态；
- * 9. 往 IP COMMAND WRITE PORT 寫入補間命令時。

• **启动连续补间驱动的时序**



- * 4. D I R_n 信号，作为双脉冲输出方式下的一路脉衝時；
- * 5. 2 相脉冲输出方式时；
- * 6. 此处的高电平，是指输出被关闭的高阻态。

• **PPCP_n 输出时序**（補間驅動時）



* 4. 將 D I R_n 信号做为双脉冲输出中的一路脉冲时。

實動作時序表

項目	記號	M I N	M A X
寫入補間驅動命令 → BUSY_IP O N 的輸出時延 * 12	T ₁	$T_{cyc} \times 2$	$T_{cyc} \times 6 + 32$
DIR_n, PULSE_n 輸出 OFF → BUSY_IP 輸出 OFF 的輸出時延 * 4, * 12	T ₂	$T_{cyc} \times 2 - 21$	$T_{cyc} \times 2 + 15$ * 15
DIR_n, PULSE_n 最後一次的翻轉 → BUSY_IP 輸出 OFF 的輸出時延 * 5, * 12	T ₃	$T_{cyc} \times 2 - 21$	$T_{cyc} \times 2 + 15$ * 15
BUSY_IP 輸出 OFF → INT_OR* 輸出 ON 的輸出時延	T ₄	- 1 6	1 6
寫入補間驅動命令 → INT_OR* 輸出 OFF 的時延	T ₅	T_{cyc}	$T_{cyc} \times 5 + 35$
寫入 IP MODE-C WRITE PORT → INT_OR* 輸出 OFF 的輸出進延 * 10	T ₆	$T_{cyc} - 1$	$T_{cyc} \times 3 + 34$
DIR_n, PULSE_n 輸出 OFF → EMSEP_n 輸出 ON 的時延 * 4	T ₇	$- T_{cyc} \times 3 - 20$	* 14
DIR_n, PULSE_n 最後一次翻轉 → EMSEP_n 輸出 ON 的輸出時延 * 5	T ₈	$- T_{cyc} \times 3 - 20$	* 14
EMSEP_n 輸出脈衝的寬度 * 13	T ₉	$T_{cyc} \times 4 - 26$	$T_{cyc} \times 4 + 26$
寫入 INITIAL CLEAR 命令 → MCB9 ~ MCB0 變為高阻態的時延	T ₁₀	$T_{cyc} - 3$	$T_{cyc} \times 6 + 70$
寫入補間驅動命令 → SDREQ* 輸出 ON 的時延	T ₁₁	$T_{cyc} + 1$	$T_{cyc} \times 5 + 29$
DIR_n, PULSE_n 輸出 OFF → SDREQ* 輸出 OFF 的輸出時延 * 4	T ₁₂	$T_{cyc} \times 5 - 20$	$T_{cyc} \times 7 + 16$
DIR_n, PULSE_n 最後一次翻轉 → SDREQ* 輸出 OFF 的輸出時延 * 5	T ₁₃	$T_{cyc} \times 5 - 20$	$T_{cyc} \times 7 + 16$
DIR_n, PULSE_n 輸出 ON → PPCP_n 輸出 ON 的時延 * 4	T ₁₄	$T_{cyc} \times 2 - 21$	$T_{cyc} \times 2 + 15$
PPCP_n 脈衝輸出的寬度	T ₁₅	$T_{cyc} - 17$	$T_{cyc} + 17$

單位為 nSec

- * 4. D I R_n 信号做为双脉冲输出中的一路驱动脉冲时；
- * 5. 2 相脉冲输出方式时；
- * 10. 往 IP MODE-C WRITE PORT 写入数据以解除 INTERRUPT OUT ENABLE MODE 時的情景；
- * 12. 此處的 B U S Y_ I P 信号，也包括了补间模式中指定的 IPBUSY_n 信号；
- * 13. 同一个芯片内的軸做为补间运算用的脉冲发生器的情形；
- * 14. 将急停止输出的频率的周期作为 T，
 $T_7, T_8 = -T_{cyc} \times 3 + (T \div 2) + 25$ 是最大的近似值；
- * 15. 正常終了的情景。

